

Robots That Learn on the Fly Through Real-World Interaction

by

Gautham Vasan

A thesis submitted in partial fulfillment of the requirements for the degree of

Doctor of Philosophy

Department of Computing Science

University of Alberta

© Gautham Vasan, 2026

Abstract

A robot that learns from its own runtime experience can improve its behavior over its operational lifetime and adapt to conditions that were never anticipated. The prevailing practice in robot learning provides no such ability: practitioners train a control policy before deployment, from human-provided data or from simulated interaction, and hold it fixed afterward. When performance degrades after deployment, a human engineer may need to diagnose the failure, collect more data or refine the simulator, and retrain and redeploy the policy. Reinforcement learning (RL) from ongoing interaction could reduce reliance on humans for data collection, retraining, and redeployment.

Real-time reinforcement learning names the setting in which an agent senses, acts, and learns from ongoing interaction as wall-clock time advances for both the agent and the environment. Real-time RL on physical robots imposes three practical demands: (a) learning must operate within the robot’s limited computational and memory resources; (b) learning must reach competent performance reliably within a practical wall-clock budget; (c) learning must use rewards that are computable from the robot’s own sensing and simple to specify across tasks. I present three complementary contributions that address the three demands.

The first contribution, the Remote-Local Distributed (ReLoD) framework, supplements the robot’s limited computational resources with a remote workstation. The framework distributes action computation and learning updates between the two computers. Experiments on vision-based tasks showed that the performance

of an agent using Soft Actor-Critic, a widely used batch RL algorithm that learns from stored transitions, degraded under limited onboard computation. A robot using ReLoD recovered performance by drawing on remote computation. Across repeated runs, competent policies were learned from scratch within a few hours.

The second contribution is an empirical study of the *minimum-time* task specification, which assigns a reward of -1 at each timestep and terminates the episode when the goal is reached. Given matched interaction budgets, final policies learned under the minimum-time specification matched or outperformed those learned under *guiding reward* specifications, which reward progress toward the goal. Using the minimum-time specification, four physical robots learned goal-reaching policies from raw-pixel observations within hours and without external instrumentation.

The third contribution is Action Value Gradient (AVG), a computationally cheap streaming RL algorithm that learns with neural networks. An agent using AVG updates its parameters once from each transition (an observation, action, reward, and next observation) and stores no transitions for reuse. On simulated tasks, agents using AVG outperformed agents using existing streaming RL methods, often achieving final performance comparable to batch RL methods. On physical-robot tasks, agents using AVG learned competent policies within hours on an edge device.

The three contributions address the three demands in complementary ways. ReLoD and AVG meet the computational resource demand from opposite directions: ReLoD supplements a robot’s onboard computer with remote computation when feasible, and AVG is computationally cheap, enabling on-device learning. The minimum-time specification reduces specifying a goal-reaching task to one requirement, detecting the goal from onboard sensing. Across the contributions, physical robots learned competent behavior from scratch within hours. Together, the contributions advance real-time reinforcement learning on physical robots toward the longer-term goal of continual adaptation and long-term autonomy.

Preface

The contributions presented in this dissertation are from three published papers:

1. Wang, Y.^{*}, Vasan, G.^{*}, & Mahmood, A. R. (2023). Real-time reinforcement learning for vision-based robotics utilizing local and remote computers. In *Proceedings of the IEEE International Conference on Robotics and Automation*, 9435–9441. (*joint first authorship)
2. Vasan, G., Wang, Y., Shahriar, F., Bergstra, J., Jägersand, M., & Mahmood, A. R. (2024). Revisiting sparse rewards for goal-reaching reinforcement learning. *Reinforcement Learning Journal*, 4, 1841–1854.
3. Vasan, G., Elsayed, M., Azimi, S. A., He, J., Shahriar, F., Bellinger, C., White, M., & Mahmood, A. R. (2024). Deep policy gradient methods without batch updates, target networks, or replay buffers. *Advances in Neural Information Processing Systems*, 37.

I have been fortunate to work with many wonderful collaborators during my PhD. My co-authors Yan Wang, Fahim Shahriar, and Alireza Azimi contributed substantially to designing the robot tasks and running real-time reinforcement learning experiments on physical robots. Yan Wang also played a key role in setting up experiments and co-writing the first and second papers. Jiamin He developed the convergence proof for the streaming reparameterization gradient algorithm in the third paper. Mohamed Elsayed contributed to the clear presentation of ideas in the

third paper and conducted simulation experiments.

Additionally, I have other publications not included in this dissertation, which are listed below:

- Che, F., Vasani, G., & Mahmood, A. R. (2023). Correcting discount-factor mismatch in on-policy policy gradient methods. In *Proceedings of the 40th International Conference on Machine Learning*, 4218–4240.
- Grooten, B., Tomilin, T., Vasani, G., Taylor, M. E., Mahmood, A. R., Fang, M., ... Mocanu, D. C. (2024). MaDi: Learning to mask distractions for generalization in visual deep reinforcement learning. In *The 23rd International Conference on Autonomous Agents and Multiagent Systems*.
- Vasani, G. (2024). Autonomous Skill Acquisition for Robots Using Graduated Learning. In *The 23rd International Conference on Autonomous Agents and Multiagent Systems*.
- Wang, H., Shahriar, F., Azimi, S. A., Vasani, G., Mahmood, A. R., & Bellinger, C. (2024). Versatile and generalizable manipulation via goal-conditioned reinforcement learning with grounded object detection. In *CoRL 2024 Workshop on Mastering Robot Manipulation in a World of Abundant Data*.
- Elsayed, M., Vasani, G., & Mahmood, A. R. (2024). Streaming deep reinforcement learning finally works. arXiv. <https://doi.org/10.48550/arXiv.2410.14606>
- Lan, Q., Vasani, G., & Mahmood, A. R. (2025). Efficient reinforcement learning by reducing forgetting with elephant activation functions. arXiv. <https://doi.org/10.48550/arXiv.2509.19159>

- Shahriar, F., Wang, C., Azimi, A., Vasan, G., Elanwar, H. H., Naik, A., Mahmood, A. R., & Bellinger, C. (2026). Dynamic object masks as goal representations for visual goal-conditioned reinforcement learning. *Reinforcement Learning Journal*.
- Biswas, J., Schoepp, S., Vasan, G., Opiari, A., Zhang, A., Hu, Z., Joseph, S., Lease, M., Li, J. J., Stone, P., Wagstaff, K. L., Taylor, M. E., & Jenkins, O. C. (2026). AI-assisted peer review at scale: The AAAI-26 AI review pilot. arXiv. <https://doi.org/10.48550/arXiv.2604.13940>
- Rojas, J. S., Adamczyk, J., Naik, A., Makarenko, V., Vasan, G., Mukund, N., Tiomkin, S., Kulkarni, R. V., & Lee, C. G. (2026). Reinforcement learning agents are swimmers. *Advances in Neural Information Processing Systems*, 39.

In memory of Vasam Ramadoss

To the dreams inherited from my father, and the gentle strength of my mother

A mind that is full of conclusions is a dead mind, it is not a living mind.

A living mind is a free mind, learning, never concluding.

– Jiddu Krishnamurti

Acknowledgements

While this dissertation is attributed to me as its sole author, the journey has been anything but an individual accomplishment. My life has been shaped by more people than I can name, in more ways than I will ever know. My heartfelt thanks to all of them — the ones I name in these pages, and the ones I have surely missed.

Mentors

My deepest gratitude goes to my advisor Rupam Mahmood, who has been teaching me since my days at Kindred, long before this PhD began. From him I learned to think from first principles, to sit with the fundamental problems of the field, and to build up answers patiently and honestly. I am better for having been held to his high standards for rigor and clarity, and I hope to carry both into everything I do.

I met Patrick Pilarski as an excitable kid, with only enthusiasm to compensate for my lack of research skills. Without his patience and boundless energy, I never would have learned to apply reinforcement learning on real robots, back when doing so was not popular, if not frowned upon. Building intelligent prostheses with Patrick felt futuristic then and still does now.

I am also grateful to Richard Sutton, who has inspired me for as long as I have known the field of reinforcement learning. His CMPUT 609 class captivated me more than a decade ago, and I have been drawn to the beauty and simplicity of his ideas ever since. Rich instilled in me that ideas matter, and my research has

benefited from several clarifying conversations with him.

Matt Taylor has been a constant source of warmth and guidance, and I am glad to have collaborated with him. I am thankful for the amazing opportunities he has sent my way. His sharp questions about human feedback in robot learning changed how I think about the problem. For someone so busy and diligent, his cheerfulness and energy seem inexhaustible — and I hope some of that has rubbed off on me.

I also thank Florian Shkurti and Martin Müller for their time and insightful questions during my defense. Martin brought a planning perspective to the discussion that I had not previously considered. Florian’s vast knowledge of robotics and his appreciation for real-time learning made our discussions a real pleasure; his questions looked past what I had done to where it could go, and several of them have since become directions I hope to pursue.

I also want to thank two researchers whose work and character I deeply admire, Martin Riedmiller and George Konidaris. Martin answered a cold email from a stranger, met me in London, served on my candidacy committee, and opened doors to the ideas and literature of real-time learning. George welcomed me to Providence, shifted how I saw my own work, and encouraged me to pursue computationally cheap methods in an age of scaling to bigger compute clusters.

Lastly, I owe the beginning of it all to my high school teachers in Chennai, India. They saw an interest in science before I could name it myself. For a middle-class kid who had seen money run short, the well-traveled road promised stability, and it would have been easy to take. Instead, they gave me every chance to glimpse a bigger world and, in turn, to pursue a road less traveled. And that did make all the difference. Without them there would have been no path to engineering, let alone to scientific research in a faraway land. In no particular order: Parvati, Nalini, Srilatha, Latha Swaminathan, Subhashini, Geetha Ganesh, Kaanti Ramakrishnan, Satyamoorthy, Vijaya, Santhanam, Venkata Subbarao and TR Subramaniam.

Collaborators, Colleagues, Family and Friends

My research has been collaborative from the start, and I have had the pleasure of working with many wonderful people during my PhD: Mohamed Elsayed, Jiamin He, Fahim Shahriar, Alireza Azimi, Bram Grooten, Yan Wang, Hany Hamed, Cheryl Wang, Fengdi Che, Qingfeng Lan, Valliappan, Martin Jagersand, Martha White, Colin Bellinger, and James Bergstra. Each of them shaped how I think and work. Mohamed's resolve and ambition were instrumental in making our big push on Streaming RL; sharing a common vision made the work enjoyable and the path forward easier. Thanks to Jiamin's efforts, my appreciation and understanding of RL theory have deepened. Working with Bram during his exchange visit was a genuine pleasure, and it's been a delight to keep crossing paths with him at conferences every year since.

Huge thanks to Jonathan and Bhuva from the UofA Writing Centre for their invaluable feedback on my drafts, and to Martha Steenstrup for her writing advice and inspirational pep talks. Thanks as well to Balaraman Ravindran, Nishanth Anand, Rahul Narava and Rosanne Liu for hosting my talks and giving my ideas an audience. And thanks to Vivek Veeriah, Akhil Bagaria, Jacob Adamczyk, Kenny Young, Shibhansh Dohare and Sudharshan Suresh for many interesting conversations and their support with my job search.

Much of the day-to-day joy of this PhD came from the people around me on campus and in Edmonton. To my friends Roberto, Anali, Gowthami, Debanjan, Sheila, Tata Ganesh, Colin, Farzane, Alex, Katie, Bryan Chan, Esraa Elelimy, Jim, Tiffany, Megha, Navaneeth, Michael, Samuele, Fatima, Vera, Martin, Subhojeet, Aakash and Aidan: my life in Edmonton would not have been half as fun without you. Thank you for all the memories. To Shivam, among the most intellectually curious and playful people I have had the pleasure of knowing: thank you for some of the most fun and stimulating conversations of my life. To Abhishek, for hosting the

Making Minds Reading Group and for all the great conversations in between. And to my mates on campus, Homayoon, Shang Wang, Haruto, Fiona, Callie, Blanca and Graves: thank you for making the university a place worth showing up to.

Old friendships carried me through this journey just as much as new ones. To my high school friends Ramkumar and Vishaal, for hosting me in Boston and Pittsburgh; I am grateful we kept running into each other at different points in our lives, despite the chaos. To my undergrad friends Naresh, Boxer, TSB, Ramaravind, Satish, Sarathy and SV, for the great times and all the mind-expanding conversations over the years. The impromptu bungee jumping and skydiving adventures with Satish and Naresh in Delft and Stockholm are memories I will carry for life. Thanks to Ruthrash and Keerthi for the meetups in Toronto and Edmonton. To PV and Bala, for all the meetups in Chennai; I am especially glad I could get PV that autograph from Ted Chiang, which was in itself an experience to remember.

Along the way, I was also lucky to find welcoming communities far from home. I thank the Sanctuary AI team, Olivia, Geordie, Salar and Mani; the Freiburg folks, Joschka, Jan Ole, Erfan, Yannick, Lisa and Jasper, and Santosh for letting me fly his plane; the Brown University crew, Sam Lobel, David Tao and Saket Tiwari, for making my Providence visit feel special; and my team at Zyphra, Tom, Beren, Nick and Fernando, for supporting my thesis writing alongside a full-time job.

To Vanaja Paati and Sambandam Thatha, for their unwavering belief in me and my academic pursuits. To the Ramadoss family, for their warmth and good wishes over the years. To Malavika, Balakumar, Latha, Shweta and Seethalakshmi, for their well-wishes and cheerful support. And to my future in-laws, Prakash and Nithya, for welcoming me with so much love and kindness.

To my family, Srinath, Appa and Amma: your unconditional love and support have been the quiet foundation beneath everything I have done. You believed in this long before there was anything to show for it, and that belief carried me further

than you know. I hope I have made you proud.

Finally, none of this would have been possible without my Varshini. She has been with me through thick and thin, patiently supporting me through every deadline while filling our days with amazing experiences—from spontaneous trips to walks along the river valley that made the hard stretches easier. Through it all, she never stopped believing in me. I cannot wait to spend a lifetime together.

Contents

1	Introduction	1
1.1	Research Objective and Contributions	6
1.2	Layout of the Dissertation	8
2	Background	10
2.1	Reinforcement Learning Preliminaries	11
2.2	Reinforcement Learning in the Context of Robotics	13
2.3	Function Approximation with Neural Networks	16
2.4	Estimating the Policy Gradient	18
2.5	Likelihood Ratio Policy Gradient Methods	19
	REINFORCE	20
	Actor-Critic Methods	21
	Proximal Policy Optimization	24
2.6	Reparameterization Policy Gradient Methods	27
	Twin Delayed Deep Deterministic Policy Gradient	30
	Soft Actor-Critic	32
2.7	Streaming Versus Batch Reinforcement Learning	36
3	Real-Time Reinforcement Learning with Physical Robots	39
3.1	Ordering of Computations for Real-Time Reinforcement Learning	41
3.2	Prior Work: Learning in Real Time With Physical Robots	43
3.3	Discussion	50
4	Real-Time Reinforcement Learning for Vision-Based Robotics Utilizing Local and Remote Computers	54
4.1	The ReLoD Framework	57
4.2	Vision-Based Reinforcement Learning Tasks	60
4.3	Policy Parameterization and Network Architecture	62
4.4	SAC and PPO Under Ample and Limited Computation	64
4.5	Effect of Distributing Action Computation and Learning Updates on SAC and PPO	66
4.6	Real-Time Learning on a Mobile Robot	68
4.7	Conclusion	69
4.8	Limitations and Future Work	70

5	Minimum-Time Task Specification for Goal-Reaching Reinforcement Learning	72
5.1	Related Work	73
5.2	Specifying Goal-Reaching Tasks	76
5.3	Minimum-Time Policies Outperform Guiding-Reward Policies on Reacher	79
	Learning Under the Three Specifications	79
	Evaluating the Final Policies	80
	Evaluating Policies Across Specifications	81
5.4	Target Hits of the Initial Policy Can Predict Learning Success	83
5.5	Learning Vision-Based Minimum-Time Tasks on Physical Robots . . .	85
5.6	Minimum-Time Policies Remain Better on Physical Robots	91
5.7	Conclusion	91
5.8	Limitations and Future Work	92
6	Deep Policy Gradient Methods Without Batch Updates, Target Networks, or Replay Buffers	94
6.1	Impact of Buffer Size on Deep Reinforcement Learning Methods . . .	95
6.2	The Action Value Gradient Method	97
6.3	AVG on Simulated Benchmark Tasks	105
6.4	Stabilizing Streaming Policy Gradient Methods	108
	Instability Without Normalization	108
	Disentangling the Effects of Normalization	110
	Impact of Normalization on OAC, SAC-1, and TD3-1	112
	AVG with Target Q-networks	114
6.5	AVG with Resource-Limited Robot Learning	116
6.6	Conclusion	118
6.7	Limitations and Future Work	119
7	Conclusions and Future Work	123
7.1	Contributions	123
7.2	Directions for Future Work	126
	Combining Pre-Training with Real-Time Learning	126
	Streaming Model-Based Reinforcement Learning	128
	Continual Learning for Long-Term Autonomy	129
	References	130
	Appendix A	162
A.1	Theoretical Foundations	162
	Proof Sketch of the Reparameterization Policy Gradient Theorem . .	163
	Action Value Gradient Theorem	164
A.2	Estimation of FLOPs per Learning Update	165
A.3	SAC Hyperparameters for Experiments with Minimum-Time Tasks . .	166
A.4	AVG with Target Q Networks	166

A.5	Hyperparameters for Simulation Experiments	168
	PPO Hyperparameters	168
	TD3 Hyperparameters	169
	SAC Hyperparameters	170
A.6	Hyperparameter Optimization Using Random Search	171
A.7	AVG Hyperparameters Across Tasks	172
A.8	The Effect of Other Network Normalization Techniques	172
A.9	AVG Hyperparameters for Physical Robot Experiments	174

List of Tables

3.1	Real-time reinforcement learning on physical robots, organized along three axes: policy/value function class (tabular, linear, instance-based, or neural network) and policy/value update regime (streaming or batch) for rows, and update scheduling (synchronous, asynchronous, or inter-episode) for symbols. References in blue are the contributions of this dissertation.	46
4.1	The processes of a SAC agent and their distribution between the local and the remote computers in the three operational modes of the ReLoD framework.	59
5.1	Task specifications for Reacher: choices for reward function and termination conditions. Agents learned under all three specifications with a discount factor of $\gamma = 0.99$	77
5.2	Comparison of the three task specifications on three criteria.	80
5.3	Robot experiment setup. The average target hits were calculated over five independent runs using the initial Gaussian policy $\mathcal{N}(0, 1)$ over 20K timesteps for each task. Robot experience time excludes time spent resetting the environment and charging batteries.	87
A.1	SAC hyperparameters for the experiments with minimum-time tasks.	166
A.2	PPO hyperparameters for the simulation experiments, the defaults of the CleanRL implementation.	168
A.3	TD3 hyperparameters for the simulation experiments, the defaults of the CleanRL implementation.	169
A.4	SAC hyperparameters for the simulation experiments.	170
A.5	Hyperparameter values and search ranges for the random search procedure.	171
A.6	Best AVG hyperparameters for each simulation task, found by random search.	172
A.7	AVG and SAC-1 hyperparameters for the robot experiments.	174

List of Figures

2.1	The agent-environment interface	11
2.2	Inference and learning in agents under streaming and batch reinforcement learning.	36
2.3	Categorization of the policy gradient methods discussed in this chapter along two axes: streaming versus batch learning, and likelihood ratio versus reparameterization gradients. Action Value Gradient (AVG) of Chapter 6 is a streaming deep reinforcement learning method contributed by this dissertation.	38
3.1	Sequential computations in an agent-synchronized simulated task. .	41
3.2	Concurrent computations in real-world tasks.	42
3.3	The figure covers a subset of the surveyed work through 2025 for which per-update computation could be estimated; Table 3.1 lists the full set. Each point gives the estimated floating-point operations per learning update for one system, plotted on a logarithmic scale in chronological order. Marker shape and color denote the learning regime. Background shading denotes the dominant hardware era by publication year. Streaming methods with linear function approximation remain within a few orders of magnitude of their 1990s cost, whereas batch methods with neural networks grow with available hardware, exceeding streaming linear methods by up to ten orders of magnitude. Appendix A.2 describes the estimation method. . . .	51
4.1	The ReLoD framework distributes the computations of a learning agent between a local and a remote computer.	57
4.2	A SAC agent in the Remote-Local mode. The transition queues on the local and the remote computers have lengths equal to the maximum episode length to avoid losing data.	58
4.3	UR5-VisualReacher	60
4.4	Create-Reacher	61
4.5	The convolutional network architecture. The yellow blocks are convolutional layers, the teal block is the non-image observation vector, the red block is the previous action, and the magenta blocks are fully connected layers.	63

4.6	Learning curves of SAC under three tiers of computation and of PPO on the workstation, on UR5-VisualReacher. The wide lines are averaged over five independent runs.	65
4.7	Learning curves of SAC in the three modes of operation of ReLoD on UR5-VisualReacher. The wide lines are averaged over five independent runs.	66
4.8	Learning curves of PPO in the three modes of operation of ReLoD on UR5-VisualReacher. The wide lines are averaged over five independent runs.	67
4.9	The Create-Reacher hardware setup.	68
4.10	Learning curves of SAC in the Remote-Local and Remote-Only modes on Create-Reacher. The wide lines are averaged over five independent runs.	69
5.1	Reacher-Easy (left) and Reacher-Hard (right) from the DeepMind Control Suite.	76
5.2	Learning curves of SAC under the three reward specifications on Reacher. Each solid curve shows the mean of 30 independent runs, with shaded regions indicating the 95% confidence interval.	79
5.3	Comparison of the three specifications of the reaching task using the number of steps it requires to reach the goal (left) and the number of steps it stays within the target sphere upon reaching it (right). The reported mean is estimated over 30 independent runs.	81
5.4	Evaluating the policies of all three specifications on each specification's objective, on Reacher-Easy (left column) and Reacher-Hard (right column). Each solid learning curve is an average of 30 independent runs. The shaded regions represent a 95% confidence interval.	82
5.5	Target hits achieved by a random initial policy within 20K timesteps, across a range of time limits.	83
5.6	Learning performance of SAC on three non-visual and one visual task in simulation for multiple choices of time limit. Each solid curve is averaged over 30 independent runs. The shaded regions represent a 95% confidence interval.	84
5.7	Learning performance of SAC on four vision-based robot tasks. Each thin learning curve represents an independent run. The bold learning curves of the same color are the average of all independent runs. The initial policy is excluded.	88
5.8	Frames sampled from single episodes of learned policies completing Create-Reacher (top) and UR5-VisualReacher (bottom), superimposed. Episodes start from random configurations.	89
5.9	Frames sampled from single episodes of learned policies completing Vector-ChargerDetector (top) and Franka-VisualReacher (bottom), superimposed. Episodes start from random configurations.	90

5.10	Comparison of the guiding reward and minimum-time specifications of the reaching task using the number of steps it requires to reach the goal (left) and the number of steps it stays on target upon reaching it (right).	91
6.1	Impact of buffer size on PPO, SAC, and TD3. Reducing the buffer size degraded learning, and agents trained with all three methods failed when the buffer held only the most recent sample. AVG, which requires no replay buffer, is plotted at a buffer size of 1. Each point is the mean episodic return over the final 100K timesteps, averaged over 30 runs; all agents were trained for 10M timesteps. . .	96
6.2	AVG on Gymnasium and DeepMind Control Suite tasks. Each solid curve is an average over 30 independent runs; shaded regions are 95% bootstrap confidence intervals. The gray dashed line, labeled SAC, shows the mean performance of agents trained with SAC and a replay buffer of 1M transitions over the final 10K timesteps of their 1M timesteps of training. Continued on the next page.	106
6.2	AVG on Gymnasium and DeepMind Control Suite tasks, continued.	107
6.3	Average episodic returns, gradient norms of the actor and critic networks, and mean absolute action values, averaged over action dimensions, for AVG, AVG^- , and SAC on Humanoid-v4. AVG^- denotes AVG without any normalization. A mean absolute action value near 1 indicates saturation at the action bounds. Dark lines are averages over 30 runs; light lines are individual runs. The y-axes of the gradient-norm plots are logarithmic.	109
6.4	Ablation study of normalization techniques used with AVG. Each curve shows the best hyperparameter configuration for one variant on one task, averaged over 30 independent runs. The shaded regions represent a 95% confidence interval.	111
6.5	Impact of normalization on OAC, SAC-1 and TD3-1. Suffix “+” denotes each algorithm plus normalization. Each solid learning curve is an average of 30 independent runs. The shaded regions represent a 95% confidence interval. Continued on the next page. . .	113
6.5	Impact of normalization on OAC, SAC-1 and TD3-1, continued. . .	114
6.6	Impact of a target Q-network on AVG for different values of τ , the Polyak averaging coefficient. Here, $\tau = 0$ corresponds to a fixed target Q-network, and $\tau = 1$ makes the target identical to the current Q-network, that is, no target network. Each solid learning curve is an average of 30 independent runs. The shaded regions represent a 95% confidence interval.	115
6.7	The two robot tasks: (a) a UR5 arm on UR-Reacher-2 and (b) an iRobot Create 2 on Create-Mover.	116
6.8	Learning curves on UR-Reacher-2 (left) and Create-Mover (right). Dark lines are averages over five runs; light lines are individual runs.	117

A.1	Learning curves of AVG agents with penultimate normalization (pnorm), layer normalization (layer norm), and RMS normalization (RMS norm).	173
-----	---	-----

Chapter 1

Introduction

Learning from experience is essential for an agent operating in a large, dynamic world. Consider a squirrel in the Canadian Rockies: in summer, greenery flourishes and food is abundant, whereas in winter, trees wither and resources become scarce. To survive, the squirrel must adapt continuously, adjusting foraging behavior across seasons, detecting predators, navigating shifting terrain, and remembering the locations of hidden food caches. A robot navigating a warehouse faces the same challenge: inventory shifts, routes change, and new obstacles appear. Like the squirrel, the robot can be modeled as an agent that must make decisions over time in an unpredictably changing environment. Without the ability to adapt, a robot must be reprogrammed for every new situation, requiring costly human effort that scales poorly. A robot that learns continuously and autonomously could adapt to a non-stationary world without costly human intervention.

In this dissertation, I develop a computational framework and an algorithm, each instantiated in complete robotic systems that learn on the fly through interaction with the world, and I investigate whether simple task specifications suffice for goal-reaching problems. I use the word *system* to mean an integrated collection of hardware and software components that together produce behavior. To learn from

ongoing interaction with the world, a robot must repeatedly sense its current situation, take actions, and update its behavior from the feedback that follows. The real world does not pause for the robot’s computation; other agents move, objects shift, and conditions change. *Real-time learning* names the setting in which an agent senses, acts, and learns from ongoing interaction as wall-clock time advances for both the agent and the environment.

Reinforcement learning (RL) is a natural approach to learning from ongoing interaction: rather than being told which actions to take, an agent discovers which actions yield the most reward through trial-and-error interaction with its environment (Sutton & Barto 2018). An RL agent learns a policy that maps situations to actions to maximize cumulative reward over time. A practitioner need not specify behavior directly. Instead, the practitioner specifies the task through the reward signal, and the agent discovers the behavior. The choice of task specification shapes both what the agent learns and how quickly it learns. RL offers a powerful way to acquire sophisticated robotic skills that are otherwise difficult to hand-engineer (Kober et al. 2013). Applying RL in a real-time setting, however, is not straightforward: standard RL methods do not immediately translate to the practical demands of real-time operation.

Real-time reinforcement learning on physical robots imposes three practical demands. First, learning must operate within the robot’s limited computational and memory resources, which are also used for sensing, acting, and other processes. The robot must compute each action within its *action cycle time*, the fixed interval between successive action selections. The requirement is soft: a late action does not halt the robot, but control degrades with the delay. Learning updates need not finish within a single action cycle, but slower updates can delay adaptation. Learning must therefore make economical use of the limited resources while still adapting quickly. Second, a robot must learn reliably in limited wall-clock time. Learning on phys-

ical robots is costly: experience consumes equipment time and energy, hardware wears down with use, and poor exploration risks damage. Therefore, both learning time and run-to-run consistency matter. Across repeated runs with fixed hyperparameters and the same experimental protocol, learning should reach a competent level of performance within a practical wall-clock budget. Third, learning should use rewards that the robot can compute from its own sensing and that are simple to specify across tasks. A reward signal that depends on resources absent where and when the robot operates, such as external instrumentation or sustained human feedback, cannot support learning during deployment (Hafner et al. 2021, Ibarz et al. 2021). Rewards computed from onboard sensing alone free the robot from that dependence. A long-lived robot would also be expected to solve many tasks over its lifetime, and engineering a reward function for every task by hand scales poorly. Rewards that are simple to specify limit the per-task effort. Real-time learning must therefore meet the robot’s action-cycle, computation, and memory constraints, reach competent performance reliably within a practical wall-clock budget, and use rewards computable onboard and simple to specify across tasks.

In *streaming reinforcement learning*, an agent learns from a stream of experience by using each arriving transition—an observation, action, reward, and next observation—for one learning update and then discarding it rather than retaining it for later reuse. This regime requires no memory for experience replay, and each update processes one transition rather than a batch, making it well suited to real-time RL on resource-constrained robots. Early demonstrations of real-time RL on physical robots used streaming updates (Mahadevan & Connell 1991, Benbrahim et al. 1992, Gullapalli et al. 1994, Gaskett et al. 2000). Chapter 3 documents the later adoption of replay buffers and batch updates alongside increases in available computational resources (Hafner & Riedmiller 2003, Riedmiller 2005b, Lange et al. 2012, Haarnoja et al. 2019).

Replay and batch updates introduce a resource–performance tradeoff. Reusing transitions can improve sample efficiency, and averaging updates over batches can reduce the variance of individual updates, important benefits when physical-robot experience is costly (Gu et al. 2017, Riedmiller et al. 2022). A fixed-capacity replay buffer has bounded memory, but its experience-storage requirement scales with the chosen capacity. A batch update processes multiple transitions, and its arithmetic cost therefore scales with batch size, although its wall-clock duration also depends on the model, hardware, parallelism, and update schedule (Yuan & Mahmood 2022, Smith et al. 2023). Whether replay and batch updates reduce total wall-clock learning time therefore depends on whether their gains in sample efficiency outweigh their additional computation.

Two widely used approaches in robot learning—*learning from demonstrations* and *simulation-to-reality transfer*—often use train-then-deploy workflows rather than learning in real time. In these workflows, practitioners train a policy before deployment and hold it fixed during deployment. In learning from demonstrations, practitioners train policies from human-provided trajectories, including teleoperated robot demonstrations and video demonstrations (Fu et al. 2020, Chi et al. 2024). In simulation-to-reality transfer (sim-to-real), practitioners train policies using large amounts of simulated interaction before transferring them to a physical robot (Akkaya et al. 2019, Miki et al. 2022, Degraeve et al. 2022).

When training is separate from deployment, learning updates need not run under the constraints of real-time operation. Training need not meet the robot’s action-cycle deadline and can use computational resources unavailable to the deployed robot. During deployment, policy inference must still fit within the action cycle, but the robot need not allocate compute or memory to learning updates or experience replay. Human demonstrations reduce the need for autonomous trial and error on physical hardware. Sim-to-real shifts experience collection to simulation, where

interaction is inexpensive, exploration poses no risk to hardware, and privileged information unavailable during deployment can be used to compute rewards. The separation is also attractive because reliable real-time learning remains difficult. Successful real-time RL on physical robots often depends on careful task setup, well-tuned hyperparameters, and substantial computation during operation (Mahmood et al. 2018a, Mahmood et al. 2018b, Haarnoja et al. 2019). Unsuitable task setups, poorly chosen hyperparameters, or insufficient computation can result in slow or failed learning (Dulac-Arnold et al. 2021, Ibarz et al. 2021).

Separating training from deployment limits adaptation after deployment. Two-stage workflows are well suited to tasks and environments known in advance, but they are less suited to conditions that change in ways not anticipated during training. Before deployment, demonstrations must be performed, collected, and curated (Brohan et al. 2022, Amin et al. 2025), or simulators must be designed, calibrated, and refined (Akkaya et al. 2019, Torne et al. 2024). After deployment, degraded performance must be detected and corrected. An automated monitor can detect it, but correction currently falls to a human engineer. The engineer diagnoses the problem, collects additional demonstrations or refines the simulator, retraining the policy, and deploys the new policy (Amin et al. 2025, Luo et al. 2025); each unanticipated change repeats the cycle. The required time and expertise grow with the number of robots and the length of their operation.

For robots operating over long periods in changing environments, autonomy requires more than a policy that performs well at initial deployment. A policy whose parameters remain fixed may generalize to unfamiliar conditions, but it cannot update its behavior from experience collected after deployment. Real-time RL provides a mechanism for updating behavior after deployment and could reduce reliance on humans for data collection, retraining, and redeployment. Robots that update their behavior from post-deployment experience could therefore operate au-

tonomously for longer with less human intervention.

Learning reliably from runtime experience is necessary, but not sufficient, for continual adaptation. This dissertation studies that requirement by examining how physical robots can learn control policies during operation while meeting action-cycle, computation, and memory constraints, reaching competent performance reliably within practical wall-clock budgets, and limiting task-specific reward engineering. The empirical studies evaluate learning within bounded experimental runs, not adaptation across a sequence of environmental or task changes. Together, my contributions advance real-time robot learning toward the longer-term goal of continual adaptation and autonomy.

1.1 Research Objective and Contributions

My objective in this work is to advance real-time reinforcement learning on physical robots by addressing three practical demands: (a) selecting each action within its deadline and learning within the computation and memory available to the robot; (b) reaching competent performance reliably within a practical wall-clock budget; and (c) using rewards that are computable onboard and simple to specify across tasks.

I pursue this objective through three complementary contributions to real-time RL: a distributed computing framework, an empirical study of a simple task specification for goal reaching, and a streaming reinforcement learning algorithm.

1. **The Remote-Local Distributed (*ReLoD*) computational framework for real-time RL.** I develop ReLoD, a framework that supplements the robot’s onboard computational resources with a remote workstation. The framework

distributes action computation and learning updates between the two computers. I first show that the performance of an agent using Soft Actor-Critic, a widely used RL algorithm, degrades as onboard computational resources are reduced. Under the same onboard limits, a robot using ReLoD recovers performance by drawing on remote computational resources during operation. Across repeated physical-robot runs, robots using ReLoD learn vision-based control policies from scratch within a few hours. ReLoD addresses the computational part of the first demand, while the repeated-run results provide evidence of reliable learning within a practical wall-clock budget.

2. **An empirical study of the minimum-time task specification for goal reaching.**

I evaluate the *minimum-time task specification*, which assigns a reward of -1 at each timestep and terminates the episode when the goal is reached. Under matched interaction budgets, the final policies learned under the minimum-time specification outperform those learned under the *guiding-reward* specifications, which provide intermediate feedback based on progress toward the goal. Using the minimum-time specification, four physical robots learn goal-reaching policies from raw-pixel observations within practical wall-clock budgets and without external instrumentation. Both the minimum-time and guiding-reward specifications were computed from onboard sensing. The comparison therefore isolates simplicity of specification. The same minimum-time specification serves every task, addressing the third demand.

3. **A streaming reinforcement learning algorithm: Action Value Gradient (AVG).**

I develop AVG, a computationally inexpensive streaming reinforcement learning algorithm. An agent using AVG updates its parameters once from each arriving transition and does not retain transitions for experience re-

play. In the survey presented in Chapter 3, I find that prior physical-robot evidence for streaming RL with neural networks was limited to a small number of robot implementations and environments. I evaluate agents using AVG on ten simulated continuous-control tasks and two physical-robot tasks with continuous action spaces. Across repeated runs, these agents achieve competent performance within practical wall-clock budgets. The single-transition updates and absence of a replay buffer address the computation and experience-storage parts of the first demand, while the repeated-run results provide evidence that the second demand is met.

I investigate complementary responses to limited memory and computational resources through ReLoD and AVG. A robot using ReLoD draws on remote computational resources during operation at the cost of a more complex architecture. An agent using AVG can instead learn on-device under computation and memory constraints by updating its parameters from individual transitions without retaining them for experience replay. The minimum-time specification is computed from onboard sensing and serves every studied goal-reaching task, addressing the third demand. Under matched interaction budgets, the final policies learned under the minimum-time specification outperform those learned under guiding-reward specifications, although effective exploration remains necessary. Across all three contributions, repeated physical-robot runs provide evidence about learning performance and run-to-run consistency within practical wall-clock budgets.

1.2 Layout of the Dissertation

Chapter 2 presents the concepts used throughout the dissertation. I formalize reinforcement learning as a Markov decision process, situate reinforcement learning in the context of robotics, and review function approximation with artificial neural

networks. I then introduce likelihood-ratio and reparameterization policy gradient methods and distinguish streaming from batch learning, an organizing distinction that recurs in later chapters.

Chapter 3 surveys three decades of real-time reinforcement learning on physical robots and organizes the surveyed work by function approximator, update regime, and update scheduling. I also estimate the computation required for selected learning updates and trace how increasing update costs changed the scheduling of learning relative to action selection. I use this analysis to motivate the distributed computation in ReLoD and the streaming updates in AVG.

Chapter 4 presents the development and evaluation of ReLoD, a distributed computing framework for action computation and learning updates across an on-board computer and a remote workstation. The experiments examine how agent performance depends on available computation and show that drawing on a remote workstation recovers the performance lost to limited onboard resources.

In Chapter 5, I use ReLoD to study task specification for physical-robot goal reaching, comparing the minimum-time task specification with guiding-reward specifications. This comparison examines whether the minimum-time specification can reduce task-specific reward-design effort without sacrificing learning performance.

Chapter 6 returns to computational constraints at the algorithmic level. I develop and evaluate AVG, a streaming reinforcement learning algorithm. Agents using AVG update their parameters from individual transitions and do not retain transitions for experience replay.

Chapter 7 synthesizes the contributions, discusses their implications for real-time robot learning, and identifies directions toward continual adaptation and long-term robot autonomy.

Chapter 2

Background

This dissertation builds on artificial neural networks, reinforcement learning, and robotics. In this chapter, I review the concepts the later chapters require. Section 2.1 introduces the reinforcement learning problem, formalizes it as a Markov decision process (MDP), and defines returns and value functions. Section 2.2 situates reinforcement learning in the context of robotics, where continuous action spaces and high-dimensional observations favor policy-based methods, and highlights the challenges of robot learning, particularly reward specification. Section 2.3 describes function approximation with neural networks and defines deep reinforcement learning. Section 2.4 formulates the problem of estimating the policy gradient. Sections 2.5 and 2.6 present the two families of policy gradient methods the dissertation uses, one based on likelihood ratio gradients and the other on the reparameterization technique. Section 2.7 distinguishes *streaming* from *batch* learning, an organizing distinction that recurs in later chapters.

Notation

Capital letters denote random variables, such as the state S_t and action A_t , and lowercase letters denote their realizations, such as s and a . State and action realizations

may be vector-valued despite being unbolded, following conventional reinforcement learning notation. Algorithm pseudocode uses capital letters such as S , A , R , and S' for the state, action, reward, and next state generated during stochastic execution. On any particular execution, these variables take realized values. Script letters denote sets, such as the state space $\mathcal{S}$, action space $\mathcal{A}$, and reward set $\mathcal{R}$. Bold lowercase Roman or Greek symbols denote other vectors, such as the value-function parameters $\mathbf{w}$, policy parameters $\boldsymbol{\theta}$, and neural network activations $\mathbf{h}_\ell$. Bold uppercase symbols denote matrices, such as a neural network weight matrix $\mathbf{W}_\ell$. The operator $\text{sg}(x)$ denotes a stop-gradient operation: it has value x , but its derivative with respect to x is zero. Gradients of scalar-valued functions are column vectors. For a vector-valued function $\mathbf{f} = (f_1, \dots, f_m)$, $\nabla_{\mathbf{x}} \mathbf{f}$ denotes the matrix whose j th column is $\nabla_{\mathbf{x}} f_j$. The symbol $\doteq$ marks a definition.

2.1 Reinforcement Learning Preliminaries

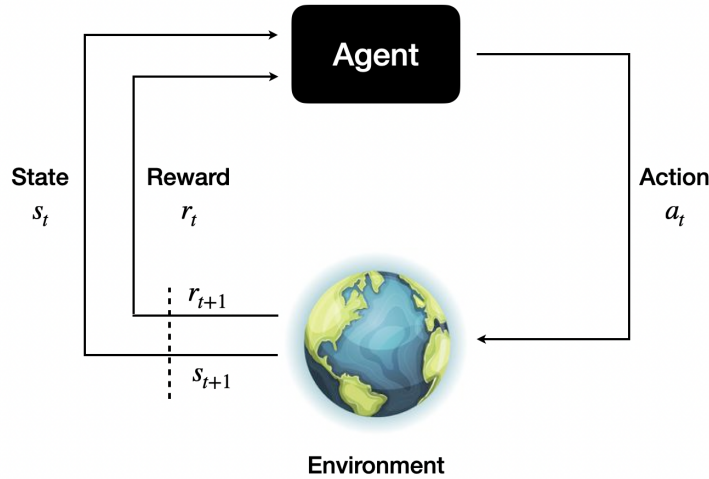


Figure 2.1: The agent-environment interface

In reinforcement learning, an agent learns through trial-and-error interactions with its environment to choose actions that maximize cumulative reward (Sutton & Barto 2018). The Markov decision process (MDP) is the standard formalism for

how a learning agent interacts with its environment, and this dissertation adopts it throughout. The agent and environment interact at a sequence of discrete timesteps $t = 0, 1, 2, \dots$. At each timestep, the agent receives a state $S_t \in \mathcal{S}$, selects an action $A_t \in \mathcal{A}$, and one timestep later receives a scalar reward $R_{t+1} \in \mathcal{R}$ together with the next state $S_{t+1} \in \mathcal{S}$ (Figure 2.1). The environment dynamics are specified by the conditional distribution $(S_{t+1}, R_{t+1}) \sim P(\cdot, \cdot \mid S_t, A_t)$. For discrete state and reward spaces, I write the corresponding probability mass function as $p(s', r \mid s, a) \doteq \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\}$. For continuous variables, p denotes a probability density when one exists.

The goal of a reinforcement learning agent is to learn a policy $\pi(\cdot \mid s)$, which assigns a probability distribution over actions to each state, that maximizes the expected return. When the action space is discrete, $\pi(a \mid s)$ is the probability of selecting action a in state s ; when the action space is continuous, it denotes a probability density. The return G_t is the discounted sum of future rewards from timestep t onward:

$$\begin{aligned} G_t &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T \\ &= \sum_{k=t+1}^T \gamma^{k-t-1} R_k, \end{aligned} \tag{2.1}$$

where $\gamma \in [0, 1]$ is a discount factor and T is the final timestep, including the possibility that $T = \infty$ or $\gamma = 1$, but not both.

A *value function* gives the expected return from a state, or from taking an action in a state, under a given policy. Value functions serve to state objectives, such as the expected return from the start state under the policy, and most reinforcement learning algorithms estimate them. The state-value function $v_\pi(s)$ gives the expected

return when starting in state s and following policy π thereafter:

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s]. \quad (2.2)$$

The action-value function $q_\pi(s, a)$ gives the expected return when starting in state s , taking action a , and following π thereafter:

$$q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]. \quad (2.3)$$

The advantage function $\Psi_\pi(s, a) \doteq q_\pi(s, a) - v_\pi(s)$ gives how much better it is to take action a in state s than to act according to π .

The Bellman equations express each value function recursively, relating the value of a state or state-action pair to the immediate reward and the value of its successors:

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s], \\ q_\pi(s, a) &= \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]. \end{aligned}$$

The state-value function is the policy-weighted expectation of the action-value function:

$$v_\pi(s) = \mathbb{E}_{A \sim \pi(\cdot \mid s)}[q_\pi(s, A)].$$

For terminal states, the successor value is defined to be zero.

2.2 Reinforcement Learning in the Context of Robotics

The agent-environment boundary is functional rather than physical, even in robotics, where the robot's body suggests a physical boundary. The agent is the

learner and decision maker, and the environment is everything else that surrounds it. The processes governing a robot’s body, such as motor dynamics and sensing, and even some of its internal computations, belong to the environment whenever the agent cannot directly modify them.

A Markov state contains the information needed to predict the distribution of the next state and reward given an action. An *observation* is the information available to the agent, such as measurements from a robot’s sensors, and need not contain the full Markov state. In the robot experiments, S_t denotes this information supplied to the agent and may therefore be noisy or incomplete. Later chapters use *observation* in prose while retaining S_t in equations and pseudocode for notational uniformity. If S_t is not a sufficient statistic of the interaction history, the MDP identities stated in Section 2.1 do not apply exactly to it.

Applying reinforcement learning methods to a robot task begins with specifying the task as a Markov decision process. A human designer chooses the observations available to the agent, the actions it can take, the reward signal, and the termination conditions that encode the task. Each of these choices shapes what the agent can learn and how efficiently it can learn it. Without exploitable structure, the number of samples required to learn an effective policy grows with the dimensionality of the state-action space, a phenomenon known as the curse of dimensionality (Bellman 1957). Specifying a reward signal that accurately captures the behavior the designer intends is difficult (Kober et al. 2013). An agent maximizes a poorly chosen reward as specified, through behaviors that are unsafe or that fail to capture the designer’s intent (Amodei & Clark 2016, Booth et al. 2023).

Robot tasks typically provide the agent with high-dimensional sensor inputs and require continuous actions. The input may include joint positions and velocities, inertial measurements, contact forces, and camera images; a single image alone may contribute tens of thousands of dimensions. These measurements are often noisy

and may not reveal the complete physical state because sensors have limited precision and coverage. Robot actions also commonly have several continuous dimensions. Tabular representations, which maintain a separate value for each state or state-action pair, are infeasible in such spaces.

Reinforcement learning in high-dimensional state spaces and continuous action spaces requires methods that generalize across states and actions. This dissertation uses function approximation to obtain this generalization by learning parameterized value functions and policies. Value-based methods like Q-learning (Watkins 1989) select the action with maximum value at a given state and have proven effective in games with discrete actions (e.g., up, down, left, right). In a continuous action space, selecting the action with maximum value requires solving an optimization problem at each decision. This optimization can be computationally difficult in high-dimensional action spaces, which limits the direct application of value-based methods to robot control. *Policy-based* methods instead parameterize the policy directly, so action selection requires no maximization. They can also represent stochastic policies, which may aid exploration (Jaakkola et al. 1994). Robot learning systems therefore often use policy gradient methods, the policy-based methods this dissertation builds on, for continuous control (Gullapalli et al. 1994, Kober & Peters 2008b, Kober et al. 2013, Smith et al. 2022).

To guide the learning agent to a reasonable solution, task designers often use task-specific domain knowledge to hand-craft a reward function with state-to-state differences that facilitate faster learning (Matarić 1994, Booth et al. 2023); I call these *guiding rewards*. In contrast, a *sparse* reward signal changes only at rare events, for example, a reward of zero every timestep and +1 upon completing the task, leaving the agent without informative feedback over long stretches. Sparse rewards are simple to specify for a robot task, but agents may fail to learn from them within a feasible amount of physical interaction, if exploration rarely encounters the

rewarding events (Riedmiller et al. 2018).

A long-lived general-purpose robot would be expected to solve a wide range of tasks over its lifetime. Engineering a reward function for every task by hand is cumbersome at that scale. Two properties of the reward function ease the burden. First, rewards computed from onboard sensors alone allow the robot to evaluate its own behavior without external instrumentation, motion capture systems, or human supervision (Hafner et al. 2021). Second, rewards that are simple to specify, rather than requiring elaborate hand-engineering, can capture high-level goals such as catching a ball or opening a door (Riedmiller et al. 2018), and the same specification can extend to many tasks.

2.3 Function Approximation with Neural Networks

Function approximation methods maintain parameterized estimates of value functions and policies and learn the parameters from experience. The estimate of the state-value function is written $\hat{v}(s; \mathbf{w})$, with parameters $\mathbf{w} \in \mathbb{R}^d$, where d is the number of parameters, and the estimate of the action-value function is written $\hat{q}(s, a; \mathbf{w})$. The policy is written $\pi(a \mid s; \boldsymbol{\theta})$, with parameters $\boldsymbol{\theta} \in \mathbb{R}^{d'}$, where d' may differ from d . This dissertation parameterizes value estimates and policies with artificial neural networks.

The artificial neural networks in this dissertation are feedforward networks, fully connected or convolutional, in which each layer multiplies its input by a weight matrix and passes each element of the result through a nonlinear function. A fully connected network with L layers computes its output by applying $\mathbf{h}_\ell = \sigma(\mathbf{W}_\ell \mathbf{h}_{\ell-1} + \mathbf{b}_\ell)$ at each layer $\ell = 1, \dots, L$, where $\mathbf{h}_0$ is the input, $\mathbf{W}_\ell$ and $\mathbf{b}_\ell$ are the weight matrix and bias vector of layer ℓ , and σ is a nonlinear function applied to each element, with the final layer often omitting the nonlinearity. A convolutional layer takes the same form with a weight matrix that applies the same local

filter across spatial positions (LeCun et al. 1998). The parameters of the network are all the weight matrices and bias vectors, which constitute $\mathbf{w}$ when the network represents a value function and $\boldsymbol{\theta}$ when it represents a policy. Learning adjusts these parameters with gradients of a scalar objective, taken with respect to every weight and bias. The gradient is computed by applying the chain rule repeatedly, layer by layer, a procedure known as backpropagation (Rumelhart et al. 1986).

Linear function approximation, a common alternative, represents value functions and policies as linear combinations of fixed features constructed by methods such as tile coding (Sutton & Barto 2018, Chapter 9) or radial basis functions (Geramifard et al. 2013). Linear methods are computationally cheap, but their performance depends on the choice of fixed features, which often requires task-specific design. They are generally unsuitable for raw high-dimensional inputs such as images unless paired with an effective feature representation. Neural networks instead learn feature representations from data, which reduces the need for task-specific feature design and enables them to process high-dimensional inputs.

The parameters of a value estimate are learned from experience by temporal-difference (TD) learning (Sutton 1988a). After each transition, the one-step TD update moves the estimate at the current state toward a target formed from the observed reward and the estimate at the next state:

$$\begin{aligned}\delta_t &= R_{t+1} + \gamma \hat{v}(S_{t+1}; \mathbf{w}) - \hat{v}(S_t; \mathbf{w}), \\ \mathbf{w} &\leftarrow \mathbf{w} + \alpha \delta_t \nabla_{\mathbf{w}} \hat{v}(S_t; \mathbf{w}),\end{aligned}$$

where $\alpha > 0$ is a step size and δ_t is the TD error. When $\hat{v}$ is a neural network, $\nabla_{\mathbf{w}} \hat{v}(S_t; \mathbf{w})$ is computed by backpropagation. The target is treated as fixed during the update; the gradient therefore is taken only through $\hat{v}(S_t; \mathbf{w})$, making this a semi-gradient method (Sutton & Barto 2018). Learning the policy parameters $\boldsymbol{\theta}$

requires the policy gradient, which Sections 2.4 through 2.6 develop.

I use the term *deep reinforcement learning* to refer to RL methods that represent the policy or value function with artificial neural networks of one or more hidden layers and adjust the network parameters by stochastic gradient methods, with gradients computed by backpropagation.

2.4 Estimating the Policy Gradient

Policy gradient methods parameterize the policy directly as $\pi(a \mid s; \boldsymbol{\theta})$, abbreviated $\pi_{\boldsymbol{\theta}}$, and optimize a scalar performance measure by gradient ascent on $\boldsymbol{\theta}$. The performance measure is the expected return under the policy, $J(\boldsymbol{\theta}) \doteq v_{\pi_{\boldsymbol{\theta}}}(s_0) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}[G_0 \mid S_0 = s_0]$ in the episodic setting with start state s_0 . Gradient ascent updates the parameters as

$$\boldsymbol{\theta}_{k+1} \doteq \boldsymbol{\theta}_k + \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_k),$$

with step size $\alpha > 0$.

In the model-free settings considered here, the agent generally cannot compute $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ exactly. Direct evaluation would require integrating over the trajectories the policy could produce and, in general, access to the environment dynamics. Expectations that cannot be computed exactly are common in machine learning, and an *unbiased sample estimator*, a function of samples whose expected value equals the quantity of interest, typically estimates them. When a gradient can be written as an expectation, $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{X \sim p_{\boldsymbol{\theta}}}[\mathbf{g}(X)]$ for some function $\mathbf{g}$, the agent can estimate it from samples $x_1, \dots, x_N$ drawn from $p_{\boldsymbol{\theta}}$ as $\widehat{\nabla_{\boldsymbol{\theta}} J}(\boldsymbol{\theta}) \doteq \frac{1}{N} \sum_{i=1}^N \mathbf{g}(x_i)$, and stochastic gradient methods converge under standard conditions using such estimates in place of the true gradient (Robbins & Monro 1951).

The difficulty for policy gradient methods is that $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ does not immediately take this form. The sampling distribution itself depends on $\boldsymbol{\theta}$. Therefore, the gradient and expectation operators cannot be exchanged directly. For any random vari-

able X with parameterized distribution represented by $p_{\theta}(x)$ ¹ and any real-valued function f that does not depend explicitly on θ ,

$$\begin{aligned}\nabla_{\theta} \mathbb{E}_{X \sim p_{\theta}}[f(X)] &= \nabla_{\theta} \int p_{\theta}(x) f(x) dx \\ &= \underbrace{\int f(x) \nabla_{\theta} p_{\theta}(x) dx}_{\text{not an expectation under } p_{\theta}}. \end{aligned} \quad (2.4)$$

The right-hand side of Equation 2.4 is not an expectation under p_{θ} , because the integrand is weighted by $\nabla_{\theta} p_{\theta}(x)$ rather than by $p_{\theta}(x)$. No sample estimator is therefore immediate. In the policy gradient setting, X is a trajectory, f is the return, and p_{θ} is the distribution over trajectories induced by the policy. The likelihood ratio and reparameterization methods of the following sections rewrite the gradient as an expectation that can be estimated from samples.

2.5 Likelihood Ratio Policy Gradient Methods

The *likelihood ratio estimator*, also known as the score function estimator, rewrites Equation 2.4 as an expectation under p_{θ} using the log-derivative relation $\nabla_{\theta} \log p_{\theta}(x) = \nabla_{\theta} p_{\theta}(x) / p_{\theta}(x)$ (Rubinstein 1986, Glynn 1990, L’Ecuyer 1990):

$$\begin{aligned}\int f(x) \nabla_{\theta} p_{\theta}(x) dx &= \int f(x) p_{\theta}(x) \frac{\nabla_{\theta} p_{\theta}(x)}{p_{\theta}(x)} dx \\ &= \int p_{\theta}(x) f(x) \nabla_{\theta} \log p_{\theta}(x) dx \\ &= \mathbb{E}_{X \sim p_{\theta}}[f(X) \nabla_{\theta} \log p_{\theta}(X)].\end{aligned}$$

The right-hand side can be estimated from samples drawn from p_{θ} and does not require f to be differentiable, which distinguishes the estimator from the reparameterization methods of Section 2.6.

¹This derivation uses density and integral notation. For a discrete random variable, replace the integrals with sums and interpret p_{θ} as a probability mass function.

The policy gradient theorem gives a form of $\nabla_{\theta} J(\theta)$ that can be estimated from agent-environment interactions (Sutton et al. 1999, Nota & Thomas 2020).

Theorem 1 (Policy Gradient Theorem). *For a parameterized policy π_{θ} differentiable in θ ,*

$$\nabla_{\theta} J(\theta) \propto \mathbb{E}_{S \sim d_{\pi_{\theta}, \gamma}, A \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(A \mid S) q_{\pi_{\theta}}(S, A)],$$

where $d_{\pi_{\theta}, \gamma}$ is the normalized discounted state-visitation distribution. For a discrete state space, its probability mass function satisfies $d_{\pi_{\theta}, \gamma}(s) \propto \sum_{t=0}^{\infty} \gamma^t \Pr\{S_t = s, t < T \mid S_0 = s_0, \pi_{\theta}\}$. For a continuous state space, $d_{\pi_{\theta}, \gamma}$ denotes the corresponding discounted visitation density when one exists.

I use the shorthand $A \sim \pi_{\theta}$ to abbreviate $A \sim \pi_{\theta}(\cdot \mid S)$. The expectation contains the log-derivative form $\nabla_{\theta} \log \pi_{\theta}(A \mid S)$, the same form produced by the likelihood ratio estimator, weighted by the action value $q_{\pi_{\theta}}(S, A)$. Subsequent work made the discount factor’s role in the state distribution explicit (Nota & Thomas 2020), and some algorithms adopt the corrected form (Che, Vasan & Mahmood 2023).

Sample-based algorithms differ in how they estimate $q_{\pi_{\theta}}(S, A)$: REINFORCE uses the sampled return G_t (Williams 1992), actor-critic methods use a bootstrapped return computed with a learned state-value function (Sutton & Barto 2018), and Proximal Policy Optimization (PPO) uses a weighted sum of temporal-difference errors, paired with a clipped surrogate objective that constrains the policy update (Schulman et al. 2017).

REINFORCE

REINFORCE estimates the policy gradient from complete episodes (Williams 1992); it predates the policy gradient theorem and is recovered as an instance of it.

REINFORCE replaces $q_{\pi_{\theta}}(S_t, A_t)$ with the observed return G_t , which satisfies

$$\mathbb{E}_{\pi_{\theta}} [G_t \mid S_t, A_t] = q_{\pi_{\theta}}(S_t, A_t).$$

For the discounted start-state objective defined in Section 2.4, an unbiased gradient estimator from one episode is

$$\widehat{\nabla_{\theta} J} \doteq \sum_{t=0}^{T-1} \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(A_t \mid S_t).$$

The parameters are updated by $\theta \leftarrow \theta + \alpha \widehat{\nabla_{\theta} J}$, where $\alpha > 0$ is a step size. This estimator is unbiased but can have high variance.

Subtracting a baseline $b(S_t)$ from the return can reduce the variance:

$$\widehat{\nabla_{\theta} J} \doteq \sum_{t=0}^{T-1} \gamma^t (G_t - b(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t \mid S_t).$$

The estimator remains unbiased for any baseline that does not depend on the action, because $\mathbb{E}_{A \sim \pi_{\theta}} [b(S) \nabla_{\theta} \log \pi_{\theta}(A \mid S)] = 0$ (Williams 1992).

A natural choice is $b(S_t) = \hat{v}(S_t; \mathbf{w})$, an estimate of the state-value function under the current policy. Then $G_t - \hat{v}(S_t; \mathbf{w})$ estimates the advantage $\Psi_{\pi_{\theta}}(S_t, A_t)$.

Actor-Critic Methods

Actor-critic methods learn two components: a policy, called the actor, and a value function, called the critic. The actor selects actions, and the critic estimates state values. The temporal-difference error, computed from the critic’s estimates, evaluates the action after each transition and updates the actor. REINFORCE, in contrast, must wait until an episode ends to compute the return. Barto and colleagues introduced an early actor-critic architecture for adaptive control (Barto et al. 1983).

The simplest actor-critic method is *one-step actor-critic* (Sutton & Barto 2018).

It uses $R_{t+1} + \gamma \hat{v}(S_{t+1}; \mathbf{w})$ as a one-step estimate of the action value and subtracts $\hat{v}(S_t; \mathbf{w})$ to form the temporal-difference error. The temporal-difference error is an estimate of the advantage: it is unbiased when $\hat{v} = v_{\pi_\theta}$, and with an imperfect critic it is biased but has lower variance than the estimate based on the sampled return. Eligibility traces extend actor-critic methods beyond one-step updates: a trace is a short-term memory vector that accumulates recent gradients and decays them by $\gamma\lambda$ per timestep. $\text{AC}(\lambda)$ maintains one trace for the actor and one for the critic and updates each parameter vector in proportion to the temporal-difference error times its trace (Sutton & Barto 2018).

This dissertation modifies the one-step actor-critic of Sutton and Barto in two ways for use in the experiments. The first modification omits the factor γ^t that weights the actor update in an unbiased estimator of the discounted start-state policy gradient. The omission is common in practical implementations of one-step actor-critic (Pilarski et al. 2011, Degris et al. 2012a, Vasan & Pilarski 2017). The factor scales the update at timestep t by γ^t , which leaves little learning from states visited late in an episode, and omitting it works better in practice. The resulting actor update is not generally an unbiased estimator of $\nabla_{\theta} J(\theta)$ for the discounted start-state objective (Nota & Thomas 2020). I nevertheless use this practical update throughout the experiments. The updates are

$$\begin{aligned}\delta_t &\doteq R_{t+1} + \gamma \hat{v}(S_{t+1}; \mathbf{w}) - \hat{v}(S_t; \mathbf{w}), &> \text{temporal-difference error} \\ \mathbf{w} &\leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta_t \nabla_{\mathbf{w}} \hat{v}(S_t; \mathbf{w}), \\ \theta &\leftarrow \theta + \alpha^{\theta} \delta_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t),\end{aligned}$$

where $\alpha^{\mathbf{w}}$ and α^{θ} are the critic and actor step sizes, respectively. The critic uses the semi-gradient update, which treats the one-step target as fixed when updating $\mathbf{w}$.

The second modification adds entropy regularization to the actor update. During

learning, the action distribution may narrow before the critic has learned accurate value estimates, reducing exploration. The entropy of the policy at state s ,

$$\mathcal{H}(\pi_{\theta}(\cdot \mid s)) \doteq \mathbb{E}_{A \sim \pi_{\theta}} [-\log \pi_{\theta}(A \mid s)],$$

measures how spread out the action distribution is, and is differential entropy when the action space is continuous. *Entropy regularization* adds the gradient of the weighted entropy, $\eta \nabla_{\theta} \mathcal{H}(\pi_{\theta}(\cdot \mid S_t))$, to the actor update at each visited state, where $\eta \geq 0$ is a coefficient. The term discourages the action distribution from narrowing toward a single action. Likelihood ratio policy gradient methods often use entropy regularization to promote exploration (Mnih et al. 2016, Schulman et al. 2017). Setting $\eta = 0$ recovers the unregularized actor update above.

Algorithm 1 gives the pseudocode with both modifications.

Algorithm 1 One-Step Actor-Critic with an Entropy Bonus

Require: policy parameters θ ; state-value parameters $\mathbf{w}$; discount factor γ ;

entropy coefficient $\eta \geq 0$; step sizes $\alpha^{\mathbf{w}}$ and α^{θ}

- 1: **for** each episode **do**
 - 2: Initialize S (initial state)
 - 3: **while** S is not terminal **do**
 - 4: $A \sim \pi_{\theta}(\cdot \mid S)$
 - 5: Take action A , observe reward R and next state S'
 - 6: $\delta \leftarrow R + \gamma \hat{v}(S'; \mathbf{w}) - \hat{v}(S; \mathbf{w}) \quad \triangleright \hat{v}(S'; \mathbf{w}) \doteq 0$ if S' is terminal
 - 7: $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla_{\mathbf{w}} \hat{v}(S; \mathbf{w})$
 - 8: $\theta \leftarrow \theta + \alpha^{\theta} (\delta \nabla_{\theta} \log \pi_{\theta}(A \mid S) + \eta \nabla_{\theta} \mathcal{H}(\pi_{\theta}(\cdot \mid S)))$
 - 9: $S \leftarrow S'$
 - 10: **end while**
 - 11: **end for**
-

Proximal Policy Optimization

Proximal Policy Optimization (PPO) is an actor-critic method in which the policy is updated several times on each batch of experience (Schulman et al. 2017); it serves as a baseline in the experiments of this dissertation. The methods of this section so far use each transition once. PPO instead collects a batch with the current policy and runs several passes of stochastic gradient ascent over it, sweeping the batch in minibatches. The PPO variant used here omits the outer factor γ^t in the weighting of each update but retains γ in the λ -return and the value function.

PPO builds on one-step actor-critic with two changes. First, the advantage is estimated from many temporal-difference errors rather than one. Second, each batch is reused for several updates, with an objective that discourages moving the policy far from the one that collected the batch.

The one-step temporal-difference error inherits the full bias of an imperfect critic, and the sampled return has high variance. The advantage estimate in PPO interpolates between the two: a discounted sum of temporal-difference errors,

$$\hat{\Psi}_t \doteq \sum_{k=0}^{T-t-1} (\gamma\lambda)^k \delta_{t+k},$$

where $\lambda \in [0, 1]$ trades bias against variance: $\lambda = 0$ gives the one-step temporal-difference error, and $\lambda = 1$ recovers the sampled return (Schulman et al. 2016).

Reusing a batch creates a problem: each update moves the policy away from the policy that collected the batch, and the action probabilities under the two policies therefore differ. PPO accounts for the difference in its surrogate objective with the probability ratio

$$\rho_t(\boldsymbol{\theta}) \doteq \frac{\pi_{\boldsymbol{\theta}}(A_t \mid S_t)}{\pi_{\boldsymbol{\theta}_{\text{old}}}(A_t \mid S_t)},$$

where $\boldsymbol{\theta}_{\text{old}}$ denotes the parameters that collected the batch. The clipped objective compares the unclipped term $\rho_t(\boldsymbol{\theta})\hat{\Psi}_t$ with a term in which the ratio is clipped to

$[1 - \epsilon, 1 + \epsilon]$, where ϵ is a small positive constant, and keeps the smaller of the two.

On each minibatch $\mathcal{B}$ of size M , the policy parameters are updated by

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^\theta \nabla_{\boldsymbol{\theta}} \frac{1}{M} \sum_{i \in \mathcal{B}} \min \left(\rho_i(\boldsymbol{\theta}) \hat{\Psi}_i, \text{clip}(\rho_i(\boldsymbol{\theta}), 1 - \epsilon, 1 + \epsilon) \hat{\Psi}_i \right),$$

where $\mathcal{B}$ is a minibatch of timesteps drawn from the batch and i indexes those timesteps. Algorithm 2 gives the pseudocode for the PPO variant used in this dissertation.

Algorithm 2 Proximal Policy Optimization (Schulman et al. 2017)

Require: policy parameters $\boldsymbol{\theta}$; state-value parameters $\mathbf{w}$; discount factor γ ; clipping parameter ϵ ; advantage estimation parameter λ ; rollout length N ; number of epochs K ; minibatch size M ; step sizes $\alpha^{\mathbf{w}}$ and α^θ

- 1: **for** each iteration **do**
- 2: $\boldsymbol{\theta}_{\text{old}} \leftarrow \boldsymbol{\theta}; \quad \mathbf{w}_{\text{old}} \leftarrow \mathbf{w}$
 ▷ Collect Rollout
- 3: Collect N transitions under $\pi_{\boldsymbol{\theta}_{\text{old}}}$ and split them at episode boundaries into J contiguous trajectory segments. For $j = 1, \dots, J$, let segment τ_j contain L_j transitions:

$$D_{t+1}^{(j)} \leftarrow 1 \text{ if } S_{t+1}^{(j)} \text{ is terminal, and } 0 \text{ otherwise}$$

$$\tau_j \doteq \left(S_t^{(j)}, A_t^{(j)}, R_{t+1}^{(j)}, S_{t+1}^{(j)}, D_{t+1}^{(j)} \right)_{t=0}^{L_j-1},$$

$$\sum_{j=1}^J L_j = N.$$

 ▷ Compute Advantages and Critic Targets

- 4: **for** $j = 1, \dots, J$ **do**
- 5: $\hat{\Psi}_{L_j}^{(j)} \leftarrow 0$
- 6: **for** $t = L_j - 1, L_j - 2, \dots, 0$ **do**

```

7:       $\delta_t^{(j)} \leftarrow R_{t+1}^{(j)} + \gamma \left(1 - D_{t+1}^{(j)}\right) \hat{v} \left(S_{t+1}^{(j)}; \mathbf{w}_{\text{old}}\right) - \hat{v} \left(S_t^{(j)}; \mathbf{w}_{\text{old}}\right)$ 
8:       $\hat{\Psi}_t^{(j)} \leftarrow \delta_t^{(j)} + \gamma \lambda \left(1 - D_{t+1}^{(j)}\right) \hat{\Psi}_{t+1}^{(j)}$ 
9:       $Y_t^{(j)} \leftarrow \hat{\Psi}_t^{(j)} + \hat{v} \left(S_t^{(j)}; \mathbf{w}_{\text{old}}\right)$ 
10:    end for
11:  end for
    ▷ Update Actor and Critic
12:  Define the set of transition indices:
      
$$\mathcal{I} \doteq \{(j, t) \mid j \in \{1, \dots, J\}, \quad t \in \{0, \dots, L_j - 1\}\}, \quad |\mathcal{I}| = N.$$

13:  for  $k = 1, \dots, K$  do
14:    Randomly permute  $\mathcal{I}$  and partition it into minibatches of size at most  $M$ .
15:    for each minibatch  $\mathcal{B} \subseteq \mathcal{I}$  do
16:      for  $(j, t) \in \mathcal{B}$  do
17:
          
$$\rho_t^{(j)}(\boldsymbol{\theta}) \leftarrow \frac{\pi_{\boldsymbol{\theta}} \left(A_t^{(j)} \mid S_t^{(j)}\right)}{\pi_{\boldsymbol{\theta}_{\text{old}}} \left(A_t^{(j)} \mid S_t^{(j)}\right)}.$$

18:      end for
19:      Update the policy by gradient ascent on the clipped objective:
          
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \nabla_{\boldsymbol{\theta}} \frac{1}{|\mathcal{B}|} \sum_{(j,t) \in \mathcal{B}} \min \left( \rho_t^{(j)}(\boldsymbol{\theta}) \operatorname{sg} \left( \hat{\Psi}_t^{(j)} \right), \right. \\ \left. \operatorname{clip} \left( \rho_t^{(j)}(\boldsymbol{\theta}), 1 - \epsilon, 1 + \epsilon \right) \operatorname{sg} \left( \hat{\Psi}_t^{(j)} \right) \right).$$

20:      Update the state-value function by gradient descent:
          
$$\mathbf{w} \leftarrow \mathbf{w} - \alpha^{\mathbf{w}} \nabla_{\mathbf{w}} \frac{1}{2|\mathcal{B}|} \sum_{(j,t) \in \mathcal{B}} \left( \operatorname{sg} \left( Y_t^{(j)} \right) - \hat{v} \left( S_t^{(j)}; \mathbf{w} \right) \right)^2.$$

21:    end for
22:  end for
23: end for

```

2.6 Reparameterization Policy Gradient Methods

The *reparameterization gradient estimator*, also known as the pathwise gradient estimator, offers an alternative to the likelihood ratio estimator for the same target from Equation 2.4, $\nabla_{\theta} \mathbb{E}_{X \sim p_{\theta}}[f(X)]$ (Kingma & Welling 2014, Rezende et al. 2014, Parmas & Sugiyama 2021). The estimator rests on an equivalence between two ways of sampling from p_{θ} :

$$X \sim p_{\theta} \quad \Longleftrightarrow \quad X = g_{\theta}(Z), \quad Z \sim h, \quad (2.5)$$

where h is a distribution that does not depend on θ and g_{θ} is a differentiable function. Sampling X directly and transforming a sample of Z through g_{θ} produce the same distribution. The familiar case is the Gaussian: a sample $X \sim \mathcal{N}(\mu_{\theta}, \sigma_{\theta}^2)$ can be written as $X = \mu_{\theta} + \sigma_{\theta}Z$ with $Z \sim \mathcal{N}(0, 1)$. The randomness lives in Z , which does not involve θ , and the sample is a differentiable function of θ .

Let f be a differentiable real-valued function. Then,

$$\begin{aligned} \nabla_{\theta} \mathbb{E}_{X \sim p_{\theta}}[f(X)] &= \nabla_{\theta} \mathbb{E}_{Z \sim h}[f(g_{\theta}(Z))] &> \text{change of variables, } X = g_{\theta}(Z) \\ &= \mathbb{E}_{Z \sim h}[\nabla_{\theta} f(g_{\theta}(Z))] &> h \text{ independent of } \theta \end{aligned}$$

Sampling first and differentiating second has replaced differentiating a distribution: the noise supplies the randomness, and the parameters enter only through an ordinary differentiable function. The right-hand side is an expectation under a fixed distribution and can be estimated from samples of Z . The reparameterization gradient estimator often produces lower-variance gradient estimates than the likelihood ratio estimator (Fan et al. 2015, Mohamed et al. 2020). The estimator requires f to be differentiable; the likelihood ratio estimator does not.

The *reparameterization policy gradient* combines the reparameterization tech-

nique with a second ingredient, pathwise differentiation through the action into a differentiable critic. The deterministic policy gradient exhibits the second ingredient in isolation, because a deterministic action is already a differentiable function of the policy parameters (Silver et al. 2014).

Theorem 2 (Deterministic Policy Gradient Theorem). *Let μ_θ be a deterministic policy differentiable in θ , and suppose q_{μ_θ} is differentiable in the action. Then*

$$\nabla_\theta J(\theta) \propto \mathbb{E}_{S \sim d_{\mu_\theta, \gamma}} \left[\nabla_\theta \mu_\theta(S) \nabla_a q_{\mu_\theta}(S, a) \Big|_{a=\mu_\theta(S)} \right].$$

The gradient applies the chain rule to the composition $q_{\mu_\theta}(s, \mu_\theta(s))$. The factor $\nabla_\theta \mu_\theta$ captures how the action moves with the parameters, and the factor $\nabla_a q_{\mu_\theta}$ captures how the value responds to the action, which is why the theorem requires q_{μ_θ} to be differentiable in the action.

The reparameterization technique can be used to extend the same differentiation to stochastic policies. Writing a sampled action as $a = g_\theta(z; s)$ makes the action a differentiable function of the parameters, with the randomness isolated in the parameter-free noise z , so that the chain rule again passes through $\nabla_a q_{\pi_\theta}$. Combining the two gives the following theorem (Lan et al. 2022).

Theorem 3 (Reparameterization Policy Gradient Theorem). *Let π_θ be a stochastic policy whose actions are reparameterized as $a = g_\theta(z; s)$, where z is a realization of a noise vector with fixed distribution h independent of θ . Then*

$$\nabla_\theta J(\theta) \propto \mathbb{E}_{S \sim d_{\pi_\theta, \gamma}, A \sim \pi_\theta} \left[\nabla_\theta g_\theta(z; S) \Big|_{z=g_\theta^{-1}(A; S)} \nabla_a q_{\pi_\theta}(S, a) \Big|_{a=A} \right].$$

Here g_θ^{-1} denotes the inverse of the reparameterization map in its first argument, so that $z = g_\theta^{-1}(a; s)$ whenever $a = g_\theta(z; s)$; the result requires the inverse to exist and requires g_θ and q_{π_θ} to be differentiable in θ and in the action, respectively. The agent took action A in state S ; the inverse map recovers the noise z that produced

the action; the factor $\nabla_{\theta} g_{\theta}(z; S)$ gives how the action would have shifted had the parameters shifted, with the noise held fixed; and the factor $\nabla_a q_{\pi_{\theta}}$ tells how the value shifts when the action shifts.

Origins of differentiation through the critic. Policy improvement by differentiating a learned action-value estimate with respect to the action goes back to the adaptive critic designs of the neural-network control literature (Werbos 1990). In those designs, a critic network estimates the value, and an policy network that selects the action is adjusted along the derivative of that estimate with respect to the action. The designs were later organized into a family, including an action-dependent variant whose critic takes the action as input (Prokhorov & Wunsch 1997). The same derivative-based policy improvement appears within approximate dynamic programming (Powell et al. 2004). The deterministic policy gradient theorem (Theorem 2) gave the idea its exact form for deterministic policies. Combined with a replay buffer and target networks, it produced Deep Deterministic Policy Gradient (DDPG, Lillicrap et al. 2016) and Twin Delayed Deep Deterministic Policy Gradient (TD3, Fujimoto et al. 2018), described later in this section.

Origins of policy reparameterization. Reparameterization of the policy extends the same pathwise differentiation to stochastic policies and appears in the stochastic value gradient methods (Heess et al. 2015). SVG(0), the model-free member of the family, writes a Gaussian action as a function of noise and differentiates a learned action-value function through it, learning off-policy from a replay buffer in the manner of DDPG. Soft Actor-Critic (SAC) differentiates a learned action-value function through a reparameterized action in the same way, with an entropy term added (Haarnoja et al. 2018a). Neither work states a theorem for what the resulting gradient estimates. Both SVG and SAC use the estimator without a theorem stating what it estimates. Theorem 3 was supplied later with the discounted state

distribution made explicit, but no algorithm that follows it was proposed (Lan et al. 2022). Chapter 6 introduces Action Value Gradient (AVG), the first practical algorithm built on Theorem 3; it updates from each transition as it arrives, without storing any transition.

Twin Delayed Deep Deterministic Policy Gradient

Twin Delayed Deep Deterministic Policy Gradient (TD3) is an off-policy actor-critic method (Fujimoto et al. 2018); it serves as a baseline in the experiments of Chapter 6. An *off-policy* method learns about one policy from data generated by another and can therefore reuse experience collected by earlier versions of the policy. TD3 stores each transition in a replay buffer and updates on minibatches drawn from it. It learns a deterministic policy by following the deterministic policy gradient (Thm. 2, Silver et al. 2014) and explores by adding noise to the selected actions. The design extends Deep Deterministic Policy Gradient (DDPG), an earlier method that pairs the deterministic policy gradient with a replay buffer and target networks (Lillicrap et al. 2016).

When the critic is learned with bootstrapped targets and the policy is adjusted toward the actions with the highest estimates, estimation errors become systematic overestimation (van Hasselt 2010). TD3 counters the overestimation with three mechanisms. First, two critics are learned in parallel, and the bootstrap target is formed from the smaller of their two estimates. Taking the minimum reduces the propagation of a positive estimation error from either critic, although it can introduce underestimation. Second, small clipped noise is added to the action in the target, so that sharp peaks in the estimate at single actions are not copied into the targets. The two mechanisms combine in the shared critic target: on a minibatch $\mathcal{B}$ of size M with transitions $(S_i, A_i, R_i, S'_i, D_i)$ drawn from the replay buffer, where $D_i = 1$ if S'_i is terminal and $D_i = 0$ otherwise,

$$\begin{aligned}\tilde{A}'_i &\doteq \text{clip}_{\mathcal{A}}(\mu_{\bar{\theta}}(S'_i) + \text{clip}(Z_i, -c, c)), & Z_i &\sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}), \\ Y_i &\doteq R_i + \gamma(1 - D_i) \min_{j \in \{1, 2\}} \hat{q}(S'_i, \tilde{A}'_i; \bar{\mathbf{w}}_j),\end{aligned}$$

where $\text{clip}_{\mathcal{A}}$ clips each component of the target action to the corresponding action-space bounds. The target Y_i is treated as fixed when updating either critic. Each critic is updated by gradient descent,

$$\mathbf{w}_j \leftarrow \mathbf{w}_j - \alpha^{\mathbf{w}} \nabla_{\mathbf{w}_j} \frac{1}{M} \sum_{i \in \mathcal{B}} (Y_i - \hat{q}(S_i, A_i; \mathbf{w}_j))^2, \quad j \in \{1, 2\}.$$

Third, the policy is updated less frequently than the critics, by gradient ascent on the first critic's estimate,

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \nabla_{\boldsymbol{\theta}} \frac{1}{M} \sum_{i \in \mathcal{B}} \hat{q}(S_i, \mu_{\boldsymbol{\theta}}(S_i); \mathbf{w}_1),$$

so the policy improves against value estimates that have had time to stabilize.

Following each delayed policy update, the target networks move toward the current networks:

$$\begin{aligned}\bar{\mathbf{w}}_j &\leftarrow \tau \mathbf{w}_j + (1 - \tau) \bar{\mathbf{w}}_j, & j &\in \{1, 2\}, \\ \bar{\boldsymbol{\theta}} &\leftarrow \tau \boldsymbol{\theta} + (1 - \tau) \bar{\boldsymbol{\theta}},\end{aligned}$$

where $\tau \in (0, 1]$ is the target-network update coefficient and is typically close to zero.

Soft Actor-Critic

Soft Actor-Critic (SAC) is an off-policy actor-critic method that learns a stochastic policy by following the reparameterization policy gradient (Haarnoja et al. 2018a); it serves as a baseline in the experiments of this dissertation. Like TD3, SAC reuses experience collected by earlier versions of the policy.

The objective in SAC is modified to sustain exploration. A stochastic policy explores through the randomness of its own action selection, whereas a deterministic policy, as in TD3, requires externally injected noise. To keep the action distribution from narrowing toward a single action, each reward R_{t+1} is augmented with the weighted entropy of the policy, $\eta \mathcal{H}(\pi(\cdot \mid S_t))$, where η is a coefficient (Ziebart et al. 2010). The augmented rewards define the *soft* value functions,

$$\begin{aligned} v_{\pi}^{\text{Ent}}(s) &\doteq \mathbb{E}_{\pi} \left[\sum_{k=0}^{T-t-1} \gamma^k (R_{t+k+1} + \eta \mathcal{H}(\pi(\cdot \mid S_{t+k}))) \mid S_t = s \right], \\ q_{\pi}^{\text{Ent}}(s, a) &\doteq \mathbb{E}_{\pi} [R_{t+1} + \gamma v_{\pi}^{\text{Ent}}(S_{t+1}) \mid S_t = s, A_t = a], \end{aligned}$$

and the performance measure becomes $J(\theta) \doteq v_{\pi_{\theta}}^{\text{Ent}}(s_0)$. The entropy propagates through the value targets by bootstrapping; the entropy bonus of one-step actor-critic (see Algorithm 1), in contrast, modifies only the actor update at visited states.

The SAC actor update uses the pathwise derivative structure of Theorem 3. A sampled action is written as

$$A = g_{\theta}(Z; S), \quad Z \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

which makes the action a differentiable function of the policy parameters. On states sampled from the replay buffer $\mathcal{D}$, SAC performs gradient ascent on the surrogate

objective

$$\mathcal{L}_{\text{actor}}(\boldsymbol{\theta}) \doteq \mathbb{E}_{S \sim \mathcal{D}, Z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\min_{j \in \{1, 2\}} \hat{q}(S, g_{\boldsymbol{\theta}}(Z; S); \mathbf{w}_j) - \eta \log \pi_{\boldsymbol{\theta}}(g_{\boldsymbol{\theta}}(Z; S) \mid S) \right].$$

The gradient passes through the sampled action into the critics and also through the log-policy term. The critics estimate the soft action-value function, which includes entropy bonuses at future states, while the log-policy term supplies the entropy contribution at the current state.

Because the states are sampled from the replay buffer rather than from the discounted visitation distribution of the current policy, this update is not generally an unbiased estimator of $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$. It is an off-policy surrogate update that uses the same pathwise differentiation principle as the reparameterization policy gradient.

SAC uses three mechanisms to stabilize learning: twin critics, target critics, and random minibatches drawn from a replay buffer. The minimum of the two critic estimates reduces the propagation of positive estimation errors. Each target critic slowly tracks its corresponding current critic:

$$\bar{\mathbf{w}}_j \leftarrow \tau \mathbf{w}_j + (1 - \tau) \bar{\mathbf{w}}_j, \quad j \in \{1, 2\},$$

where τ is close to zero. The target critics therefore change more slowly than the critics being updated. Each update uses a random minibatch drawn from the replay buffer; Section 2.7 examines this update regime and its costs.

The SAC variant used in this dissertation learns the entropy coefficient rather than holding it fixed. The coefficient is parameterized as $\eta = \exp(\kappa)$ to ensure that it remains positive and is adjusted to keep the expected policy entropy near a target $\bar{\mathcal{H}}$. Following the common heuristic for continuous actions, the algorithm sets $\bar{\mathcal{H}} = -\dim(\mathcal{A})$. Algorithm 3 specifies the SAC variant used in this dissertation.

Algorithm 3 Soft Actor-Critic with Learned Entropy Coefficient (Haarnoja et al. 2018b)

Require: policy parameters θ ; action-value parameters $\mathbf{w}_1$ and $\mathbf{w}_2$; discount factor γ ; Polyak averaging coefficient τ ; entropy coefficient $\eta \doteq \exp(\kappa)$ with learnable parameter κ ; target entropy $\bar{\mathcal{H}} \doteq -\dim(\mathcal{A})$; minibatch size M ; number of gradient updates per transition U ; step sizes $\alpha^{\mathbf{w}}$, α^{θ} , and α^{κ}

- 1: Initialize replay buffer $\mathcal{D} \leftarrow \emptyset$
- 2: Initialize target parameters $\bar{\mathbf{w}}_j \leftarrow \mathbf{w}_j$ for $j \in \{1, 2\}$
- 3: **for** each episode **do**
- 4: Initialize S (initial state)
- 5: **while** S is not terminal **do**
- 6: Sample $A \sim \pi_{\theta}(\cdot \mid S)$
- 7: Take action A , observe reward R and next state S'
- 8: $D \leftarrow 1$ if S' is terminal, and 0 otherwise
- 9: Store (S, A, R, S', D) in $\mathcal{D}$
- 10: **if** $|\mathcal{D}| \geq M$ **then**
- 11: **for** $u = 1, \dots, U$ **do**
- 12: Sample M transitions from $\mathcal{D}$ and index them as

$$\mathcal{B} \doteq (S_i, A_i, R_i, S'_i, D_i)_{i=1}^M.$$

▷ Update Critics
- 13: **for** $i = 1, \dots, M$ **do**
- 14: $A'_i \sim \pi_{\theta}(\cdot \mid S'_i)$
- 15: $Y_i \leftarrow R_i + \gamma(1 - D_i) [\min_{j \in \{1, 2\}} \hat{q}(S'_i, A'_i; \bar{\mathbf{w}}_j) - \eta \log \pi_{\theta}(A'_i \mid S'_i)]$.
- 16: **end for**

```

17:   for  $j \in \{1, 2\}$  do
18:      $\mathbf{w}_j \leftarrow \mathbf{w}_j - \alpha^{\mathbf{w}} \nabla_{\mathbf{w}_j} \frac{1}{M} \sum_{i=1}^M (\text{sg}(Y_i) - \hat{q}(S_i, A_i; \mathbf{w}_j))^2.$ 
19:   end for

```

▷ Update Actor

```

20:   for  $i = 1, \dots, M$  do
21:      $\tilde{A}_i(\boldsymbol{\theta}) \doteq g_{\boldsymbol{\theta}}(Z_i; S_i), \quad Z_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
22:      $\hat{\mathcal{H}}_i \leftarrow -\log \pi_{\boldsymbol{\theta}}(\tilde{A}_i(\boldsymbol{\theta}) \mid S_i),$ 
23:   end for
24:   Update the policy by gradient ascent:

```

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \nabla_{\boldsymbol{\theta}} \frac{1}{M} \sum_{i=1}^M \left[\min_{j \in \{1, 2\}} \hat{q}(S_i, \tilde{A}_i(\boldsymbol{\theta}); \mathbf{w}_j) - \eta \log \pi_{\boldsymbol{\theta}}(\tilde{A}_i(\boldsymbol{\theta}) \mid S_i) \right].$$

▷ Update Entropy Coefficient

```

25:    $\kappa \leftarrow \kappa - \alpha^{\kappa} \nabla_{\kappa} \frac{1}{M} \sum_{i=1}^M e^{\kappa} \left( \text{sg}(\hat{\mathcal{H}}_i) - \overline{\mathcal{H}} \right).$ 

```

▷ Update Target Critics

```

26:   for  $j \in \{1, 2\}$  do
27:      $\overline{\mathbf{w}}_j \leftarrow \tau \mathbf{w}_j + (1 - \tau) \overline{\mathbf{w}}_j.$ 
28:   end for
29: end for
30: end if
31:    $S \leftarrow S'$ 
32: end while
33: end for

```

2.7 Streaming Versus Batch Reinforcement Learning

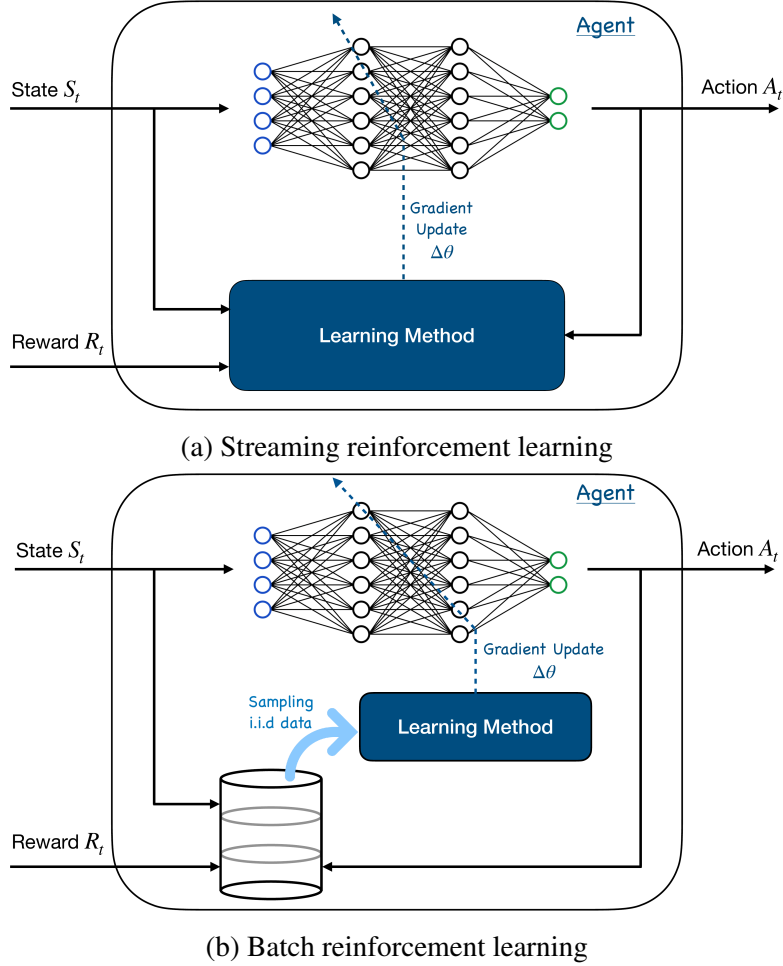


Figure 2.2: Inference and learning in agents under streaming and batch reinforcement learning.

In this dissertation, *streaming* and *batch* reinforcement learning denote update regimes rather than data-collection settings. The interaction process is the same in both regimes: at every timestep, the agent receives a state, selects an action, transitions to a new state, and receives a reward, yielding one *transition*, the tuple of state, action, reward, and next state. In streaming reinforcement learning, the agent performs one parameter update from each transition and then discards it (Fig-

ure 2.2a). In batch reinforcement learning, the agent stores transitions and computes parameter updates from minibatches of stored experience (Figure 2.2b). Streaming methods are also called *incremental* methods in the literature; this dissertation uses streaming throughout. Batch reinforcement learning in this sense is distinct from *offline reinforcement learning*, in which the agent learns from a fixed dataset and collects no further experience. Every agent in this dissertation continues to collect experience during learning.

Streaming methods are attractive for real-time learning on physical robots because they require little memory and interleave one parameter update with each environment transition. When the agent learns and acts in the same thread, the interval between actions bounds the computation available for each update. If the update fits within this interval, the agent can update on one transition and discard it before selecting the next action. Deep RL methods commonly use minibatch rather than streaming updates because updating a neural network from individual, temporally correlated transitions can destabilize learning. Consecutive transitions are temporally correlated, which can increase the variance of gradient estimates and destabilize stochastic optimization. Streaming deep RL methods often learn slowly, plateau at poor solutions, or diverge outright (Riedmiller 2005a, Mnih et al. 2015).

Deep RL relies on batch updates and large replay buffers for its strong results in challenging domains such as games (Mnih et al. 2015) and robotics (Ibarz et al. 2021, Haarnoja et al. 2018a). Experience replay stores each transition in a replay buffer and reuses it across many updates rather than discarding it after one use (Lin 1992). Sampling each minibatch uniformly at random from the buffer decorrelates the training samples and approximately satisfies the i.i.d. assumption. Averaging updates over minibatches also lowers their variance and makes learning more stable (Riedmiller 2012, Goodfellow et al. 2016). The regime borrows the training practice of the fixed-dataset setting: the buffer stands in for the dataset. An

early demonstration was neural fitted Q-iteration (NFQ; Riedmiller 2005a), which recast value learning as supervised training on stored transitions and learned reliably from reasonably few interactions.

The stability of batch methods comes at a cost in memory and computation. Storing transitions occupies memory, and each update sweeps a minibatch rather than a single sample. Figure 2.3 categorizes the policy gradient methods of this chapter by update regime, streaming or batch, and by policy gradient estimator, likelihood ratio or reparameterization.

	Streaming	Batch
Likelihood Ratio Policy Gradients	One-Step Actor Critic $AC(\lambda)$	REINFORCE PPO
Reparameterization Policy Gradients	AVG (Contribution)	SAC DDPG TD3

Figure 2.3: Categorization of the policy gradient methods discussed in this chapter along two axes: streaming versus batch learning, and likelihood ratio versus reparameterization gradients. Action Value Gradient (AVG) of Chapter 6 is a streaming deep reinforcement learning method contributed by this dissertation.

Chapter 3

Real-Time Reinforcement Learning with Physical Robots

Real-time reinforcement learning is a setting in which a learning agent runs, not a new problem formulation. The setting has three defining facets: (a) the environment continues to evolve while the agent computes, making elapsed computation time also elapsed environment time; (b) actions are due at target intervals, imposing a deadline on the computations that produce each action; and (c) the agent continues learning from runtime experience rather than executing only a fixed-parameter policy. The *action cycle time* is the prescribed wall-clock period between consecutive action deadlines. The reinforcement learning formalism neither enforces nor prohibits these facets.

This chapter surveys real-time reinforcement learning on physical robots. Work that relaxes any facet falls outside the chapter’s scope: a simulator that pauses while the agent computes relaxes the first facet, a task without action deadlines relaxes the second, and a train-then-deploy approach that executes only a fixed-parameter policy relaxes the third. Warm-starting an agent with pretrained weights remains within scope if learning continues during operation.

The three facets make the placement of computation a design decision with a cost. Every operation between an observation and its action adds delay the agent cannot recover. A learning update placed before action selection delays the action

by the duration of the update. A learning update placed on a separate thread does not delay the action, and the agent then selects actions using parameters that lag those held by the learner. Every real-time learning system on a robot resolves the tradeoff somehow, and the resolution has always been constrained by what the available hardware could compute within one action cycle.

Choosing the action cycle time is a consequential part of task design because the cycle bounds available computation. It should reflect the timescale of the controlled dynamics, the operating rates of the sensors and actuators, and the available computational resources. A shorter cycle permits more frequent actions but leaves less time for computation, whereas a longer cycle provides more computation time but reduces responsiveness.

Across three decades of prior work, the learning update moved further from the thread that selects actions as learning methods became more computationally expensive. Early methods updated one sample at a time inside that thread because a linear update finished well within the action cycle time. Neural networks were unstable when updated one sample at a time. The later methods therefore stored past transitions in a buffer and averaged each update over a minibatch drawn from it. An update averaged over a minibatch took longer than the action cycle time. The update therefore moved out of the thread, first between episodes, then into a separate thread, and finally onto a remote workstation serving one robot or many.

I organize the survey along three axes: policy/value function class (tabular, linear, instance-based, or neural network), update regime (streaming or batch), and update scheduling (synchronous, asynchronous, or inter-episode). The taxonomy separates the form and update regime of the policy or value function from when and where updates run. To compare computational cost, I estimate floating-point operations (FLOPs) per update and plot the estimates chronologically for the surveyed methods where estimation was possible (Fig. 3.3).

3.1 Ordering of Computations for Real-Time Reinforcement Learning

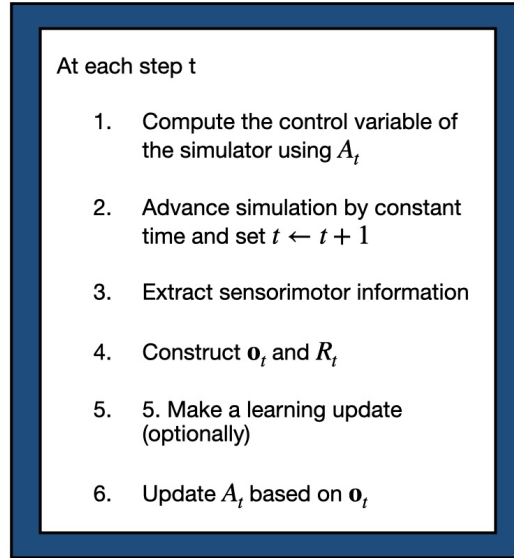


Figure 3.1: Sequential computations in an agent-synchronized simulated task.

The contrast between an agent-synchronized simulation and the concurrent computations of a physical robot shows where delay enters a learning task. An agent-synchronized simulator pauses while the agent computes (e.g., MuJoCo [Todorov et al. 2012]), whereas wall-clock-synchronized (e.g., Physical Atari [Javed et al. 2026]) and free-running simulators continue independently of the agent, at a real-time rate and a compute-dependent rate, respectively. At each timestep, an agent-synchronized simulator applies the current action, advances its clock, and returns the new sensorimotor reading. The agent then constructs its observation and reward, optionally updates its parameters, and selects the next action (Fig. 3.1). Each operation completes before the next begins.

The sensors, actuators, and environment of a physical robot operate concurrently and often at different rates. A sensor reading describes the environment at the time of measurement. Communication and processing create observation-to-action latency, during which the measurement becomes increasingly stale. Process-

ing pipelines such as those used for image-based control can extend this latency beyond one action cycle. Waiting for a newer observation does not eliminate latency because the environment continues to evolve while the agent waits. By contrast, latency introduced by placing a learning update before action selection can be reduced by issuing the action first and deferring the update (Travnik et al. 2018).

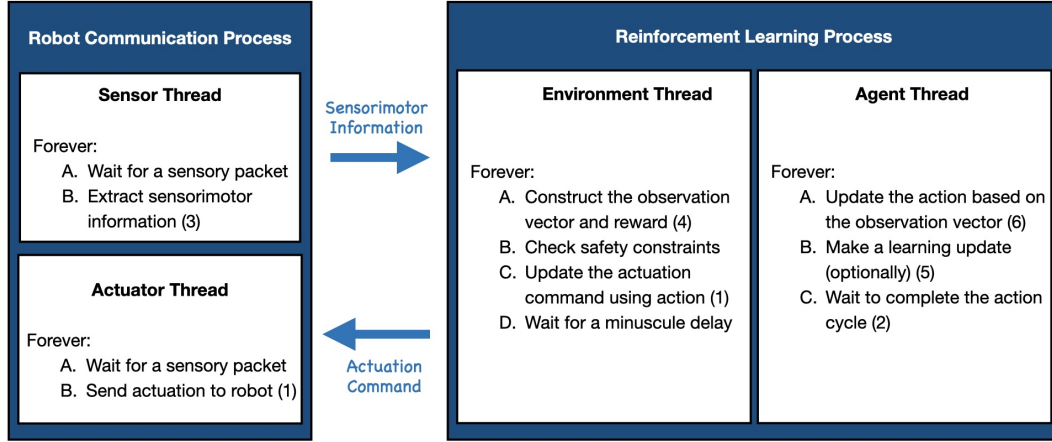


Figure 3.2: Concurrent computations in real-world tasks.

Two approaches compensate for remaining observation-to-action latency: augmenting the observations with recent actions (Katsikopoulos & Engelbrecht 2003, Ramstedt & Pal 2019) and constructing an internal agent state from a model (Walsh et al. 2009). Mahmood and colleagues instead examined the avoidable latency introduced by the organization of a physical learning task and reordered its computational stages to reduce that latency (Mahmood et al. 2018a). The task computations divide across two concurrent processes, one for robot communication and one for the learning agent, which exchange sensorimotor packets and actuation commands (Fig. 3.2). Sensor and actuator threads within the communication process run at the rates of their devices. Within the learning process, an environment thread constructs observations and rewards, while an agent thread selects actions by their deadlines. The resulting latency depends on where each operation runs and in what order.

SenseAct is a computational framework that orders the computations of a learn-

ing task and distributes them across concurrent processes (Mahmood et al. 2018a). It provides sensorimotor interfaces and communication through TCP and shared memory, while allowing the experimenter to assign computations to processes and specify the order of those computations. The shared structure supports reproducible experiments across robot platforms while leaving the learning algorithm and placement of its updates to the experimenter. The scheduling axis of the survey classifies prior work by the latter choice (Table 3.1). The robot experiments in this dissertation use SenseAct to set up real-time learning tasks.

3.2 Prior Work: Learning in Real Time With Physical Robots

The rows of Table 3.1 classify each surveyed work by the class of its policy or value function and by its update regime. Tabular, linear, and instance-based functions operate over designer-specified encodings: a state enumeration, a feature vector, or a similarity metric, respectively. A neural network instead learns its internal representation from data. An instance-based function stores examples and predicts from similar stored examples under a specified metric, rather than fitting a single global parametric function. Rows classify the policy or value function, not a learned dynamics model; model classes are identified separately in the prose.

A policy or value update regime is *streaming* when each new transition is used once, at the time it arrives, and never stored for later policy or value updates. An update is *batch* when it uses experience beyond the new transition, drawn from rollouts, datasets, replay minibatches, stored examples, or minibatches of model-generated transitions. I classify methods with instance-based functions as batch because the function itself retains experience, and its updates draw on experience beyond the current transition. Instance-based functions appear only in the batch rows.

Symbols mark how the learning update is scheduled relative to the agent thread in Figure 3.2: synchronous ($\circ$; learning updates interleave with action computations in the agent thread, every timestep or every few timesteps), asynchronous ($\bullet$; learning updates run in a separate thread, concurrently with action computations), or inter-episode ($\bullet$; learning updates run between episodes, when no action computations occur). A reference appears in more than one row when the work reports learning setups with different function classes or update schedules.

Real-time reinforcement learning on physical robots dates back more than three decades. Early examples include a mobile robot that learned component behaviors for box pushing and a ball-and-beam system that learned to keep the ball away from either end of the beam (Mahadevan & Connell 1991, Benbrahim et al. 1992). Both agents updated their parameters on the robot once per timestep, demonstrating that streaming updates are feasible on physical hardware. Updating every timestep requires an update cheap enough to finish before the next action is due, a constraint both agents met with the modest computers of the period.

Streaming RL methods with linear function approximation were applied across a range of robot platforms and tasks. Mobile robot navigation and behavior learning formed the largest subset (Dorigo & Colombetti 1994, Matarić 1994, Asada et al. 1996, Matarić 1997, Kelly & Keating 1998, Willgoss & Iqbal 1999). Locomotion work spanned quadruped gaits (Kimura et al. 2001), bipedal walking (Benbrahim & Franklin 1997, Tedrake et al. 2004, Tedrake et al. 2005, Endo et al. 2008), standing up (Morimoto & Doya 2001), and crawling (Tokic et al. 2009, Nemec et al. 2010). A smaller subset tackled manipulation and grasping (Huber & Grunpen 1997, Fagg et al. 1998, Platt et al. 2006). In a separate body of work, agents learned many predictions in parallel with TD learning and tile coding, and control policies alongside them in some cases (Sutton et al. 2011, Modayil et al. 2014, Edwards et al. 2014, Sherstan et al. 2015, Günther et al. 2016). Adaptive prosthetic control

formed a distinct application area, where the agent learned from a human feedback (Pilarski et al. 2011, Pilarski et al. 2013, Mathewson & Pilarski 2017, Vasan & Pilarski 2017, Vasan 2017, Vasan & Pilarski 2018).

A central limitation of streaming methods with linear function approximation was their dependence on designer-specified features. A linear function can represent only the relationships encoded by those features, and constructing expressive features becomes difficult as observations and actions grow in dimensionality (Kober et al. 2013). Some work on multi-joint robots reduced the action space faced by each policy by assigning a separate policy to each joint (Degris et al. 2012a, Vasan & Pilarski 2017). Neural networks address the representation problem differently by learning internal features from data rather than relying entirely on features supplied by the designer.

Early work applied streaming RL with neural networks to physical robots for peg-in-hole insertion (Gullapalli et al. 1994), mobile robot navigation (Thrun 1995, Millán 1996, Touzet 1997, Hailu & Sommer 1998, Gaskett et al. 2000), and bipedal walking (Geng et al. 2005). A single-transition update does not average variation across samples in the way that a minibatch update does. Single-transition updates challenge neural networks in particular because an update can change both the network’s predictions and its learned internal representation, thereby changing the features on which subsequent updates depend. Linear approximation with fixed features does not undergo this representation drift, although linear methods are not necessarily stable in every setting. Later work in the survey largely favored batch updates and experience replay for learning with neural networks. The Action Value Gradient algorithm developed in this dissertation revisits streaming RL with neural networks and learns stably on physical robots (Vasan et al. 2024b).

Table 3.1: Real-time reinforcement learning on physical robots, organized along three axes: policy/value function class (tabular, linear, instance-based, or neural network) and policy/value update regime (streaming or batch) for rows, and update scheduling (synchronous, asynchronous, or inter-episode) for symbols. References in [blue](#) are the contributions of this dissertation.

○ Synchronous:	learning updates interleave with action computations in the agent thread, every timestep or every few timesteps
● Asynchronous:	learning updates run in a separate thread, concurrently with action computations
⦿ Inter-episode:	learning updates run between episodes, when no action computations occur
Category	References
Streaming policy/ value updates with tabular or linear functions	○ Mahadevan & Connell (1991), Benbrahim et al. (1992), Dorigo & Colombetti (1994), Matarić (1994), Asada et al. (1996), Yamaguchi et al. (1996), Benbrahim & Franklin (1997), Huber & Grupen (1997), Matarić (1997), Fagg et al. (1998), Kelly & Keating (1998), Willgoss & Iqbal (1999), Kimura et al. (2001), Morimoto & Doya (2001), Tedrake et al. (2004), Tedrake et al. (2005), Platt et al. (2006), Soni & Singh (2006), Paletta et al. (2007), Endo et al. (2008), Tamei & Shibata (2008), Yasuda & Ohkura (2008), Tokic et al. (2009), Barrett et al. (2010), Nemec et al. (2010), Konidaris et al. (2011), Piater et al. (2011), Pilarski et al. (2011), Sutton et al. (2011), Konidaris et al. (2012), Degris et al. (2012a), Pilarski et al. (2013), Edwards et al. (2014), Modayil et al. (2014), Shertan et al. (2015), Günther et al. (2016), Mathewson & Pilarski (2017), Vasan & Pilarski (2017), Vasan (2017), Travník et al. (2018), Travník (2018), Vasan & Pilarski (2018), Lupu et al. (2026)
Streaming policy/ value updates with neural networks	○ Gullapalli et al. (1994), Thrun (1995), Millán (1996), Touzet (1997), Hailu & Sommer (1998), Gaskett et al. (2000), Geng et al. (2005), Vasan et al. (2024b)

Continued on next page

Table 3.1 (continued)

Category	References
Batch policy/ value updates with tabular, linear, or instance-based functions	○ Kalmár et al. (1997), Schaal (1996), Smart & Kaelbling (2002), Morimoto et al. (2005) ● Bagnell & Schneider (2001), Fiedelman & Stone (2004), Kohl & Stone (2004), Kwok & Fox (2004), Kober et al. (2008a), Kober & Peters (2008b), Peters & Schaal (2008), Vlassis et al. (2009), Kober et al. (2010), Theodorou et al. (2010), Kormushev et al. (2010a), Kormushev et al. (2010b), Buchli et al. (2011), Deisenroth et al. (2011), Kalakrishnan et al. (2011), Pastor et al. (2011), Tamosiunaite et al. (2011), Stulp & Sigaud (2012), Daniel et al. (2016) ● Hester et al. (2012), Hester & Stone (2013), Caarls & Schuitema (2015)
Batch policy/ value updates with neural networks	○ Mahmood et al. (2018a), Mahmood et al. (2018b), Javed et al. (2026), Lupu et al. (2026) ● Hafner & Riedmiller (2003), Riedmiller (2005b), Riedmiller et al. (2007), Riedmiller et al. (2009), Lange et al. (2012), Chebotar et al. (2017), Mahmood et al. (2018a), Mahmood et al. (2018b), Korenkevych et al. (2019), Nair et al. (2020), Smith et al. (2022), Che, Vasan & Mahmood (2023), Feng et al. (2023), Wang, Vasan & Mahmood (2023), Lan, Vasan & Mahmood (2025) ● Gu et al. (2017), Yahya et al. (2017), Kalashnikov et al. (2018), Riedmiller et al. (2018), Haarnoja et al. (2019), Schwab et al. (2019), Yang et al. (2019), Ha et al. (2020), Hertweck et al. (2020), Bloesch et al. (2022), Hafner et al. (2021), Julian et al. (2021), Yuan & Mahmood (2022), Smith et al. (2023), Wu et al. (2023), Bi & D’Andrea (2024), Grooten et al. (2024), Luo et al. (2024), Smith et al. (2024), Wang, Vasan & Mahmood (2023), Vasan et al. (2024a), Chen et al. (2025), Farrahi & Mahmood (2025), Luo et al. (2025), Shahriar et al. (2026)

Storing or generating experience for additional updates allows a method to perform more learning computation per environment transition and can improve data efficiency, at the cost of additional memory and computation (Moore & Atkeson 1993). In the Dyna architecture, the agent learns a model and generates simulated transitions from it for additional updates (Sutton 1991). Experience replay reuses stored transitions to stabilize learning (Lin 1992). Both update the policy or value function using experience beyond the current transition and are therefore batch under the definition used here. TEXPLORE brought this pattern to real-time control on a physical robot: the agent thread queues each transition, a background thread refits a random-forest dynamics model, and a planning thread runs UCT(λ) rollouts to update a tabular Q-function over a discretized state space (Hester et al. 2012, Hester & Stone 2013). Caarls and Schuitema used a related architecture, with locally weighted linear regression over a k-d tree of stored transitions for the model and fourteen planning threads using lock-free writes to update a tile-coded value function (Caarls & Schuitema 2015). These systems illustrate three recurring features of real-time batch RL on robots: retained or generated experience, concurrent learning updates, and computational pipelines tailored to the platform.

Inter-episode update scheduling confines policy or value updates to episode boundaries: the agent executes one or more episodes and then updates from the collected experience. The policy or value function remains fixed within each episode and cannot incorporate new experience until the episode ends. Early examples include autonomous helicopter flight (Bagnell & Schneider 2001) and quadruped gait learning (Fidelman & Stone 2004, Kohl & Stone 2004, Kwok & Fox 2004). A larger body of work parameterized policies as Dynamic Movement Primitives, a trajectory representation with tunable weights (Ijspeert et al. 2002), and learned those weights with policy-gradient or path-integral methods (Kober et al. 2008a, Kober & Peters 2008b, Peters & Schaal 2008, Vlassis et al. 2009, Kober et al.

2010, Theodorou et al. 2010, Buchli et al. 2011, Tamosiunaite et al. 2011, Stulp & Sigaud 2012, Daniel et al. 2016).

An early example of inter-episode batch learning with value functions parameterized by neural networks appeared in soccer-robot control, where the value function was fitted to stored transitions between episodes (Hafner & Riedmiller 2003). Riedmiller developed the pattern into Neural Fitted Q-iteration, a general method that collects transitions during an episode and refits the Q-function to the stored data between episodes (Riedmiller 2005b, Riedmiller et al. 2007, Riedmiller et al. 2009). Later variants learned representations with autoencoders (Lange et al. 2012) and combined model-based trajectory optimization with policy learning (Chebotar et al. 2017). In the surveyed robot implementations, on-policy actor-critic methods such as trust region policy optimization (TRPO; Schulman et al. 2015) and proximal policy optimization (PPO) follow a related schedule: they collect rollouts while the robot acts and update their policy and value functions between episodes (Mahmood et al. 2018a, Mahmood et al. 2018b, Korenkevych et al. 2019, Che, Vasan & Mahmood 2023).

Asynchronous batch RL with neural networks runs policy or value updates concurrently with robot behavior in a separate thread or process, typically using transitions stored in a replay buffer. One line of work used a central learner that received experience from multiple robots and returned updated policies to them (Gu et al. 2017, Yahya et al. 2017, Kalashnikov et al. 2018). Another paired a single robot with a workstation-class learner to acquire continuous-control policies on manipulators (Riedmiller et al. 2018, Schwab et al. 2019, Hertweck et al. 2020) and legged robots (Haarnoja et al. 2019, Yang et al. 2019, Ha et al. 2020, Hafner et al. 2021, Smith et al. 2022, Smith et al. 2023, Smith et al. 2024). In some of these systems, the robot selected actions onboard and synchronized policy parameters with the remote learner only between episodes, even though learning ran concur-

rently. A third line extended asynchronous learning to control from image observations, using model-free methods (Bloesch et al. 2022, Yuan & Mahmood 2022) and model-based methods that learned a forward model alongside the policy (Wu et al. 2023). Two contributions of this dissertation belong to this category: ReLoD partitions computation between a robot and a remote workstation for vision-based control (Wang, Vasan & Mahmood 2023), and the minimum-time reward study uses an asynchronous learner for goal-reaching tasks (Vasan et al. 2024a).

3.3 Discussion

Among the systems for which per-update computation could be estimated, batch methods with neural networks grew substantially more expensive over time (Fig. 3.3). Their estimated costs exceed those of streaming linear methods by as much as ten orders of magnitude, whereas streaming linear methods remain within a few orders of magnitude of their 1990s costs. The growth in per-update cost coincides with increasingly capable CPUs, GPUs, and distributed clusters, as well as the broader increase in computing capacity associated with Moore’s law (Moore 1965). The estimates measure computation per update, however, not update frequency, total training computation, runtime, or whether an update meets a particular action deadline.

Although computing capacity has doubled roughly every two years in accordance with Moore’s law, on-device real-time learning remains rare among the surveyed systems. Each hardware era in Figure 3.3 brought methods that used the capacity of that era, so that the newest methods remained beyond the onboard computers of their day.

Table 3.1 shows a corresponding scheduling pattern: computationally expensive batch updates are often performed between episodes or by asynchronous learners,

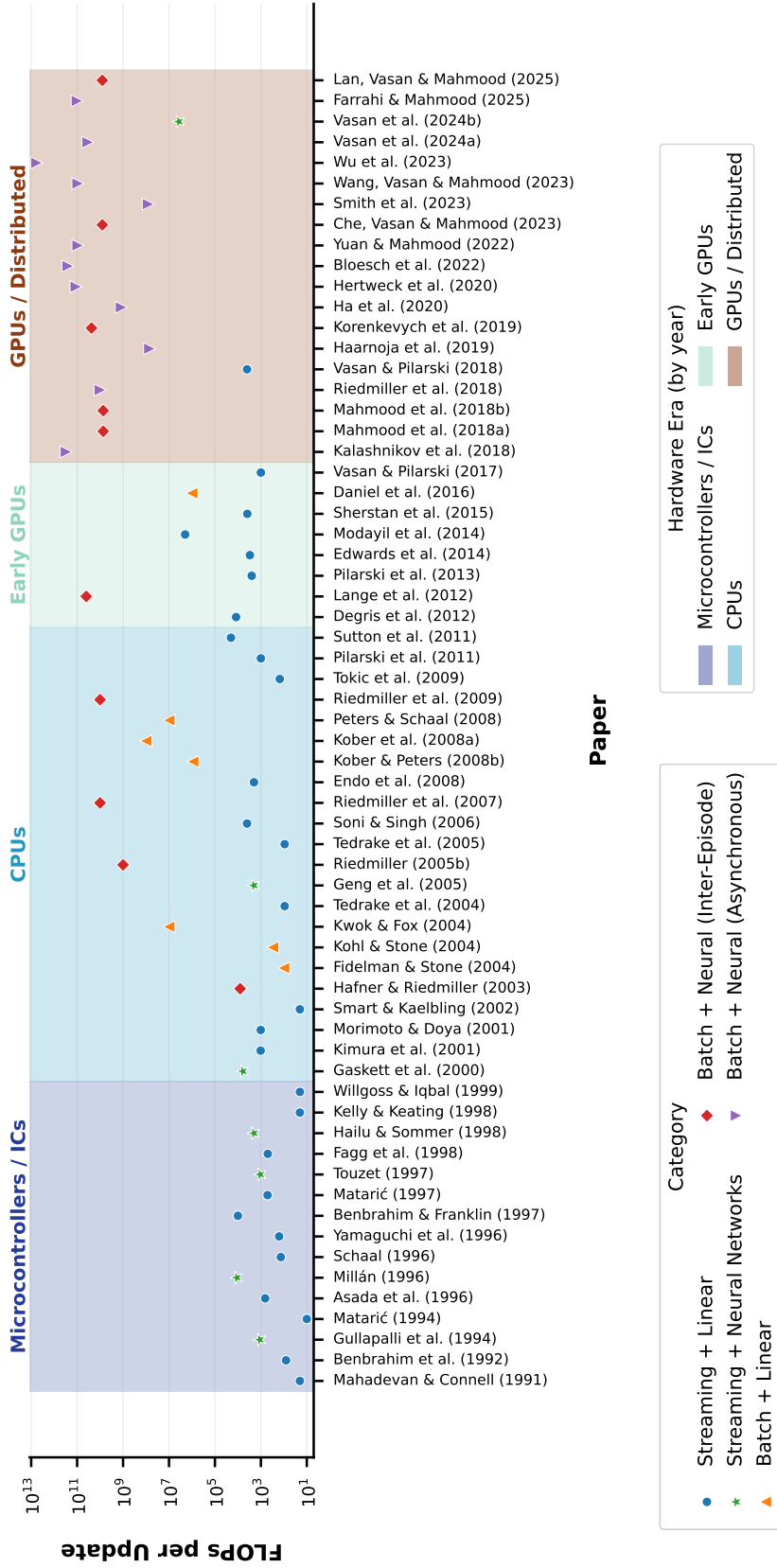


Figure 3.3: The figure covers a subset of the surveyed work through 2025 for which per-update computation could be estimated; Table 3.1 lists the full set. Each point gives the estimated floating-point operations per learning update for one system, plotted on a logarithmic scale in chronological order. Marker shape and color denote the learning regime. Background shading denotes the dominant hardware era by publication year. Streaming methods with linear function approximation remain within a few orders of magnitude of their 1990s cost, whereas batch methods with neural networks grow with available hardware, exceeding streaming linear methods by up to ten orders of magnitude. Appendix A.2 describes the estimation method.

sometimes on remote hardware. Inter-episode and asynchronous scheduling keep the update out of the agent thread and allow it to use more computation, but they may require stored experience, concurrent execution, or communication with an external learner. This dissertation responds to the scheduling pattern in two ways.

The first response accepts the computational demands of batch deep RL and asks how its computations should be divided between the robot and remote hardware. The asynchronous systems in the survey use different, platform-specific divisions, but each paper generally evaluates only one division between local and remote computation. Within the physical-robot literature surveyed here, I did not find a systematic comparison of what should run on the robot, what should run remotely, and how the two should communicate. Chapter 4 presents the Remote-Local Distributed (ReLoD) framework, which distributes action computation and learning updates between a robot’s resource-limited onboard computer and a remote workstation while preserving real-time operation.

The second response retains the streaming update regime and addresses the instability that limited its use with neural networks. In Figure 3.3, single-transition updates remain far less expensive than the largest batch updates with neural networks. For a fixed-size parametric method that does not replay experience or expand a stored representation, memory and per-update computation also remain constant as the interaction history grows. Chapter 6 presents Action Value Gradient (AVG), a streaming actor-critic algorithm that updates from each new transition without a replay buffer, auxiliary learning threads, or planning. In the simulated and physical-robot experiments reported in that chapter, AVG remains stable in settings where the standard streaming methods I evaluated diverge or fail.

The two responses make different tradeoffs. ReLoD retains batch deep-RL algorithms by allocating computation between the robot and remote hardware; configurations that place learning remotely therefore depend on a reliable connection and

sufficient remote capacity. AVG instead reduces memory and computation by eliminating replay and batch updates, enabling on-device real-time learning in the settings evaluated in this dissertation. A method that learns within onboard resources removes the dependence on external infrastructure altogether: a robot could learn wherever it operates, for as long as it operates, and the memory and computation that replay and batch updates would have consumed become available for learning other skills and knowledge. The minimum-time reward study uses an asynchronous learner but addresses reward design rather than the allocation or reduction of learning computation, and is therefore separate from these two responses.

Chapter 4

Real-Time Reinforcement Learning for Vision-Based Robotics Utilizing Local and Remote Computers

Modern deep reinforcement learning methods rely on abundant computation, which limits their use on physical robots. For decades, computing hardware improved at the exponential pace described by Moore’s law (Moore 1965), and RL methods grew to consume the resulting computation, a trend documented in Chapter 3 (Fig. 3.3). Virtual agents, such as those playing computer games, benefited from the increased computation because their environments are simulated: a simulator can run on the same hardware as the learner, be replicated across machines to generate experience in parallel, and be sped up beyond real time (Nair et al. 2015). Real-time learning on physical robots cannot exploit the increased computation as easily. The environment is the physical world, which cannot be replicated or accelerated. The robot therefore learns from a single stream of experience that arrives in real time. Computation is constrained as well: a mobile robot must carry its computer onboard, and the weight, size, and power that a robot can spare limit that computer to a small, power-efficient device.

One remedy pairs two computers during learning: a resource-limited local computer, carried by or tethered to the robot, and a powerful remote computer connected wirelessly. The Perseverance Mars rover is an extreme instance of the setup. The

rover operates under stringent limits on onboard computation and storage (Verma et al. 2023), and its remote computers, workstations on Earth, sit behind a slow, intermittent, low-bandwidth connection. The setup trades resources against delay: a computation placed on the local computer contends with limited resources, and a computation placed on the remote computer incurs wireless communication delays. Deciding which computations run on the rover and which run on Earth is therefore a central design concern. Prior systems distributed computations between the two computers in platform-specific ways, and each paper evaluated only one division between local and remote computation (Haarnoja et al. 2019, Bloesch et al. 2022). Within the physical-robot literature surveyed in Chapter 3, I did not find a systematic comparison of what should run on the robot, what should run remotely, and how the two should communicate.

I introduce Remote-Local Distributed (ReLoD), a real-time RL framework for vision-based control that supports systematic study of how to distribute learning computations between local and remote computers. ReLoD is a computational framework: an integrated collection of software components that runs on the two computers together and executes a real-time RL method on a physical robot, spanning sensing, action computation, learning updates, and inter-computer communication. The framework, a learning method, particular computers, and a particular robot together instantiate a real-time learning system. Deep RL methods involve three computationally expensive operations: action computation, policy and value function updates, and minibatch sampling. ReLoD runs each operation as a separate process on either computer. In each comparison, the learning method, the task, the robot, and the computers stay fixed, and I vary only where the processes run. Varying one thing at a time makes the comparison systematic and distinguishes the framework from any single system built with it. The robot experiments use SenseAct, which orders the computations of a learning task across concurrent pro-

cesses but does not prescribe where the agent’s computations run (Mahmood et al. 2018a); ReLoD distributes them between the two computers.

The experiments use two deep RL methods with contrasting computational profiles: Proximal Policy Optimization (PPO; Schulman et al. 2017) and Soft Actor-Critic (SAC; Haarnoja et al. 2018a). An agent using PPO stores the rollouts of a few recent episodes, updates its policy and value functions from the stored rollouts between episodes, and then discards them. An agent using SAC stores transitions in a large replay buffer and updates its policy and value functions from minibatches drawn from the buffer, in a process that runs concurrently with action computation. In the update scheduling terms of Chapter 3, the updates of an agent using PPO are inter-episode and the updates of an agent using SAC are asynchronous. The rate at which the robot generates experience bounds the updates of PPO, because each update consumes only the newly collected on-policy data. The replay buffer of SAC, by contrast, retains old experience for off-policy reuse, and additional computation therefore yields more updates from the same stream. The bound and the schedule together make PPO a control: additional computation cannot increase its updates, and its inter-episode updates run when no action computations occur. Moving the updates between computers therefore cannot change the actions taken within an episode. Any benefit of a remote computer to learning updates should therefore appear for SAC and not for PPO.

Using ReLoD, I investigate three questions in sequence. First, given ample computation, which agent attains higher return in real time, and does the advantage persist under limited computation? The SAC agent attained more than twice the return of the PPO agent within the same wall-clock time, and on the weakest computer its return fell below that of the PPO agent. Second, can distributing action computation and learning updates between the two computers recover the lost return? The SAC agent, computing actions on the local computer and updating

on the remote computer, recovered most of the lost return, whereas the PPO agent attained similar returns in every mode. Third, how far does the recovery extend? An onboard computer too limited to sustain the learning updates of the SAC agent supported real-time learning when paired with a remote workstation. I study these questions on two vision-based tasks, one with a UR5 robotic arm and the other with an iRobot Create2 mobile robot.

4.1 The ReLoD Framework

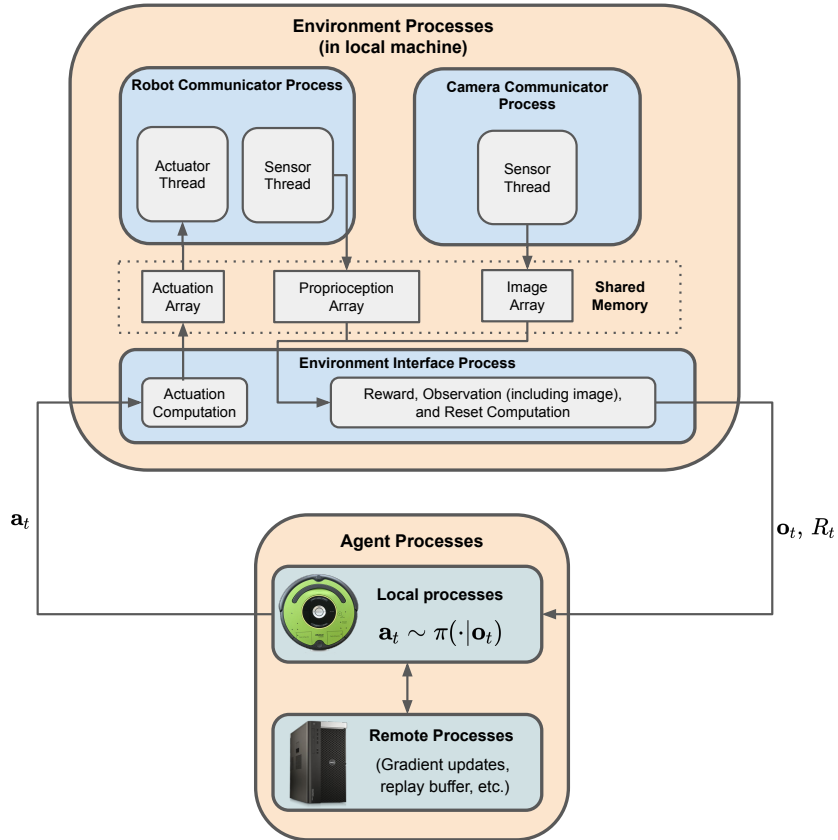


Figure 4.1: The ReLoD framework distributes the computations of a learning agent between a local and a remote computer.

The ReLoD framework runs action computation, policy and value function updates, and minibatch sampling as separate processes, placed on either computer and connected by TCP sockets. An update runs in its own process; an action therefore

never waits for an update to finish, although the two contend for resources when they share a computer. Minibatch sampling always resides with the update process. The placement of action computation and the learning updates, on the local or the remote computer, therefore determines the mode of operation.

ReLoD supports three modes of operation: *Local-Only*, which allocates all computations to the local computer; *Remote-Only*, which allocates all computations to the remote computer; and *Remote-Local*, which computes actions on the local computer and performs learning updates on the remote computer (Fig. 4.1).

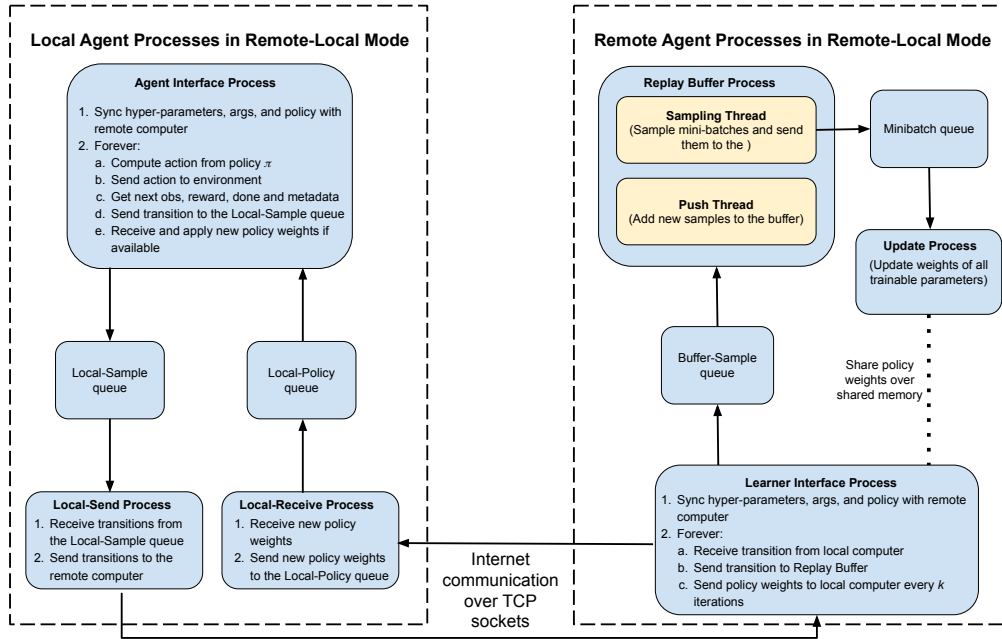


Figure 4.2: A SAC agent in the Remote-Local mode. The transition queues on the local and the remote computers have lengths equal to the maximum episode length to avoid losing data.

The Remote-Local mode serves mobile robots, where the learning agent has onboard access to a resource-limited local computer and wireless access to a powerful remote workstation. The mode distributes the computations of an agent using SAC across six processes (Fig. 4.2). On the local computer, three processes run in parallel: the *Agent Interface* process, which handles all computations of agent-environment interaction, including action computation; the *Local-Send* pro-

cess, which transmits transitions to the remote workstation; and the *Local-Receive* process, which receives new policy parameters from the remote workstation and synchronizes them locally. On the remote workstation, three more processes run in parallel: the *Learner Interface* process, which receives transitions from the local computer and sends back new policy parameters; the *Replay Buffer* process, which stores transitions and samples minibatches; and the *Update* process, which updates the policy and value function parameters from the sampled minibatches. The agent thus computes each action on the local computer within the action cycle time, and the remote workstation carries the cost of learning.

	Remote- Local	Remote- Only	Local- Only	Period of Iteration
Agent Interface Process (AIP)	Local	Local	Local	action cycle time
Action computation	Included in AIP	Included in LIP	Included in AIP	action cycle time
Local-Send Process	Local	N/A	N/A	action cycle time
Local-Receive Process	Local	N/A	N/A	every k iterations of LIP's loop
Learner Interface Process (LIP)	Remote	Remote	Included in AIP	action cycle time
Replay Buffer Process	Remote	Remote	Local	hardware dependent
Update Process	Remote	Remote	Local	hardware dependent

Table 4.1: The processes of a SAC agent and their distribution between the local and the remote computers in the three operational modes of the ReLoD framework.

An agent using PPO occupies the same process structure with two differences. The Replay Buffer process instead stores the rollouts of recent episodes, which

the Update process consumes and discards after each update. Updates occur only between episodes. New policy parameters therefore reach the local computer while no actions are due, and the parameters that compute actions never lag the learner within an episode.

The Local-Only and Remote-Only modes allocate all computations of a learning agent to a single computer. In the Local-Only mode, all computations run on the local computer tethered to the robot. In the Remote-Only mode, all computations, including action computation, run on the remote workstation, and observations and actions travel between the robot and the workstation over the wireless connection. Table 4.1 lists where the processes of a SAC agent run in each mode.

4.2 Vision-Based Reinforcement Learning Tasks

The experiments use two vision-based tasks, UR5-VisualReacher and Create-Reacher, one with a UR5 robotic arm and the other with an iRobot Create2 mobile robot. Both tasks are difficult for real-time RL methods: an agent requires tens of thousands of images to learn a good state representation and control policy.

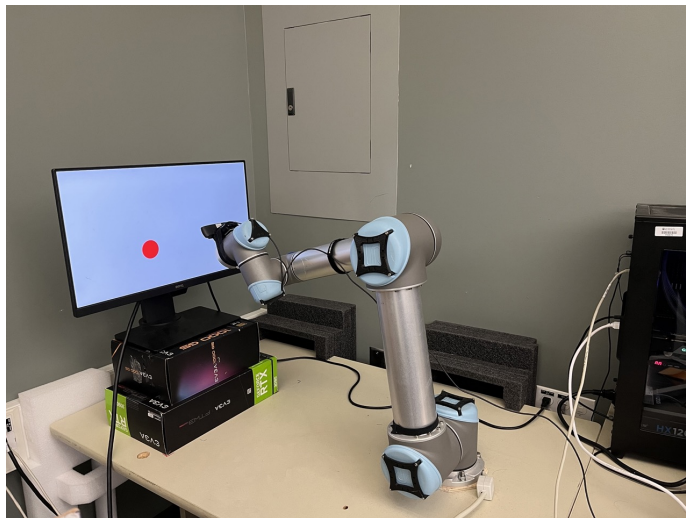


Figure 4.3: UR5-VisualReacher

UR5-VisualReacher is a vision-based task in which a UR5 arm moves its fingertip to a random target displayed as a red blob on a monitor (Yuan & Mahmood 2022). The action is the desired angular velocities of five joints, each in $[-0.7, 0.7]$ rad/s, and the action cycle time is 40 ms. The observation includes the joint angles, the joint velocities, the previous action, and three consecutive $160 \times 90 \times 3$ images from a wrist-mounted camera. Each episode lasts 4 seconds, that is, 100 timesteps. The arm moves within a bounding box to prevent collisions, and each reset displays a new random target and returns the arm to a predefined posture.

The reward combines a target-tracking term computed from the most recent image with a penalty on twisted joint configurations (Yuan & Mahmood 2022):

$$r \doteq \frac{c}{hw} \sum_{i=1}^h \sum_{j=1}^w M_{ij} W_{ij} - \beta \left(\left| \pi - \sum_{n=1}^3 q_n \right| + \left| \sum_{n=4}^5 q_n \right| \right),$$

where h and w are the height and width of the image in pixels, the entry M_{ij} is 1 if pixel (i, j) displays the target color and 0 otherwise, the entries of W decrease from 1 at the center of the image to 0 at the edges, the coefficients $c = 800$ and $\beta = 1$ weight the two terms, and $\mathbf{q}$ is the vector of the five joint angles.



Figure 4.4: Create-Reacher

Create-Reacher is a vision-based task in which the Create2 robot drives to a green paper target on a wall as quickly as possible. The action is the desired

velocities of the two wheels, each in $[-150, 150]$ mm/s, and the action cycle time is 45 ms. The observation includes the six wall-sensor values, the previous action, and three consecutive $160 \times 120 \times 3$ camera images. The reward is -1 per timestep, and an episode terminates when the target occupies more than 12% of the most recent image or after 30 seconds. The reward and the goal-based termination together instantiate the minimum-time task specification, which Chapter 5 studies in depth. The Create2 moves within a $110 \text{ cm} \times 70 \text{ cm}$ arena. Each reset moves the robot backward, reorients it in a random direction, and recharges it when its battery level falls below a threshold.

4.3 Policy Parameterization and Network Architecture

My implementations of SAC and PPO in the experiments use Gaussian policies over the continuous actions of the two tasks. A Gaussian policy represents the conditional action distribution as a multivariate normal, $\pi_{\theta}(a \mid s) = \mathcal{N}(a; \boldsymbol{\mu}_{\theta}(s), \boldsymbol{\Sigma}_{\theta}(s))$, with a diagonal covariance $\boldsymbol{\Sigma}_{\theta}(s) = \text{diag}(\sigma_1^2, \dots, \sigma_d^2)$, with a diagonal covariance $\boldsymbol{\Sigma}_{\theta}(s) = \text{diag}(\sigma_1^2, \dots, \sigma_d^2)$ for a d -dimensional action. The diagonal form treats the action components as conditionally independent given the state, reduces the number of covariance parameters from $\mathcal{O}(d^2)$ to $\mathcal{O}(d)$, and removes the need to maintain a positive-definite matrix during optimization. Both implementations parameterize $\log \sigma_i$ rather than σ_i to keep each standard deviation positive. A network maps the state to the mean; the SAC implementation computes the standard deviations from the same network, whereas the PPO implementation maintains them as a separate set of state-independent parameters. The standard deviations set the breadth of exploration around the mean action.

Both agents process an observation with the same convolutional network architecture (Fig. 4.5). The network applies four convolutional layers to the stacked

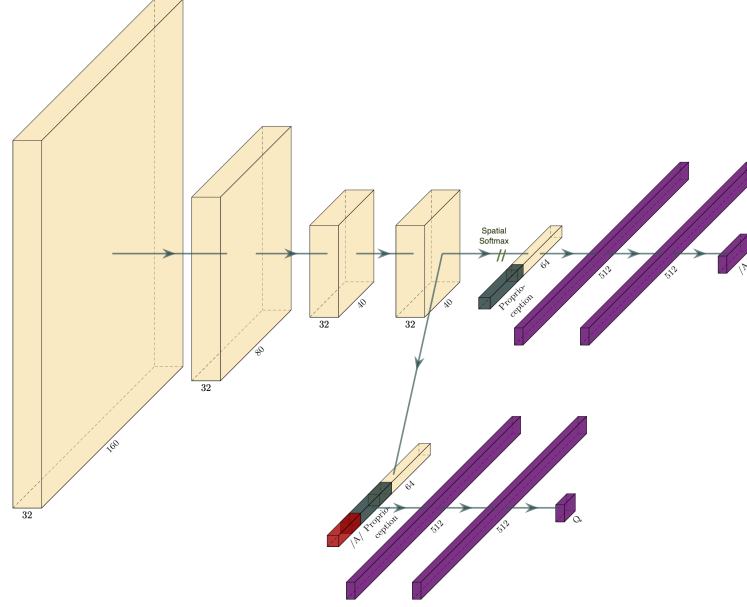


Figure 4.5: The convolutional network architecture. The yellow blocks are convolutional layers, the teal block is the non-image observation vector, the red block is the previous action, and the magenta blocks are fully connected layers.

images, each with 32 output channels, 3×3 kernels, and a ReLU activation, using a stride of two in the first three layers and one in the last. A spatial-softmax layer (Levine et al. 2016) converts each feature map into the expected pixel coordinates of its activations, producing one two-dimensional feature point per channel. Together, the convolutional and spatial-softmax layers form the *encoder*, which the actor and the critic share. The feature points are concatenated with the non-image components of the observation, which Section 4.2 lists for each task. The actor and the critic each apply two fully connected layers of 512 hidden units to the concatenated vector, with ReLU activations after each hidden layer.

The SAC implementation bounds its actions by squashing each Gaussian sample through the hyperbolic tangent, $a = \tanh(\mu_\theta(s) + \sigma_\theta(s) \odot z)$, where $z \sim \mathcal{N}(0, I)$ and $\odot$ denotes the element-wise product, so that every action component stays within its bounded range. The PPO implementation samples from the Gaussian directly and clips each sampled action to its bounded range. During SAC updates,

random cropping augments the images in each minibatch to make the learned representations more robust given the limited number of observations (Laskin et al. 2020). The critic loss alone trains the encoder; the actor propagates no gradients to the shared parameters.

4.4 SAC and PPO Under Ample and Limited Computation

Limited computation degrades learning with SAC in two ways: limited memory caps the replay buffer, and limited processing slows each learning update, which reduces how often the agent updates relative to the action cycle. To compare the two methods given ample computation and to measure the cost of limited computation, I evaluated agents using each method on UR5-VisualReacher in the Local-Only mode of ReLoD, using successively weaker computers tethered to the robot.¹ On the workstation, the agent used a minibatch of 256, a replay buffer of 100K transitions, and an update every 60–70 ms; on the laptop, a minibatch of 256, a buffer of 15K, and an update about every 200 ms; on the emulated Jetson Nano, a minibatch of 64, a buffer of 15K, and an update about every 500 ms. I emulated the Jetson Nano on the laptop by matching its update period, minibatch size, and buffer size, because the device itself became unresponsive after a few learning updates. Each run lasts 100K timesteps: about 67 minutes of interaction, and about two hours of wall-clock time including resets.

I ran the SAC agent under all three tiers but the PPO agent on the workstation only. Learning with SAC depends on computation through the replay buffer size and the update frequency; weaker hardware therefore slows it, and the three tiers show the size of the drop. Additional computation cannot increase the updates of the

¹The workstation has an AMD Ryzen Threadripper 2950X 32-core processor, a 11 GB Nvidia 2080Ti GPU, and 128 GB of RAM. The laptop is a Lenovo Ideapad Y700 with a 2 GB Nvidia GTX 960M GPU and 16 GB of RAM. The Jetson Nano, an edge device, has 4 GB of RAM.

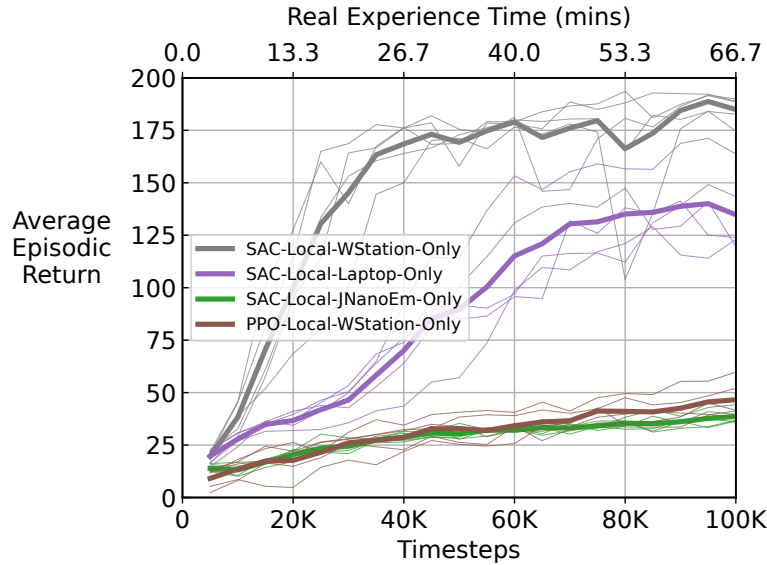


Figure 4.6: Learning curves of SAC under three tiers of computation and of PPO on the workstation, on UR5-VisualReacher. The wide lines are averaged over five independent runs.

PPO agent: each update consumes the single stream of on-policy experience that the robot provides, whereas in simulation many actors running in parallel supply additional streams (Nair et al. 2015). Weaker hardware would lengthen the pauses between episodes during which the PPO agent updates but would leave its return per timestep unchanged, as the Local-Laptop-Only mode later confirms (Fig. 4.8). The workstation therefore gives the highest return the agent can attain per unit of wall-clock time.

Given the workstation, the average episodic return of the SAC agent was more than twice that of the PPO agent within the same wall-clock time (Fig. 4.6). Limited computation removed the advantage. On the laptop, the return of the SAC agent dropped substantially, and on the emulated Jetson Nano, where the agent updated only once per 12 timesteps and from a smaller minibatch, its return fell below that of the PPO agent on the workstation within the same wall-clock time. The ordering of the curves matches the two causes: retaining less experience and updating less often both slow learning, and the returns are lowest where the two combine.

4.5 Effect of Distributing Action Computation and Learning Updates on SAC and PPO

I compared the SAC and PPO agents on UR5-VisualReacher under the three modes of operation described in Section 4.1: Local-Only, Remote-Only, and Remote-Local. The local computer was the laptop and the remote computer was the workstation of the previous section, connected over WiFi. If return depends only on the total available computation, then the Remote-Only mode should attain the highest return, because it allocates every computation to the most powerful computer.

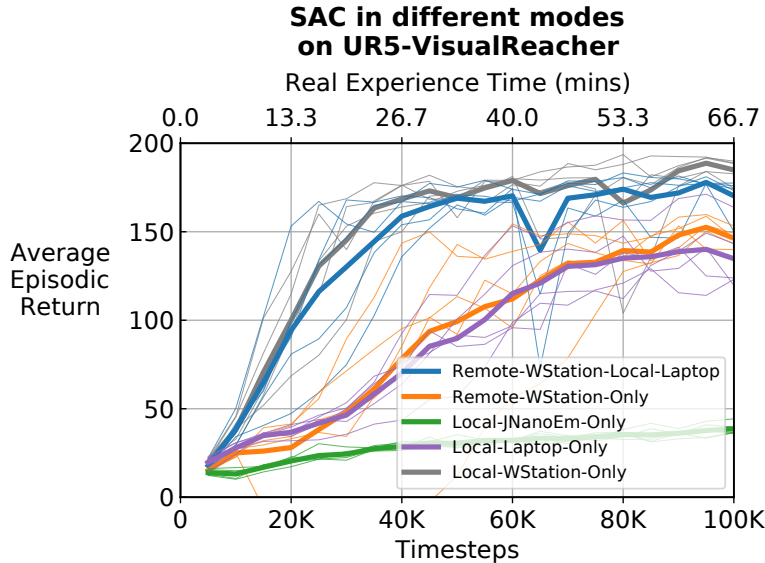


Figure 4.7: Learning curves of SAC in the three modes of operation of ReLoD on UR5-VisualReacher. The wide lines are averaged over five independent runs.

Computing actions on the local computer while updating on the remote computer recovered most of the return lost to limited computation (Fig. 4.7). The return of the SAC agent in the Remote-Local mode approached its return in the Local-Only mode on the workstation, the highest of the previous section, although only the laptop was tethered to the robot.

In the Remote-Only mode, the return of the SAC agent was only slightly above

its return in the Local-Only mode on the laptop, despite full access to the resources of the workstation, and it varied more across runs (Fig. 4.7). The result shows that a real-time learning system is sensitive to where delays occur. Delays act on at least two pathways: computing actions and updating parameters. In the Remote-Local mode, WiFi latency falls only on the update pathway, where a delayed update postpones learning. In the Remote-Only mode, the same latency falls on the action pathway, where it delays every action the robot takes, by an amount that varies from action to action. The distinction explains the low return and the larger variation of the SAC agent in the Remote-Only mode, and it is consistent with earlier findings that delays closer to robot actuation are more detrimental than delays farther from it (Mahmood et al. 2018a).

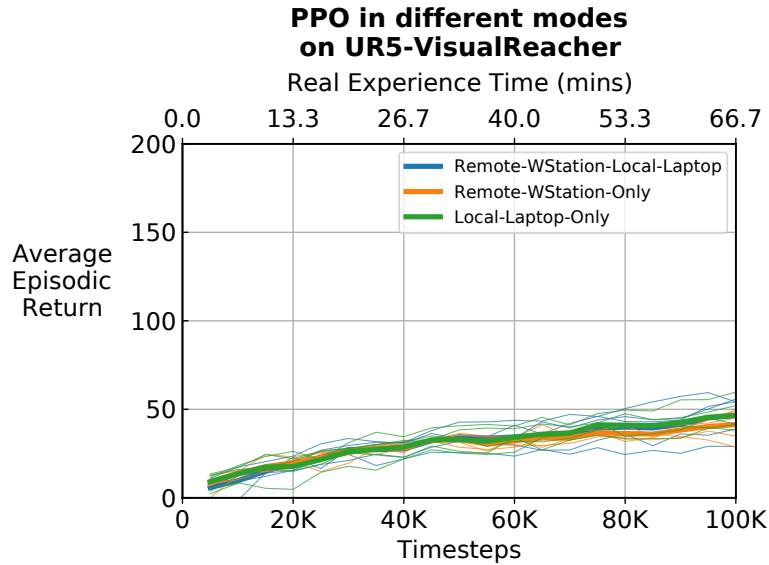


Figure 4.8: Learning curves of PPO in the three modes of operation of ReLoD on UR5-VisualReacher. The wide lines are averaged over five independent runs.

The return of the PPO agent was similar in all three modes (Fig. 4.8). The result follows from the two properties stated in the introduction: no mode supplies more on-policy experience, and the agent updates between episodes, when no actions are due. In the Remote-Only mode, the actions of the PPO agent crossed the same

wireless connection as those of the SAC agent, yet its return fell only slightly. The two results are compatible: a delayed action degrades control at every return level, and the SAC agent had far more return to lose, because in the other modes it learned policies that attained several times the return of the PPO agent. The experiments did not vary the latency or measure its effect on action timing, and therefore did not test whether the degradation grows with return.

4.6 Real-Time Learning on a Mobile Robot

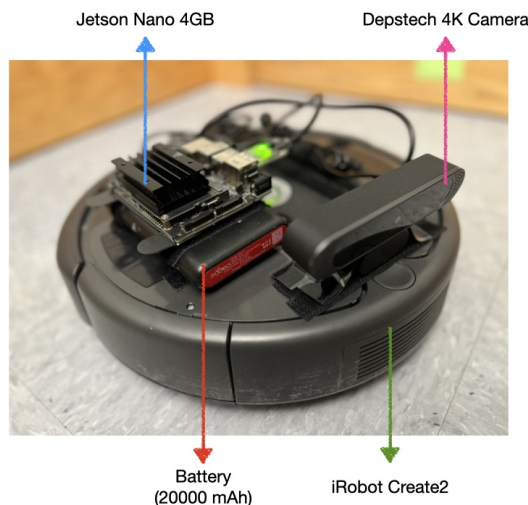


Figure 4.9: The Create-Reacher hardware setup.

The Create2 carries only a Jetson Nano, which cannot run the learning updates of the SAC agent. Create-Reacher therefore directly tests whether a remote workstation can supply the computation an onboard computer lacks. I compared the Remote-Local and Remote-Only modes on this task and excluded the Local-Only mode because the SAC agent on the emulated Jetson Nano attained the lowest return on UR5-VisualReacher (Fig. 4.6). The remote workstation for this task has an AMD Ryzen Threadripper 3970 CPU, an Nvidia 3090 GPU, and 128 GB of RAM. The Jetson Nano and the workstation communicated over WiFi through a router rated at 1 Gbps. Each run lasts 150K timesteps: about 112 minutes of interaction,

and 2.5–3.5 hours of wall-clock time including resets and recharging.

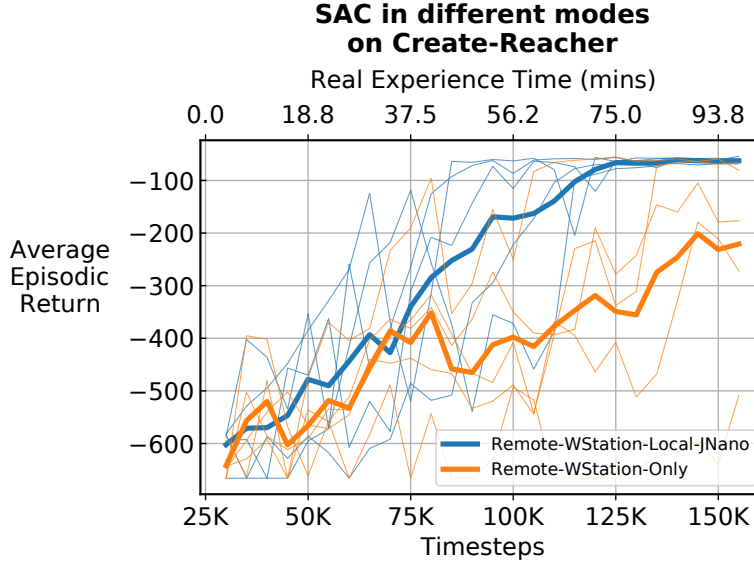


Figure 4.10: Learning curves of SAC in the Remote-Local and Remote-Only modes on Create-Reacher. The wide lines are averaged over five independent runs.

The SAC agent attained a substantially higher return in the Remote-Local mode than in the Remote-Only mode on Create-Reacher (Fig. 4.10). The result extends the pattern of the previous section to a mobile robot: computing actions on the local computer kept every action off the wireless connection, and the remote workstation ran the learning updates that the Jetson Nano cannot. A robot restricted to an edge computer can therefore learn a vision-based task with SAC in real time, given a wireless connection to a workstation.

4.7 Conclusion

Together, the three experiments show that a robot limited to an edge computer learned in real time with SAC when the learning updates ran on a remote workstation, and that moving computation to a remote computer raises return only for an agent that performs more updates when given more computation. Section 4.4 showed that the SAC agent, given ample computation, attained more than twice

the return of the PPO agent within the same wall-clock time, and that on the emulated Jetson Nano its return fell below that of the PPO agent on the workstation. Section 4.5 showed that in the Remote-Local mode the SAC agent attained nearly the return it attained on the workstation, because its replay buffer converts added computation into more updates from the same stream of experience, while the PPO agent attained similar returns in every mode. Section 4.6 showed real-time learning with SAC on a mobile robot whose onboard computer cannot run the learning updates alone. The comparison of modes also shows that a real-time learning system is sensitive to where delays occur: in the Remote-Only mode, WiFi latency delayed every action the robot took, whereas in the Remote-Local mode the same latency only postponed learning updates.

I introduced ReLoD, an open-source framework that distributes the computations of a real-time RL agent between a local and a remote computer, and used it to answer the three questions this chapter posed: on limited computers the SAC agent lost its advantage over the PPO agent, in the Remote-Local mode it attained nearly its workstation return, and a Jetson Nano paired with a remote workstation ran real-time learning with SAC on a mobile robot.

4.8 Limitations and Future Work

The experiments in this chapter cover a narrow range of tasks, methods, and hardware. Each comparison used a single task per robot and five independent runs. The results therefore establish orderings rather than precise magnitudes. I studied only two methods, chosen for their contrasting computational profiles and update schedules; an agent using another off-policy method may benefit differently from remote computation. I attributed the low return of the SAC agent in the Remote-Only mode to the variable latency of transmitting actions over WiFi; the results

on Create-Reacher are consistent with the attribution, but the experiments did not vary the latency or measure its effect on the timing of actions. The Remote-Local mode also requires a reliable wireless connection to a nearby workstation, which a laboratory provides but a robot deployed outside one may lack.

Future extensions of ReLoD should support more learning methods. Model-based RL reopens the question of where each computation should run, because decision-time planning performs heavy computation on the action pathway rather than the update pathway. In my experiments, the policy networks were small enough that the agent computed each action within the action cycle time; a larger policy network would require different design choices, such as computing every action from proprioceptive input while a slower concurrent process refreshes a cached image representation.

Future extensions should also revisit the division of computation itself, which was fixed by hand in every configuration. An agent could instead learn to allocate its own computation and memory, deciding when to compute locally, when to defer to the remote computer, and how much experience to retain, in the spirit of metareasoning (Russell & Wefald 1991, Hay et al. 2012) and adaptive recurrent computation (Graves 2016).

A truly autonomous robot cannot depend on a remote computer at all, because it may operate where no connection to one exists or lose the connection during operation. Chapter 6 develops a computationally inexpensive streaming reinforcement learning algorithm whose updates run on the onboard computer alone, without the remote computation that the Remote-Local mode requires.

Chapter 5

Minimum-Time Task Specification for Goal-Reaching Reinforcement Learning

A robot expected to solve many tasks over its lifetime cannot rely on a human to supply task-specific knowledge for each one in advance. It is often easier to recognize that a task is solved, that the mug is picked up or the door is opened, than to specify the movements required to solve it. A reward crafted by hand for each task ties each new skill to a designer who holds task-specific knowledge, and the dependence limits how many tasks the robot can acquire on its own. A reward specification built from a test for success alone narrows the designer’s burden to the easier judgment: recognizing that the task is solved.

Many real-world robot learning problems, such as pick-and-place or arriving at a destination, amount to reaching a goal state as soon as possible. Formulated as episodic reinforcement learning tasks, goal-reaching problems admit a specification that matches the designer’s intent: a reward of -1 on every timestep, with termination upon reaching the goal state. I call it the *minimum-time* specification because maximizing the undiscounted sum of the rewards minimizes the time taken to reach the goal. The reward encodes nothing about how to reach the goal and therefore does not bias the solution toward a designer’s guess at good behavior. The minimum-time specification is often overlooked in favor of *guiding rewards*

(also known as dense rewards), which provide an informative learning signal on most timesteps, typically by scoring progress toward the goal, and are perceived as easier to learn from. Guiding rewards can bias the solutions the agent finds and lead to suboptimal outcomes because they reflect the designer’s preferred behavior (Riedmiller et al. 2018, Booth et al. 2023, Knox et al. 2023).

I contrast the minimum-time specification with two alternatives, a guiding-reward and a contact-reward specification, on goal-reaching tasks in simulation, and with the guiding-reward specification on physical robots. In simulation, policies trained under the minimum-time specification reached the goal two to three times faster than policies trained under guiding rewards, remained on target longer, and matched or surpassed them on the guiding-reward objective itself. The number of times the initial policy reached the goal within a fixed budget of timesteps predicted how quickly and consistently learning succeeded. Guided by a target-hit threshold, agents using SAC learned pixel-based policies from scratch on four physical robot platforms, with a constant negative reward on every timestep, within a few hours of experience. On the two robot arms, minimum-time policies again reached the goal faster, although marginally, and remained on target longer.

5.1 Related Work

The community has met the difficulty of learning without an informative per-timestep reward signal in two ways: (a) make the reward more informative through guiding and shaping terms, crafted by hand or generated automatically; and (b) keep the reward sparse while strengthening exploration. Sutton and Barto advise that the reward signal communicate to the agent what the designer wants achieved rather than detail how to achieve it, and the minimum-time specification follows the advice (Sutton & Barto 2018). The specification requires only a test for success and leaves the objective unambiguous. The chapter therefore asks when learning

under it succeeds, a question complementary to many exploration strategies.

Reward Design Task designers commonly craft reward functions through trial and error so that agents both learn quickly and perform the task well (Lee et al. 2019, Wu et al. 2021, Knox et al. 2023). Designers often augment a reward that signals task success with guiding or shaping terms that steer the agent toward a desired policy. The resulting reward function both communicates the performance metric and prescribes how to improve it. An early instance rewarded steps up a progress gradient rather than goal attainment alone (Matarić 1994). A shaping term preserves the optimal policy when it is potential-based, taking the form $\gamma\Phi(s') - \Phi(s)$ for a potential function Φ over states (Ng et al. 1999). A guiding term need not be potential-based, and ad hoc shaping is widely recognized as unsafe because it may alter the optimal solution of the task (Amodei & Clark 2016, Knox et al. 2023). Designers who iterate by trial and error also risk overfitting the reward function to the algorithm they used while designing it (Booth et al. 2023, Muslimani et al. 2025). One approach automates the design, searching over candidate reward functions with a large language model that uses task-specific prompts, reward templates, and evolutionary search (Ma et al. 2024). Searching over reward functions, like searching over neural architectures, costs substantial computation. Other approaches derive reward signals from pre-trained vision-language models, avoiding the search, but the feedback from such models can be too imprecise for tasks that require fine spatial distinctions (Rocamonde et al. 2024, Wang et al. 2024). Each route to a more informative reward exacts a cost: designer effort and bias risk when crafted, computation or imprecision when automated.

Exploration Strategies for Learning with Sparse Rewards An agent learning from sparse rewards may take many actions before it encounters a state where the reward differs. Exploration therefore determines whether learning begins at all.

One remedy has the agent learn auxiliary tasks alongside the target task. An agent using SAC-X learns policies for a set of auxiliary tasks and schedules them to explore the observation space in search of the sparse rewards of externally defined target tasks (Riedmiller et al. 2018). A later extension defined the auxiliary tasks internally, on raw sensor streams alone, and agents using it solved tasks as complex as Ball-in-Cup from scratch (Hertweck et al. 2020). The remedy carries a cost: someone must define the auxiliary tasks, and the agent must learn and schedule them alongside the target task. Goal relabeling offers another route to learning from sparse rewards. Hindsight Experience Replay (HER) enables an agent to learn from failed episodes by treating previously seen states as pseudo-goals (Andrychowicz et al. 2017). HER applies only to environments where every reachable state can serve as a goal state and therefore does not help when the task specifies one goal state or a small set. HER also does not extend directly to vision-based tasks, where distinct visual features characterize the goal states. An extension of HER to vision-based reaching and pushing on a 7-degree-of-freedom (DOF) Sawyer arm relies on a fixed arena with an overhead camera that always views the target, and it still requires every reachable state on a 2D plane to be a valid goal (Nair et al. 2018). This chapter proposes no new exploration strategy. Instead, the chapter offers an inexpensive predictor of whether an agent can learn a minimum-time task quickly and reliably: the number of target hits under the initial policy.

Learning from Pixels from Scratch on Physical Robots With DayDreamer, a model-based method, four physical robots learned real-world tasks under a single system (Wu et al. 2023). Visual Foresight (Ebert et al. 2018) combines a learned video prediction model with online planning, but it suits only short-horizon tasks and must generate images during planning, at substantial computational cost. Both approaches require a model. In the robot experiments of this chapter, agents instead

learn pixel-based control in real time with a *model-free* algorithm.

Minimum-Time Tasks Minimum-time problems in optimal control ask for the control that transfers a system from an initial state to a final state in the shortest time (LaSalle 1959, Kahn & Roth 1971, Bobrow et al. 1985, Shin & McKay 1985, Chapter 9 of Chui & Chen 2012). One approach combines classical path planning with model-free deep RL to train a neural network policy for flying a quadrotor through a sequence of waypoints, among obstacles, in minimum time (Penicka et al. 2022). Control benchmarks in RL, such as Mountain Car (Moore 1991) and DotReacher (Garg et al. 2022), also use a reward of -1 on each timestep until termination, matching the minimum-time specification. The specification induces an exact ordering: the return of an episode is the negative of the time the agent takes to reach the goal. Expected returns therefore rank policies precisely by expected time to goal, with the time-optimal policies at the top. A guiding term replaces the ranking with a proxy, and an agent that climbs the proxy can converge to a local optimum of it, a behavior that collects the guiding signal without solving the task.

5.2 Specifying Goal-Reaching Tasks

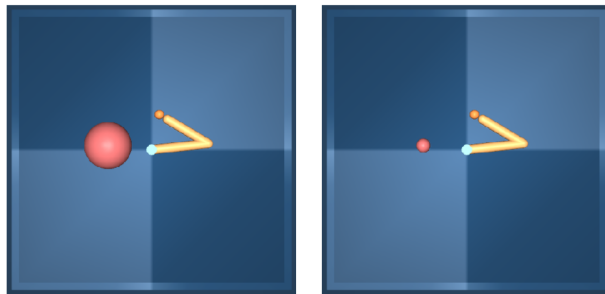


Figure 5.1: Reacher-Easy (left) and Reacher-Hard (right) from the DeepMind Control Suite.

Goal-reaching tasks admit several specifications, differing in the choice of re-

ward function and termination condition. The classic Reacher problem serves as a running example (Fig. 5.1). The task is to move the fingertip of a planar arm with two degrees of freedom to a random spherical target on a 2D plane (Tassa et al. 2018). The task designer intends for the arm to reach the target swiftly and remain there. The size of the target sets the difficulty of the task. The observation is the position and velocity of the fingertip and the vector from the fingertip to the target. The action is the torques applied to the two joints. Three specifications of the task follow, summarized in Table 5.1.

Table 5.1: Task specifications for Reacher: choices for reward function and termination conditions. Agents learned under all three specifications with a discount factor of $\gamma = 0.99$.

Specification	Reward and Termination
Guiding Reward	$R_{t+1} = \begin{cases} 1 & \text{if in target,} \\ - x_{\text{goal}} - x_{\text{pos}} & \text{otherwise.} \end{cases}$ Fixed-length episodes ($T = 1000$)
Contact Reward	$R_{t+1} = \begin{cases} 1 & \text{if in target,} \\ 0 & \text{otherwise.} \end{cases}$ Fixed-length episodes ($T = 1000$)
Minimum- Time	$R_{t+1} = -1 \text{ (until reaching the target)}$ Varying length episodes; termination upon reaching a goal state with near-zero velocity

Guiding reward specification. The reward function follows Gymnasium (Towers et al. 2024), which rewards the agent for moving as close to the target as possible. A precision term, previously used on a physical UR5 industrial arm, adds a bonus once the fingertip is inside the target (Korenkevych et al. 2019, Che, Vasan & Mahmood 2023). The reward has two components: a penalty equal to the negative Euclidean distance between the fingertip and the target, and a bonus of +1 when

the fingertip is inside the target. Each episode lasts 1000 timesteps, long enough that remaining at the target pays.

Contact reward specification. The agent receives a reward of $+1$ whenever the fingertip is inside the target and 0 otherwise. Each episode lasts 1000 timesteps. The reward and episode length follow the *dm_control* suite (Tassa et al. 2018).

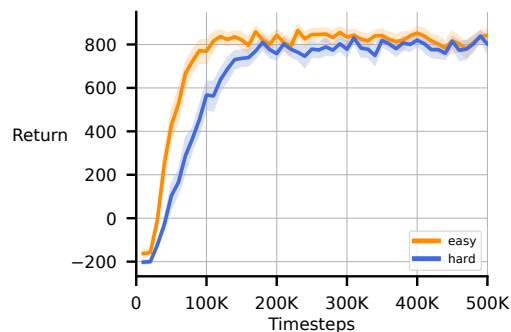
Minimum-Time specification The agent receives a reward of -1 on every timestep. The episode terminates when the fingertip reaches the target with near-zero velocity. The goal states are therefore the states that pass the success test, which the designer chooses as part of the specification, and an episode is complete only when the agent reaches one. Episodes therefore vary in length, and no time limit defines the episode. A time limit instead bounds each attempt within an episode: when the agent exceeds it, the environment moves the arm to a different starting state, leaves the target unchanged, and the episode continues. Resetting a robot costs time in the real world, and the robot performs the reset itself through a pre-programmed routine. A timeout accordingly incurs a penalty of $-K$, where K is the number of timesteps the reset requires. Consider a task with a time limit of 100 timesteps and a reset penalty of -20 . If the agent times out three times and then reaches the goal in 25 timesteps, the episode has return $-100 - 20 - 100 - 20 - 100 - 20 - 25 = -385$ and length 385. Learning curves in this chapter report undiscounted returns computed in this manner.

The contact reward and minimum-time specifications are closely related. The contact reward gives a positive reward upon success and zero everywhere else, a form often called a sparse reward because a differentiating reward signal arrives only rarely. The minimum-time reward is an extreme form of sparse reward: the reward never changes, and the differentiating signal arrives only through the return, the accumulated reward of an episode. The minimum-time specification is also simpler, because preferring to reach the goal sooner requires no discounting.

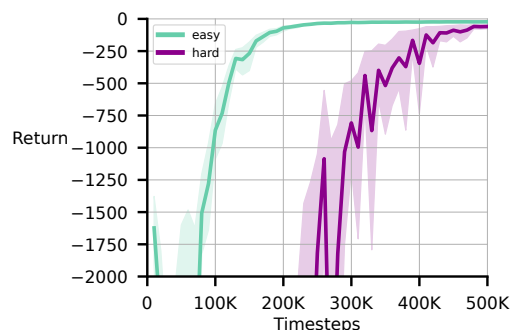
5.3 Minimum-Time Policies Outperform Guiding-Reward Policies on Reacher

Learning Under the Three Specifications

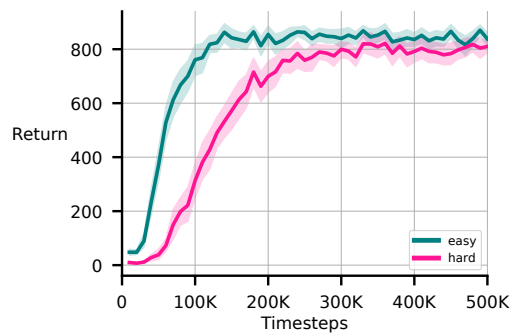
The three specifications in Table 5.1 specify the same underlying goal, yet they need not be equally easy to learn or produce equally good behavior. To compare them, I held the learning algorithm fixed and varied only the specification. SAC, described in Chapter 2, served as the algorithm, and agents using it learned under all three specifications of the Reacher task with the same hyperparameters, listed in Appendix A.3. Agents learned quickly under the guiding reward specification on both Reacher-Easy and Reacher-Hard (Fig. 5.2a). Learning was slower but still fast under the contact reward specification (Fig. 5.2c). Under the minimum-time specification, the learning curves rose more slowly and varied more across runs (Fig. 5.2b).



(a) Guiding reward specification



(b) Minimum-time specification



(c) Contact reward specification

Figure 5.2: Learning curves of SAC under the three reward specifications on Reacher. Each solid curve shows the mean of 30 independent runs, with shaded regions indicating the 95% confidence interval.

Evaluating the Final Policies

A fair comparison of the three specifications rests on three criteria (Table 5.2): ease of specification, whether crafting the reward requires domain knowledge; speed of learning, whether training is quick and consistent across runs; and final performance, whether the learned behavior matches the task designer’s intent. Two metrics measure final performance: the number of timesteps the agent needs to reach the target, termed *steps to goal*, and the number of timesteps the agent remains at the target once reached, termed *steps on goal*. I evaluated all 30 learned policies from each specification, 90 policies in total, on 500 episodes of 5,000 timesteps each, 2.5 million timesteps per policy.

	Guiding Reward Specification	Contact Reward Specification	Minimum-Time Specification
Ease of specification	×	✓	✓
Speed of learning	✓	✓	×
Superior final performance	×	×	✓

Table 5.2: Comparison of the three task specifications on three criteria.

Agents learned fastest under the guiding reward specification, yet it is the hardest to specify. Despite slower learning, the minimum-time specification produced the best final policies on both metrics (Fig. 5.3): roughly $3\times$ faster to the goal on Reacher-Easy and $2\times$ faster on Reacher-Hard (Fig. 5.3a).

Every agent learned with a discount factor of $\gamma = 0.99$ (Appendix A.3), whereas steps to goal and steps on goal are both undiscounted. The minimum-time specification needs no discount in principle, because the undiscounted return already decreases with the time to goal; in preliminary runs with $\gamma = 1$, agents learned less reliably. The contact reward specification benefits from a discount in a different way: with $\gamma < 1$, earlier arrival at the goal earns a larger discounted return,

a preference that the undiscounted sum expresses only weakly, through the additional timesteps spent on the goal. The comparison therefore holds the discount fixed across specifications and judges the learned behavior by the designer’s intent rather than by any discounted objective.

Evaluating Policies Across Specifications

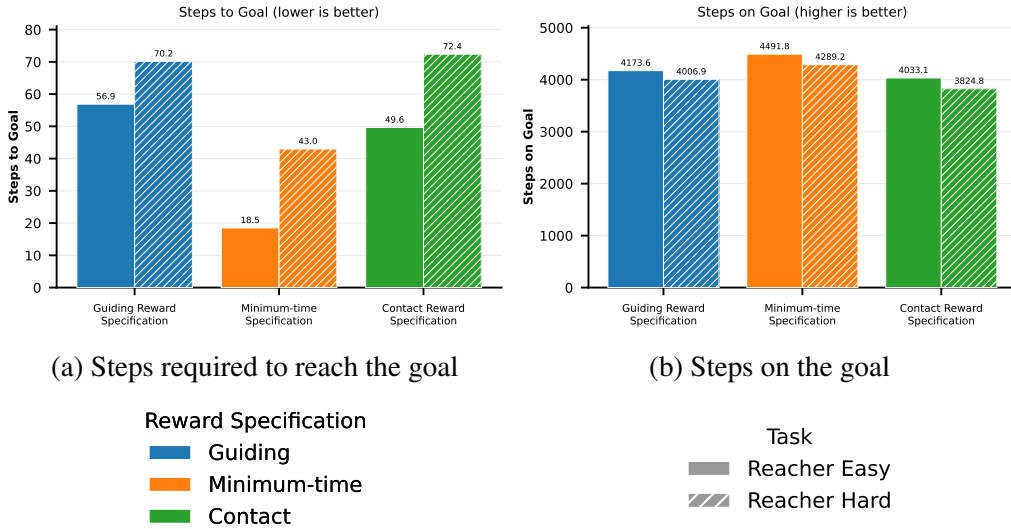


Figure 5.3: Comparison of the three specifications of the reaching task using the number of steps it requires to reach the goal (left) and the number of steps it stays within the target sphere upon reaching it (right). The reported mean is estimated over 30 independent runs.

I also evaluated the policies of each specification on the objectives of the other two. Policies trained under the contact reward and minimum-time specifications were assessed on the guiding reward objective every 10,000 timesteps of training. The minimum-time policies performed comparably to the guiding reward policies on Reacher-Easy (Fig. 5.4a) and surpassed them on Reacher-Hard (Fig. 5.4b). On the contact reward objective, the contact reward policies performed on par with or slightly better than the guiding reward policies but lagged behind the minimum-time policies (Fig. 5.4c & 5.4d). On the minimum-time objective, both the guiding reward and contact reward policies performed poorly (Fig. 5.4e & 5.4f). Episode

completion under the minimum-time specification requires reaching a goal state, and the steps-on-goal results suggest the guiding reward and contact reward policies reach the target but do not settle there (Fig. 5.3b). The minimum-time policies thus matched or surpassed the alternatives on every objective, while neither alternative matched them even on its own.

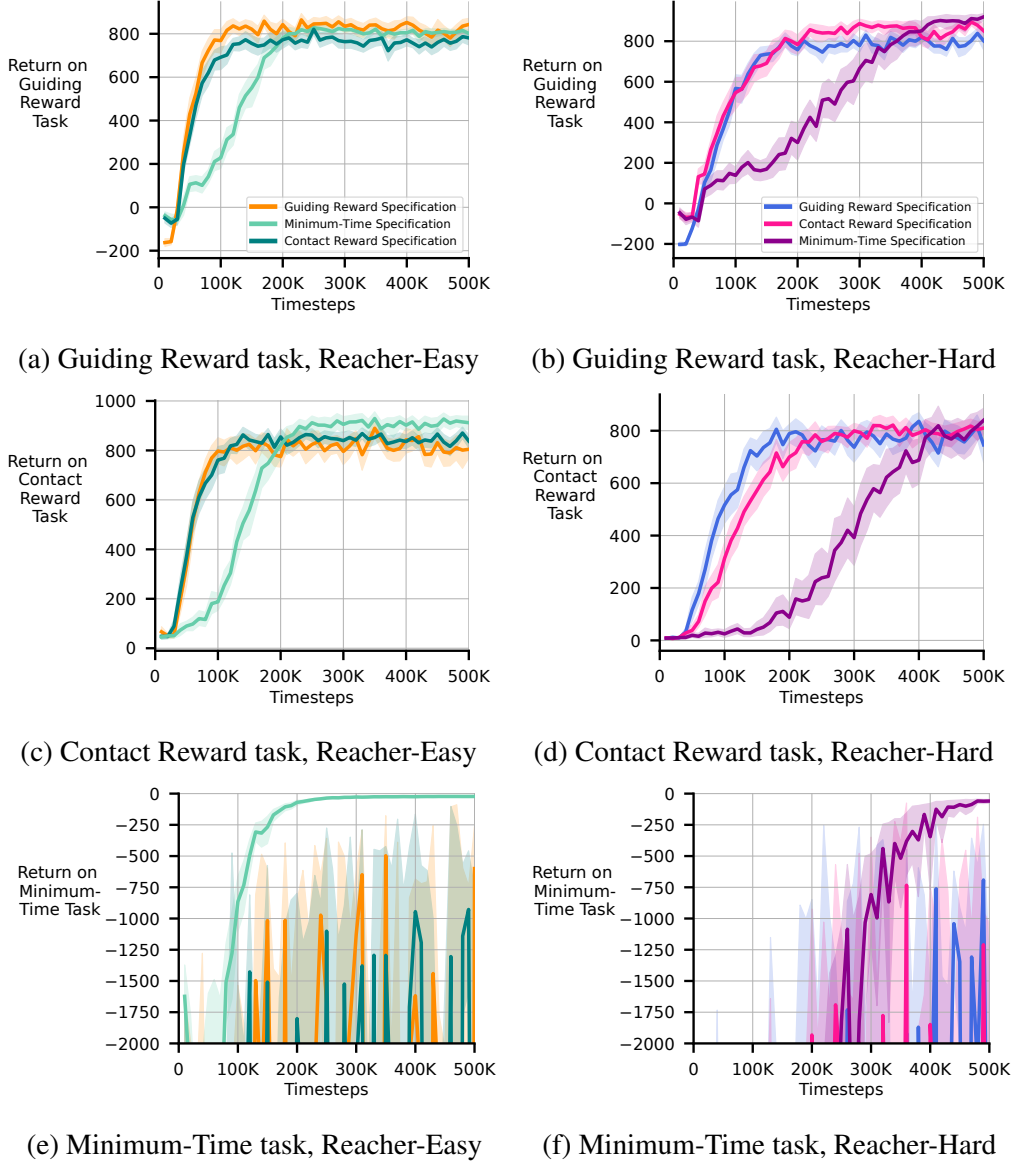


Figure 5.4: Evaluating the policies of all three specifications on each specification's objective, on Reacher-Easy (left column) and Reacher-Hard (right column). Each solid learning curve is an average of 30 independent runs. The shaded regions represent a 95% confidence interval.

5.4 Target Hits of the Initial Policy Can Predict Learning Success

Specifying a minimum-time task is straightforward, but learning it can be difficult on complex real-world problems. One factor that affects the difficulty is exploration. Resetting the environment after a fixed time limit controls exploration implicitly. A large time limit gives the agent more time to explore and increases its chances of reaching a goal state. A large time limit can also hurt exploration, perhaps because a poor policy leaves the agent stuck in uninformative states that a reset would free it from.

Benchmark suites such as Gymnasium (Towers et al. 2024) and the DeepMind Control Suite (Tassa et al. 2018), along with classic minimum-time tasks such as Mountain Car (Moore 1991), treat the time limit as an intrinsic property of the task. In the minimum-time specification, the definition of an episode is independent of the time limit: an episode ends only upon reaching a goal state, and a time-

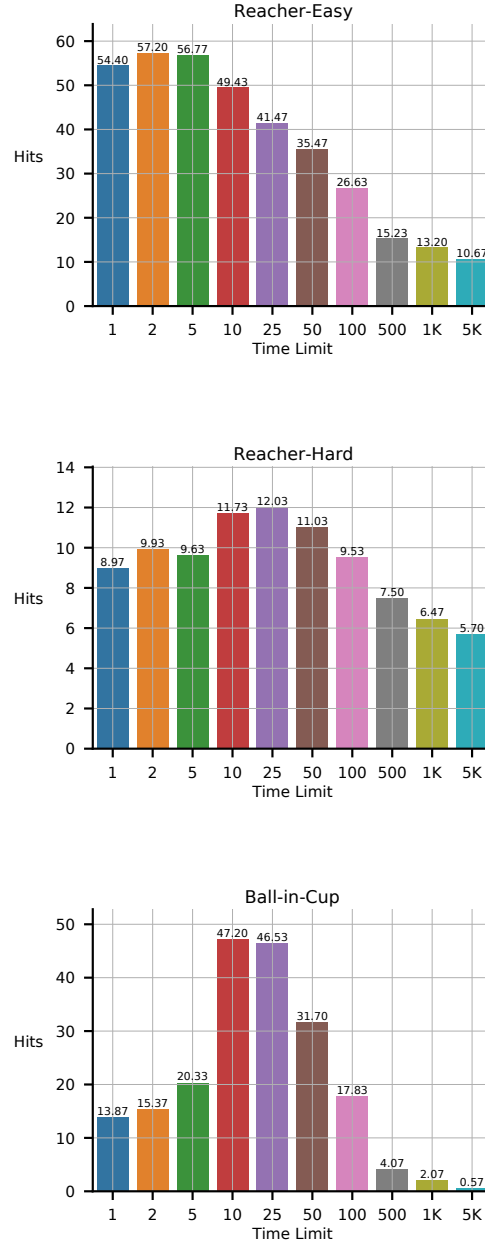


Figure 5.5: Target hits achieved by a random initial policy within 20K timesteps, across a range of time limits.

out merely relocates the agent. The time limit is therefore a solution parameter rather than a problem parameter, and a learning system is free to tune it without altering the problem. Tuning the time limit requires a guide, one obtainable before committing to a full training run.

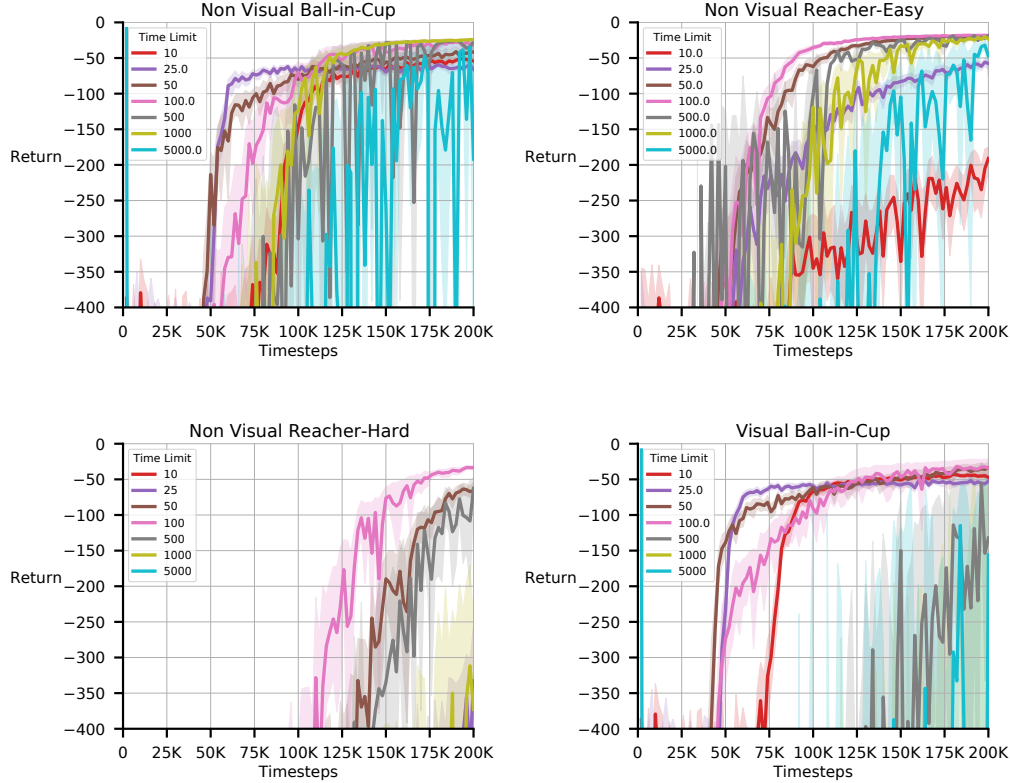


Figure 5.6: Learning performance of SAC on three non-visual and one visual task in simulation for multiple choices of time limit. Each solid curve is averaged over 30 independent runs. The shaded regions represent a 95% confidence interval.

Target hits of a random initial policy offer an inexpensive guide. A *target hit* occurs each time the agent satisfies the success test, and target hits therefore count how often the agent reaches a goal state. I recorded target hits for an initial Gaussian policy $\mathcal{N}(0, 1)$ over 20K timesteps across a range of time limits, on Reacher and on a minimum-time variant of Ball-in-Cup (Tassa et al. 2018). Target hits varied substantially with the time limit on all three tasks (Fig. 5.5), and no single time limit was best across tasks. The tuning is therefore per task.

Target hits under the initial policy may also predict whether learning will succeed, because learning cannot begin in earnest until the agent has reached a goal state at least once. I tested the connection between target hits and learning success on four tasks: Non-Visual and Visual Ball-in-Cup, Non-Visual Reacher-Easy, and Non-Visual Reacher-Hard. The learning curves tracked the target-hit counts closely (Fig. 5.6): time limits that produced many target hits under the initial policy yielded fast, consistent learning, and time limits that produced few target hits yielded slow or failed learning. A learning system can accordingly choose the time limit of a minimum-time task by measuring target hits under the initial policy across candidate limits and selecting a limit that yields many hits.

5.5 Learning Vision-Based Minimum-Time Tasks on Physical Robots

I test whether an agent can learn a minimum-time task on a physical robot from raw pixels, from scratch, within a practical wall-clock budget. Four vision-based robot tasks provide the test (Fig. 5.8, Fig. 5.9). On the physical robots, episodes terminate according to the most recent image, and termination is therefore defined on the observation rather than on an underlying state. I chose the time limit for each task by the recipe of Section 5.4, requiring the initial policy to exceed 10 target hits within 20K timesteps, with one exception: the 750-timestep limit on Franka-VisualReacher fell below the threshold and was retained as a test of learning under insufficient target hits. Robot control typically runs at 10–125 Hz. Collecting 20K timesteps therefore takes at most 35 minutes, a feasible cost for real-world experimentation. Training then proceeded with an asynchronous implementation of SAC (Yuan & Mahmood 2022), using the hyperparameters in Appendix A.3 and a reward of -1 on every timestep until the episode ends.

Create-Reacher. The task follows an existing minimum-time task (Wang,

Vasan & Mahmood 2023). A Depstech 4K camera and a Jetson Nano 4GB computer support image-based observations and onboard inference (Fig. 5.8a). The objective is to reach one of the green sticky notes swiftly. The episode terminates when a green sticky note occupies at least 20% of the most recent image.

UR5-VisualReacher. The task is a minimum-time adaptation of an existing guiding-reward task (Yuan & Mahmood 2022). The objective is to move the fingertip of the arm to a red target on the screen as fast as possible (Fig. 5.8b). The episode terminates when the target occupies more than 1.5% of the most recent image. The target remains unchanged until the arm reaches it. Upon termination, the arm resets to a predefined posture and a new random target appears.

Vector-ChargerDetector. The task is a new vision-based goal-reaching task on Anki Vector, a low-cost mobile robot (Fig. 5.9a). The agent sends velocity commands to the wheels every 100 ms over WiFi. The episode terminates when the charger symbol is centered and occupies roughly 25% of the most recent image. During a reset, Vector moves backward and reorients itself in a random direction.

Franka-VisualReacher. The task is a new task on the Franka Emika Panda, a 7-DOF robot arm. The objective is to move the wrist-mounted camera close to a bean bag on the table (Fig. 5.9b). The agent sends velocity commands for each joint at 25 Hz. Each episode places the arm and the bean bag in random positions. The episode terminates when the bean bag occupies more than 12% of the most recent image.

A practical demand of real-time learning is that the reward be computable where the robot learns. Robot learning experiments often compute rewards from external instrumentation, such as motion capture systems, that is available in the laboratory but not during deployment (Kober et al. 2013, Ibarz et al. 2021). Rewards computed from the robot’s onboard sensing free the robot from that infrastructure (Hafner et al. 2021). *Every reward in the physical-robot experiments, minimum-time and*

guiding alike, was computed from the robot’s own sensing. The minimum-time specification requires only a test for goal attainment, and the guiding-reward specifications used the same sensing. The comparisons therefore isolate the form of the reward.

Learning vision-based policies for these tasks from scratch within a practical timeframe is demanding for model-free algorithms. Data collection on physical robots is slow and costly. Therefore, I tested three time limits on Franka-VisualReacher and chose one for each remaining task, confirming each choice against the target-hit threshold (Table 5.3), with five runs per task. The ReLoD framework of Chapter 4 supported real-time learning throughout.

Task	Time Limit	Average Target Hits	Training (Steps)	Control Frequency (Hz)	Initial Hits Estimation Time (mins)	Robot Experience Time (hours)
Franka VisualReacher	75 (3s)	12.6 ± 1.0	60k	25	13.3	$0.6\bar{6}$
	150 (6s)	13.0 ± 2.08				
	750 (30s)	7.8 ± 0.87				
Create Reacher	333 (15s)	18.8 ± 3.92	100k	$22.2\bar{2}$	15	1.25
UR5 VisualReacher	150 (6s)	14.4 ± 2.5	100k	25	13.3	$1.1\bar{1}$
Vector ChargerDetector	300 (30s)	11 ± 1.1	160k	10	$33.3\bar{3}$	$4.4\bar{4}$

Table 5.3: Robot experiment setup. The average target hits were calculated over five independent runs using the initial Gaussian policy $\mathcal{N}(0, 1)$ over 20K timesteps for each task. Robot experience time excludes time spent resetting the environment and charging batteries.

Agents using SAC learned successful policies on every task (Fig. 5.7). Under the time limits that exceeded the threshold of 10 target hits, learning was fast and consistent. The 750-timestep limit on Franka-VisualReacher fell below the threshold, with 7.8 target hits, and learning under it varied more across runs and required

roughly 35K additional timesteps to match the final performance of the 75-timestep limit (Fig. 5.7c). The robot results are consistent with the simulation results: the number of target hits under the initial policy indicates how quickly and consistently an agent learns a minimum-time task.

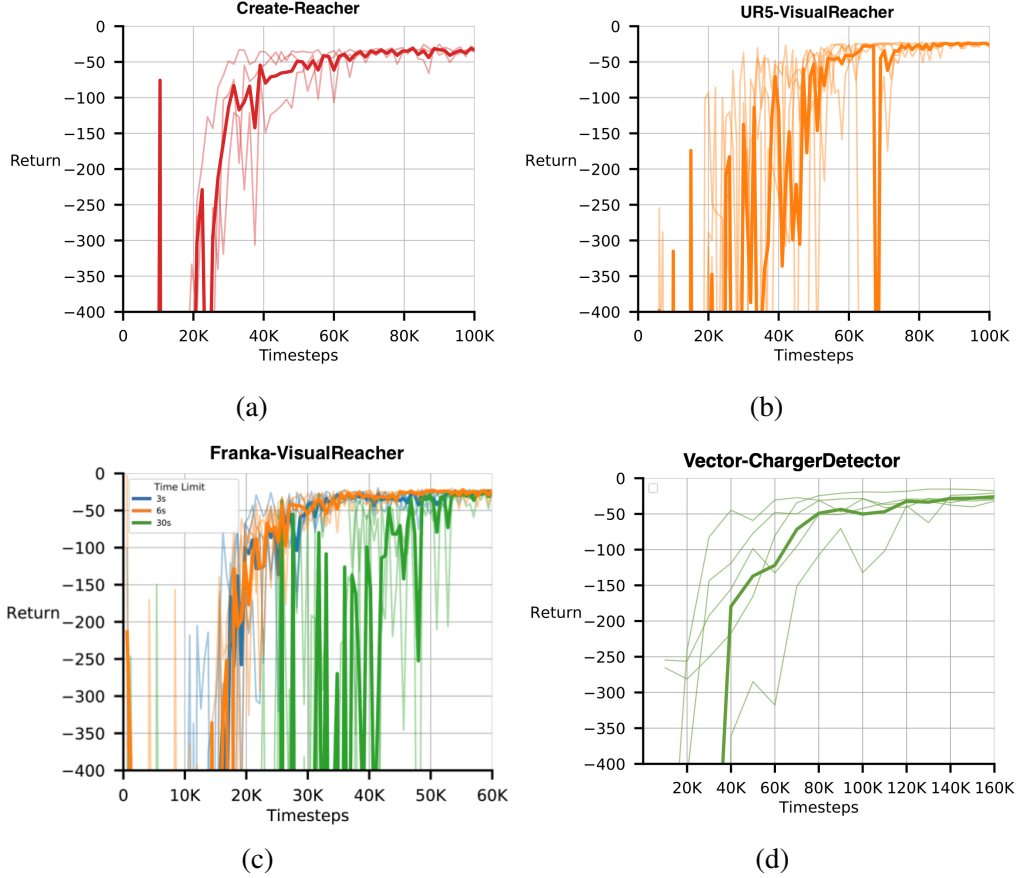
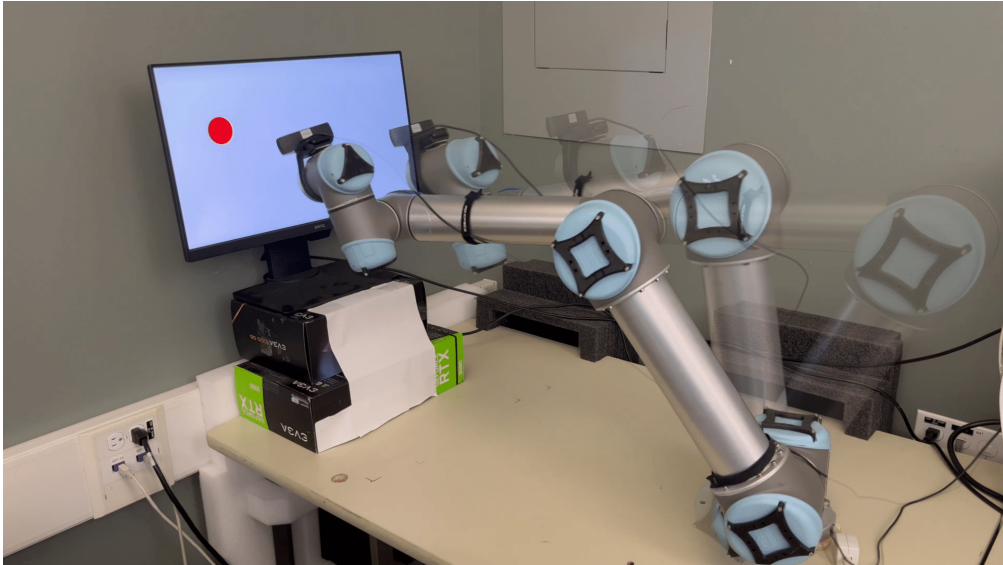


Figure 5.7: Learning performance of SAC on four vision-based robot tasks. Each thin learning curve represents an independent run. The bold learning curves of the same color are the average of all independent runs. The initial policy is excluded.



(a) Create-Reacher



(b) UR5-VisualReacher

Figure 5.8: Frames sampled from single episodes of learned policies completing Create-Reacher (top) and UR5-VisualReacher (bottom), superimposed. Episodes start from random configurations.



(a) Vector-ChargerDetector



(b) Franka-VisualReacher

Figure 5.9: Frames sampled from single episodes of learned policies completing Vector-ChargerDetector (top) and Franka-VisualReacher (bottom), superimposed. Episodes start from random configurations.

5.6 Minimum-Time Policies Remain Better on Physical Robots

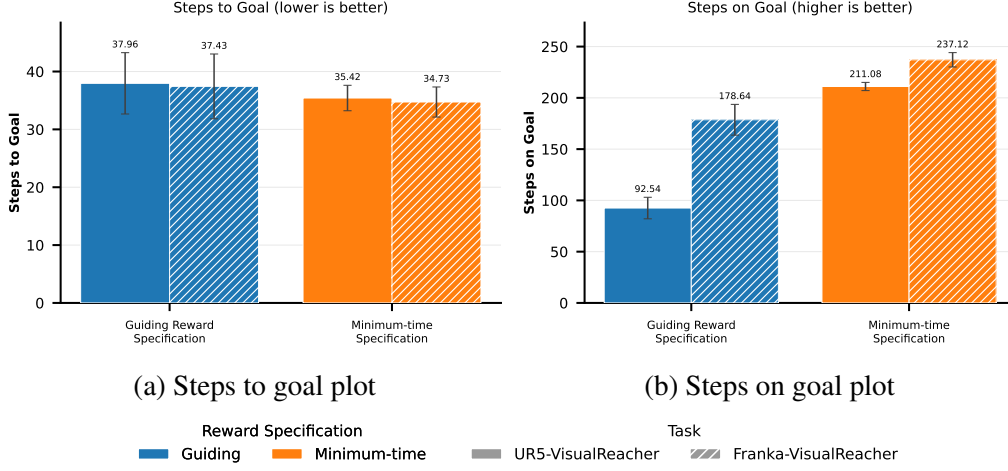


Figure 5.10: Comparison of the guiding reward and minimum-time specifications of the reaching task using the number of steps it requires to reach the goal (left) and the number of steps it stays on target upon reaching it (right).

I repeated the final-behavior comparison of Section 5.3 on UR5-VisualReacher and Franka-VisualReacher, evaluating policies trained under the minimum-time and guiding-reward specifications on steps to goal and steps on goal. The guiding-reward tasks follow prior work (Yuan & Mahmood 2022, Grooten et al. 2024). The evaluation spanned 2,500 episodes of 500 timesteps for UR5-VisualReacher (1.25M timesteps) and 1,000 episodes of 1,000 timesteps for Franka-VisualReacher (1M timesteps). Minimum-time policies reached the goal marginally faster and remained on target longer than guiding-reward policies (Fig. 5.10), consistent with the simulation results (Fig. 5.3).

5.7 Conclusion

The minimum-time specification defines a goal-reaching task with nothing more than a reward of -1 on every timestep and termination upon reaching a goal state,

narrowing task specification to recognizing success. On Reacher in simulation, policies trained under the minimum-time specification reached the goal two to three times faster than policies trained under guiding rewards, remained on target longer, and matched or surpassed them on the guiding-reward objective itself. The choice of time limit substantially affected how often the initial policy reached a goal state. The time limit is therefore better treated as a tunable solution parameter than as a fixed property of the problem. The number of target hits under the initial policy predicted how quickly and consistently learning succeeded across time limits in simulation, providing an inexpensive measurement to take before committing to a full training run. Guided by a target-hit threshold, agents using SAC learned pixel-based policies from scratch on four physical robot platforms within a few hours of experience, and learning was slower and less consistent under the one time limit that fell below the threshold. On the two robot arms, minimum-time policies again reached the goal marginally faster and remained on target longer than guiding-reward policies.

5.8 Limitations and Future Work

The central comparison in this chapter spans one family of tasks. Every task is a goal-reaching problem, and the claim that the minimum-time specification produces better final policies than guiding rewards rests on reaching variants in simulation and on robot arms. Whether the advantage holds on tasks with richer objectives, such as locomotion at speed or dexterous manipulation, is untested, and the claim should be read as scoped to goal reaching until then.

The comparison between the specifications also deserves a more even accounting of tradeoffs. The minimum-time specification assumes a reliable way to recognize success, and constructing one is not always free: each robot task in this chapter defines success through a hand-chosen threshold on the most recent image, whereas

the simulation tasks terminate on the simulator state. Some design effort therefore relocates from the reward to the termination condition. For tasks where success is easy to recognize but progress is hard to score, such as the goal-reaching tasks studied here, the minimum-time specification is the natural choice. For tasks where a meaningful progress measure is readily available and success is ambiguous, a guiding reward may be both easier to specify and faster to learn from. A systematic characterization of which task properties favor each specification remains open.

The target-hit measurement requires the initial policy to reach a goal state occasionally. On hard exploration problems where the initial policy never succeeds within a feasible budget, the measurement correctly forecasts failure but offers no remedy. Exploration strategies for sparse rewards, surveyed in Section 5.1, are complementary here: they aim to produce the early successes that the measurement requires. Curriculum learning (Narvekar et al. 2020) is also complementary: a sequence of tasks of increasing difficulty, each specified in minimum-time form, could carry the agent to competence on tasks where the initial policy never reaches a goal state.

The threshold of 10 target hits per 20K timesteps is a heuristic calibrated to one set of hyperparameters, and all results use a single algorithm, SAC. Whether the threshold transfers across algorithms, replay ratios, and batch sizes remains open.

The evaluation measures learning speed and final performance but not adaptation. The chapter motivates the minimum-time specification by competent performance within a practical budget and, beyond it, by continual adaptation. It evaluates the specification by learning curves within a fixed budget and by the policies those budgets produce, neither of which includes a change in the task. Whether a policy learned under the minimum-time specification adapts faster than one learned under a guiding reward when the goal or the dynamics change remains untested.

Chapter 6

Deep Policy Gradient Methods Without Batch Updates, Target Networks, or Replay Buffers

Learning directly on a physical robot constrains the memory and computation available to a reinforcement learning agent. Deep RL methods stabilize learning through mechanisms such as replay buffers, batch updates, and target networks, and each mechanism carries a memory or computation cost that can exceed the budget of an onboard computer. The ReLoD framework of Chapter 4 accommodates these costs by distributing computation to a remote workstation, at the price of a more complicated setup and a dependence on wireless connectivity. Learning with only onboard resources removes that dependence but requires an algorithm that forgoes the expensive stabilizing mechanisms. *Streaming learning*, in which the agent updates from each transition (an observation, action, reward and next observation) as it arrives and then discards it, eliminates the memory and computation costs of those mechanisms entirely. Agents trained with prior methods and deep neural networks, however, fail catastrophically in the streaming setting.

With the Action Value Gradient (AVG) algorithm, an agent learns stably in the streaming setting without replay buffers, batch updates, or target networks. An agent using AVG computes an entropy-regularized reparameterization policy gradient from the most recent transition and stabilizes its updates with observation

normalization, penultimate normalization, and TD-error normalization. AVG is the first algorithm to follow the reparameterization policy gradient theorem (Theorem 3) in the streaming setting: its actor update is the theorem’s single-sample estimate, taken from the on-policy stream, and the contribution of the chapter is making that update stable without batch mechanisms. In AVG, the three techniques together fill the stabilizing role that replay buffers, batch updates, and target networks serve in batch deep RL methods.

Experiments on simulated benchmarks and two physical robots established the effectiveness of AVG. On Gymnasium and DeepMind Control Suite tasks, agents trained with AVG matched the final performance of agents trained with SAC and a one-million-transition replay buffer on several tasks and surpassed them on the Dog tasks, whereas agents trained with the streaming baselines failed catastrophically. On a UR5 arm and an iRobot Create 2, agents learned control policies in roughly two hours of experience, with all computation for the mobile robot running onboard an edge device. In the survey presented in Chapter 3, I find that prior physical-robot evidence for streaming deep RL was limited to a small number of robot implementations and environments; the experiments of this chapter demonstrate stable streaming deep RL across a breadth of settings that the surveyed work does not reach.

6.1 Impact of Buffer Size on Deep Reinforcement Learning Methods

Reducing the buffer size from its large default value substantially degraded the learning performance of agents trained with three deep RL methods on four MuJoCo tasks (Todorov et al. 2012): Proximal Policy Optimization (PPO, Schulman et al. 2017), Soft Actor-Critic (SAC, Haarnoja et al. 2018a), and Twin Delayed Deep Deterministic Policy Gradient (TD3, Fujimoto et al. 2018). The buffer is the

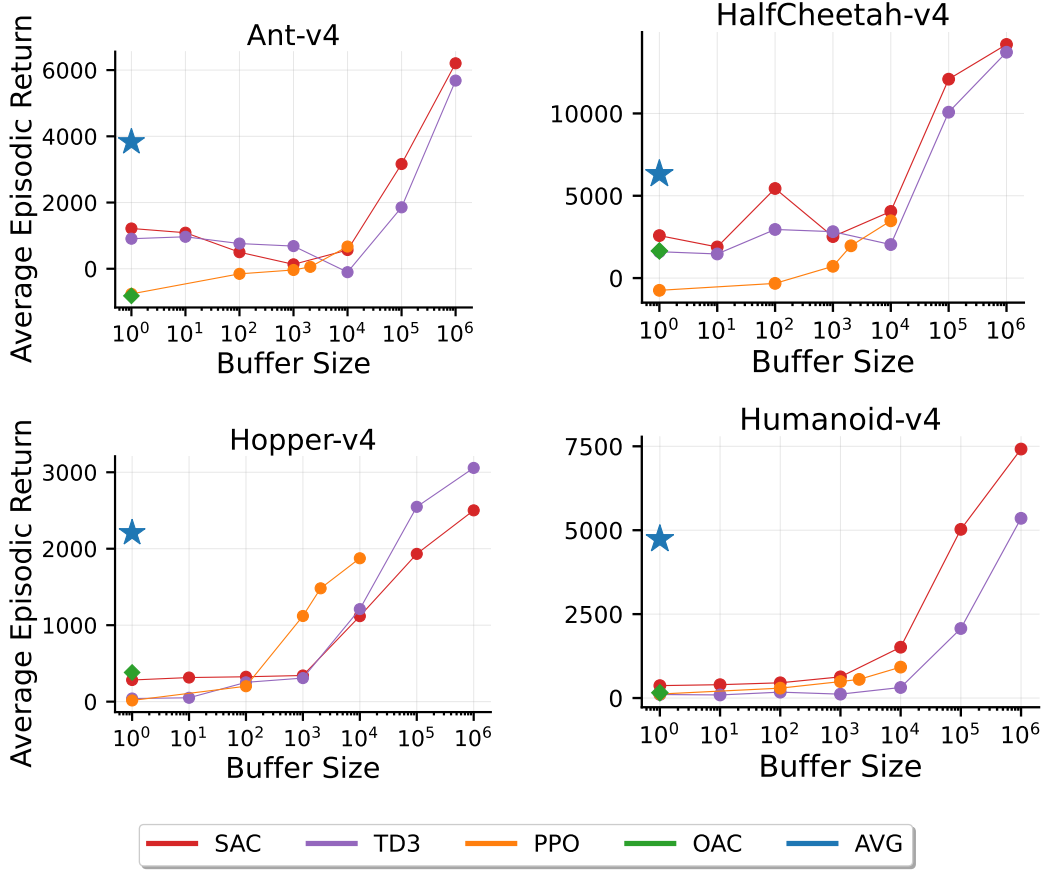


Figure 6.1: Impact of buffer size on PPO, SAC, and TD3. Reducing the buffer size degraded learning, and agents trained with all three methods failed when the buffer held only the most recent sample. AVG, which requires no replay buffer, is plotted at a buffer size of 1. Each point is the mean episodic return over the final 100K timesteps, averaged over 30 runs; all agents were trained for 10M timesteps.

replay buffer for SAC and TD3 and the rollout buffer for PPO, which is on-policy and stores only the most recently collected transitions. Agents trained with all three methods failed catastrophically when the buffer size was reduced to 1 (Figure 6.1). A buffer size of 1 corresponds to the streaming setting: each update uses only the most recent transition.

I used a minibatch size of 256 for TD3 and SAC and a minibatch size of 64 for PPO, following the default hyperparameter values reported in their respective papers. When the buffer was smaller than the minibatch, the entire buffer was used for each learning update. The comparison also included one-step actor-critic

(OAC, Algorithm 1) as a streaming baseline; agents trained with OAC failed to learn effectively when the policy and value function were represented by deep neural networks. Appendix A.5 lists the hyperparameter values for all methods.

6.2 The Action Value Gradient Method

Action Value Gradient (AVG) is a streaming actor-critic method. An agent using AVG updates its actor and critic from each transition as it arrives and then discards the transition (Algorithm 6). The method therefore requires neither a replay buffer nor minibatch updates.

The actor is a parameterized policy $\pi_{\theta}(a \mid s)$ with parameters θ . The critic $\hat{q}(s, a; \mathbf{w})$ has parameters $\mathbf{w}$ and estimates the soft action-value function $q_{\pi_{\theta}}^{\text{Ent}}(s, a)$ defined in Chapter 2. Both functions are represented by deep neural networks whose weights are initialized orthogonally (Saxe et al. 2013).

Drawing an action directly from the policy, $A \sim \pi_{\theta}(\cdot \mid S)$, does not by itself provide a differentiable path from the sampled action to the policy parameters. The reparameterization technique instead samples a noise vector $Z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and expresses the action as a deterministic differentiable function of Z and θ :

$$A_{\theta} \doteq g_{\theta}(Z; S) = \tanh(\boldsymbol{\mu}_{\theta}(S) + \boldsymbol{\sigma}_{\theta}(S) \odot Z),$$

where $\odot$ denotes elementwise multiplication and $\tanh$ is applied elementwise. The squashed normal policy produces actions in $(-1, 1)$ in each dimension. The policy density $\pi_{\theta}(a \mid s)$ includes the change-of-variables correction induced by the $\tanh$ transformation.

AVG builds on an entropy-regularized extension of the reparameterization policy gradient from Theorem 3; Appendix A.1 provides a derivation. For the entropy-regularized objective $J(\theta)$, the corresponding gradient direction is:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \propto \mathbb{E}_{S \sim d_{\pi_{\boldsymbol{\theta}}, \gamma}, A \sim \pi_{\boldsymbol{\theta}}} \left[\nabla_{\boldsymbol{\theta}} g_{\boldsymbol{\theta}}(z; S) \Big|_{z=g_{\boldsymbol{\theta}}^{-1}(A; S)} \nabla_a \left(q_{\pi_{\boldsymbol{\theta}}}^{\text{Ent}}(S, a) - \eta \log \pi_{\boldsymbol{\theta}}(a \mid S) \right) \Big|_{a=A} \right], \quad (6.1)$$

where $d_{\pi_{\boldsymbol{\theta}}, \gamma}$ is the normalized discounted state visitation distribution and $\eta \geq 0$ is the entropy coefficient. The inverse map recovers the noise z that produced the action A ; the first factor describes how the action would have shifted had the parameters shifted, with the noise held fixed. The second factor describes how the entropy-regularized policy-improvement objective changes with the action.

Both networks are updated from the most recent transition $(S, A_{\boldsymbol{\theta}}, R, S')$. Let $\tilde{S}$ and $\tilde{S}'$ denote the normalized network inputs corresponding to S and S' . The soft value of a terminal state is defined to be zero, and the bootstrap term therefore vanishes when S' is terminal. The TD error is

$$\delta \doteq \begin{cases} R + \gamma \left[\hat{q}(\tilde{S}', A'; \mathbf{w}) - \eta \log \pi_{\boldsymbol{\theta}}(A' \mid \tilde{S}') \right] - \hat{q}(\tilde{S}, A_{\boldsymbol{\theta}}; \mathbf{w}), & \text{if } S' \text{ is not terminal,} \\ R - \hat{q}(\tilde{S}, A_{\boldsymbol{\theta}}; \mathbf{w}), & \text{if } S' \text{ is terminal,} \end{cases}$$

where the agent samples $A' \sim \pi_{\boldsymbol{\theta}}(\cdot \mid \tilde{S}')$ when S' is not terminal. The normalized TD error is $\bar{\delta} \doteq \delta / \sigma_{\delta}$, where σ_{δ} is estimated online from reward, transition-discount, and return statistics.

The actor and critic gradients are computed at the current parameter values before either update is applied. The actor parameters are updated according to

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \nabla_{\boldsymbol{\theta}} \left[\hat{q}(\tilde{S}, A_{\boldsymbol{\theta}}; \mathbf{w}) - \eta \log \pi_{\boldsymbol{\theta}}(A_{\boldsymbol{\theta}} \mid \tilde{S}) \right], \quad (6.2)$$

where $\alpha^{\boldsymbol{\theta}} > 0$ is the actor step size and the gradient passes through the reparameterized action $A_{\boldsymbol{\theta}}$. Equation 6.2 is a single-sample estimate of the gradient di-

rection in Equation 6.1, with the visited state and the sampled noise standing in for the expectations. Writing the action as $A_\theta = g_\theta(Z; S)$ removes the inverse map because the noise that produced the action is available directly. Differentiating $\hat{q}(\tilde{S}, A_\theta; \mathbf{w}) - \eta \log \pi_\theta(A_\theta | \tilde{S})$ with respect to θ applies the chain rule through g_θ and yields the product of the two factors in Equation 6.1; the direct dependence of $\log \pi_\theta$ on θ contributes a term whose expectation is zero (Appendix A.1).

The critic parameters are updated according to

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \bar{\delta} \nabla_{\mathbf{w}} \hat{q}(\tilde{S}, \text{sg}(A_\theta); \mathbf{w}), \quad (6.3)$$

where $\alpha^{\mathbf{w}} > 0$ is the critic step size. The stop-gradient on A_θ in Equation 6.3 makes explicit that the critic update treats the sampled action as a fixed input; the update adjusts only $\mathbf{w}$. For clarity, Equations 6.2 and 6.3 write the optimizer steps as direct parameter updates. The experiments used the Adam optimizer to apply the displayed actor and critic gradients (Kingma & Ba 2015).

Streaming updates cannot reduce gradient variance by averaging over a mini-batch. The magnitude and variability of each update are therefore particularly important. Three normalization techniques improve learning stability in AVG: observation normalization, penultimate normalization, and TD-error normalization. All three operate without storing previous transitions. Throughout, $\varepsilon > 0$ is a small constant that prevents division by zero.

Observation normalization. The actor and critic receive the elementwise normalized input

$$\tilde{S} \doteq (S - \boldsymbol{\mu}_S) \oslash (\boldsymbol{\sigma}_S + \varepsilon),$$

where $\oslash$ denotes elementwise division and $\boldsymbol{\mu}_S$ and $\boldsymbol{\sigma}_S$ are running estimates of the observation mean and standard deviation. The statistics are estimated online

with Welford’s procedure (Welford 1962; Algorithm 4). The cumulative estimates weight all observations equally and require no averaging-rate hyperparameter. Observation normalization is also commonly used in methods such as PPO.

Penultimate normalization. For either network, let $\psi_{\boldsymbol{\theta}}(x)$ denote the output of its penultimate layer, that is, the last hidden layer, whose features the final linear layer operates on, where $(\boldsymbol{\vartheta}, x) = (\boldsymbol{\theta}, \tilde{S})$ for the actor and $(\boldsymbol{\vartheta}, x) = (\mathbf{w}, (\tilde{S}, A))$ for the critic. The vector is normalized as

$$\hat{\psi}_{\boldsymbol{\vartheta}}(x) \doteq \frac{\psi_{\boldsymbol{\theta}}(x)}{\max(\|\psi_{\boldsymbol{\theta}}(x)\|_2, \varepsilon)},$$

and gradients are computed through the normalization (Bjorck et al. 2022). The normalization is part of each network’s forward pass and therefore applies at every timestep, in both action computation and the update. Layer normalization (Ba et al. 2016), in contrast, subtracts the feature mean, divides by the feature standard deviation, and applies a learned gain and bias; penultimate normalization only divides by the Euclidean norm.

TD-error normalization. The TD error is normalized by a return-based estimate of its standard deviation (Schaul et al. 2021). Normalization here refers to division by the estimated standard deviation; it does not include mean subtraction. Let γ^{tr} denote the transition discount sample, defined as $\gamma^{\text{tr}} \doteq \gamma$ for a nonterminal transition and $\gamma^{\text{tr}} \doteq 0$ for a terminal transition. Thus, γ^{tr} is a random variable even though γ is a fixed hyperparameter. The normalization statistics use the sampled entropy-augmented reward $\tilde{R} \doteq R - \eta \log \pi_{\boldsymbol{\theta}}(A_{\boldsymbol{\theta}} \mid \tilde{S})$. Let $\tilde{G}$ denote the cumulative entropy-augmented reward from a completed episode. The TD-error variance is estimated as

$$\sigma_{\delta}^2 \doteq \mathbb{V}[\tilde{R}] + \mathbb{V}[\gamma^{\text{tr}}] \mathbb{E}[\tilde{G}^2].$$

The right-hand side approximates $\mathbb{V}[\delta]$ rather than equaling it exactly. The required quantities, $\mathbb{V}[\tilde{R}]$, $\mathbb{V}[\gamma^{\text{tr}}]$, and $\mathbb{E}[\tilde{G}^2]$, are estimated online with Welford’s procedure. In practice, sample estimates replace the expectations:

$$\sigma_\delta = \sqrt{\max\left(\hat{\mathbb{V}}[\tilde{R}] + \hat{\mathbb{V}}[\gamma^{\text{tr}}]\hat{\mathbb{E}}[\tilde{G}^2], \varepsilon^2\right)}.$$

Before the first episode has been completed, the agent uses $\sigma_\delta = 1$. TD-error normalization controls changes in error magnitude across tasks and stages of learning. It depends only on sampled rewards, transition discounts, and cumulative rewards, rather than on the internal parameters of the actor or critic, and requires constant memory and computation per transition.

Normalizing the TD error rather than the reward keeps the task unchanged. Scaling the reward by a fixed positive constant leaves the optimal policy unchanged, but shifting it does not, since a constant added to every timestep changes the value of ending an episode sooner. Reward normalization as used in PPO also estimates its scale online, and the reward and the return change with the estimate. Dividing the TD error by σ_δ instead leaves the reward and the return as the task defines them.

Connection between TD-error normalization and step-size adaptation. The TD error enters the critic update only as a scalar factor, and dividing it by σ_δ is therefore equivalent to dividing the critic’s step size by σ_δ ; the actor update contains no TD error and is unaffected. TD-error normalization is a step-size normalization for the critic, one scalar shared by all its parameters, with the scale taken from the returns rather than from the gradient. It leaves the critic’s fixed point unchanged, since a positive scaling of the update does not move its zero, and it is applied before Adam, which then rescales each parameter’s update by a running estimate of that parameter’s gradient magnitude.

Adam does not make the normalization redundant, because Adam has no access

to the scale of the targets. It normalizes each parameter’s update by that parameter’s gradient history, averaged over roughly the last thousand updates; when the return scale changes, as it did on Humanoid-v4 from tens to thousands, the history lags the change. A single large TD error then adds its squared gradient to the running average, which decays slowly. Every update that follows is divided by the inflated square root of that average, so the steps shrink for hundreds of updates until the average settles. TD-error normalization divides by a scale estimated from the returns themselves, and the update therefore tracks the target scale as it changes; it applies the same scalar to every critic parameter, which preserves the gradient’s direction and bounds only its magnitude before Adam rescales the update.

Convergence guarantees. The convergence properties of AVG are known only for a simplified variant. The paper introducing AVG (Vasan et al. 2024b) analyzes a variant that keeps the reparameterization gradient and the single-transition update but drops entropy regularization and normalization, and assumes a linear critic and independent samples. Under those conditions the policy gradient converges to a neighborhood of zero set by the critic’s approximation error, a stationary-point result rather than convergence to q_* . No result covers AVG as implemented, or any deep actor-critic method, including SAC and TD3; the stability of AVG is an empirical finding, and Section 6.4 examines the mechanisms behind it.

Algorithm 4 [Normalize](#) (Welford 1962)

Require: Input X ; count n ; running mean μ ; sum of squared deviations M ;

constant $\varepsilon > 0$

- 1: $n \leftarrow n + 1$
- 2: $d \leftarrow X - \mu$
- 3: $\mu \leftarrow \mu + d/n$
- 4: $d_2 \leftarrow X - \mu$
- 5: $M \leftarrow M + d \odot d_2$ $\triangleright$ Sum of squared deviations from the running mean
- 6: $\sigma \leftarrow \sqrt{\max(M/n, 0)}$
- 7: $X_{\text{norm}} \leftarrow d_2 \oslash (\sigma + \varepsilon)$
- 8: **return** $X_{\text{norm}}, n, \mu, M, \sigma$

Algorithm 5 [NormalizeTDError](#)

Require: Entropy-augmented reward $\tilde{R}$; transition discount γ^{tr} ; cumulative entropy-augmented reward $\tilde{G}$ or $\emptyset$; counts $\mathbf{n}_\delta$; running means $\boldsymbol{\mu}_\delta$; sums of squared deviations $\mathbf{M}_\delta$; constant $\varepsilon > 0$

- 1: $n_R, n_\gamma, n_G \leftarrow \mathbf{n}_\delta$; $\mu_R, \mu_\gamma, \mu_G \leftarrow \boldsymbol{\mu}_\delta$; $M_R, M_\gamma, M_G \leftarrow \mathbf{M}_\delta$
 $\triangleright \mu_G$ is the sample mean of $\tilde{G}^2$
- 2: $-, n_R, \mu_R, M_R, \sigma_R \leftarrow \text{Normalize}(\tilde{R}, n_R, \mu_R, M_R, \varepsilon)$
- 3: $-, n_\gamma, \mu_\gamma, M_\gamma, \sigma_\gamma \leftarrow \text{Normalize}(\gamma^{\text{tr}}, n_\gamma, \mu_\gamma, M_\gamma, \varepsilon)$
- 4: **if** $\tilde{G}$ is not $\emptyset$ **then**
- 5: $-, n_G, \mu_G, M_G, - \leftarrow \text{Normalize}(\tilde{G}^2, n_G, \mu_G, M_G, \varepsilon)$
- 6: **end if**
- 7: **if** $n_G \geq 1$ **then**
- 8: $\sigma_\delta \leftarrow \sqrt{\max(\sigma_R^2 + \mu_G \sigma_\gamma^2, \varepsilon^2)}$
- 9: **else**
- 10: $\sigma_\delta \leftarrow 1$
- 11: **end if**
- 12: $\mathbf{n}_\delta \leftarrow [n_R, n_\gamma, n_G]$; $\boldsymbol{\mu}_\delta \leftarrow [\mu_R, \mu_\gamma, \mu_G]$; $\mathbf{M}_\delta \leftarrow [M_R, M_\gamma, M_G]$
- 13: **return** $\sigma_\delta, \mathbf{n}_\delta, \boldsymbol{\mu}_\delta, \mathbf{M}_\delta$

Algorithm 6 Action Value Gradient (AVG)

```
1: Initialize  $\gamma, \eta, \alpha^\theta, \alpha^\mathbf{w}$ , and  $\varepsilon > 0$ ; actor parameters  $\theta$  and critic parameters  $\mathbf{w}$ 
   with orthogonal initialization
2: Normalize the penultimate-layer features in both the actor and critic networks
3:  $n \leftarrow 0, \quad \mu_S \leftarrow \mathbf{0}, \quad \mathbf{M}_S \leftarrow \mathbf{0}$ 
4:  $\mathbf{n}_\delta \leftarrow [0, 0, 0], \quad \mu_\delta \leftarrow [0, 0, 0], \quad \mathbf{M}_\delta \leftarrow [0, 0, 0]$ 
5: for each episode do
6:   Initialize  $S$  as the first state of the episode
7:    $S, n, \mu_S, \mathbf{M}_S, - \leftarrow \text{Normalize}(S, n, \mu_S, \mathbf{M}_S, \varepsilon)$ 
8:    $\tilde{G} \leftarrow 0$ 
9:   while  $S$  is not terminal do
10:     $A_\theta \leftarrow g_\theta(Z; S)$  where  $Z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
11:    Take action  $A_\theta$  and observe  $R$  and  $S'$ 
12:     $S', n, \mu_S, \mathbf{M}_S, - \leftarrow \text{Normalize}(S', n, \mu_S, \mathbf{M}_S, \varepsilon)$ 
13:     $\tilde{R} \leftarrow R - \eta \log \pi_\theta(A_\theta | S)$ 
14:     $\tilde{G} \leftarrow \tilde{G} + \tilde{R}$ 
15:    if  $S'$  is terminal then
16:       $\sigma_\delta, \mathbf{n}_\delta, \mu_\delta, \mathbf{M}_\delta \leftarrow \text{NormalizeTDError}(\tilde{R}, 0, \tilde{G}, \mathbf{n}_\delta, \mu_\delta, \mathbf{M}_\delta, \varepsilon)$ 
17:       $\delta \leftarrow R - \hat{q}(S, A_\theta; \mathbf{w})$ 
18:    else
19:       $\sigma_\delta, \mathbf{n}_\delta, \mu_\delta, \mathbf{M}_\delta \leftarrow \text{NormalizeTDError}(\tilde{R}, \gamma, \emptyset, \mathbf{n}_\delta, \mu_\delta, \mathbf{M}_\delta, \varepsilon)$ 
20:       $A' \sim \pi_\theta(\cdot | S')$ 
21:       $\delta \leftarrow R + \gamma [\hat{q}(S', A'; \mathbf{w}) - \eta \log \pi_\theta(A' | S')] - \hat{q}(S, A_\theta; \mathbf{w})$ 
22:    end if
23:     $\bar{\delta} \leftarrow \delta / \sigma_\delta$ 
24:     $\theta \leftarrow \theta + \alpha^\theta \nabla_\theta [\hat{q}(S, A_\theta; \mathbf{w}) - \eta \log \pi_\theta(A_\theta | S)]$ 
25:     $\mathbf{w} \leftarrow \mathbf{w} + \alpha^\mathbf{w} \bar{\delta} \nabla_\mathbf{w} \hat{q}(S, \text{sg}(A_\theta); \mathbf{w})$ 
26:     $S \leftarrow S'$ 
27:  end while
28: end for
```

6.3 AVG on Simulated Benchmark Tasks

Agents trained with AVG learned effectively across a suite of simulated benchmark tasks on which agents trained with prior streaming methods failed catastrophically. The comparison included OAC, a prior streaming method; agents using OAC with linear function approximation have learned in real time on simulated and physical-robot tasks (Degris et al. 2012a, Vasan & Pilarski 2017, Vasan & Pilarski 2018). The comparison also included streaming adaptations of SAC and TD3, which, like AVG, build on reparameterization policy gradients. To adapt SAC and TD3 to the streaming setting, the minibatch and replay buffer sizes were set to 1, so that experience was processed one transition at a time; these variants are referred to as *SAC-1* and *TD3-1*, where the suffix denotes both the replay buffer capacity and the minibatch size. Section 2.6 describes both methods, Algorithm 1 and Algorithm 3 give the OAC and SAC pseudocode respectively, and Appendix A.5 lists the hyperparameter values.

The hyperparameters of each method were optimized by random search, which is more efficient than grid search in high-dimensional search spaces (Bergstra & Bengio 2012). For each method, 300 hyperparameter configurations were evaluated; for each configuration, agents were trained with ten random seeds for 2M timesteps on five continuous-control environments: *Ant-v4*, *Hopper-v4*, *HalfCheetah-v4*, *Humanoid-v4*, and *Walker2d-v4*. Each configuration was ranked by its average undiscounted return per run, and the top configuration for each environment was then used to train agents for 10M timesteps with 30 random seeds. Appendix A.5 describes hyperparameter selection for the DeepMind Control Suite tasks.

Figure 6.2 compares the learning curves of agents trained with AVG, OAC, SAC-1, and TD3-1. For reference, the figure also shows the final performance of agents trained for 1M timesteps with SAC, a replay buffer of 1M transitions,

and default hyperparameters (the gray dashed line, referred to as *SAC*). Agents trained with AVG avoided catastrophic failure on every task, reached performance comparable to the SAC reference on several Gymnasium tasks (Towers et al. 2024), and surpassed it on the Dog tasks from the DeepMind Control Suite (Tassa et al. 2018). Nauman et al. (2024) suggest that non-default regularization, such as layer normalization, is necessary for agents trained with SAC to perform well on the Dog tasks.

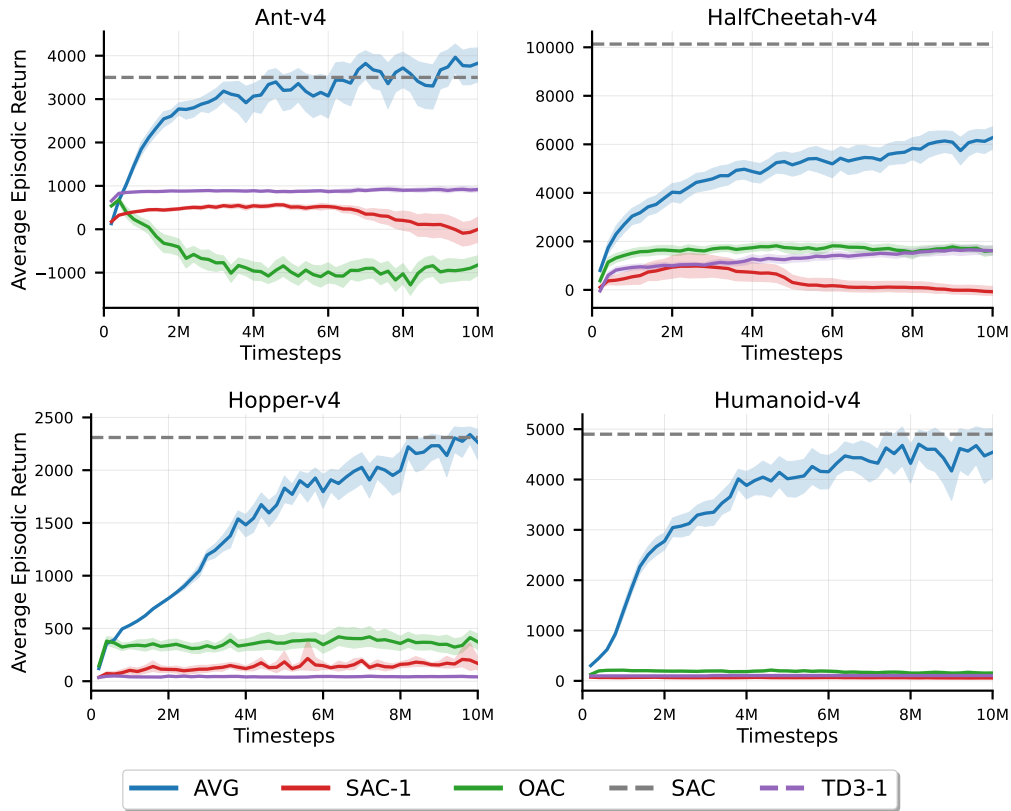


Figure 6.2: AVG on Gymnasium and DeepMind Control Suite tasks. Each solid curve is an average over 30 independent runs; shaded regions are 95% bootstrap confidence intervals. The gray dashed line, labeled *SAC*, shows the mean performance of agents trained with SAC and a replay buffer of 1M transitions over the final 10K timesteps of their 1M timesteps of training. Continued on the next page.

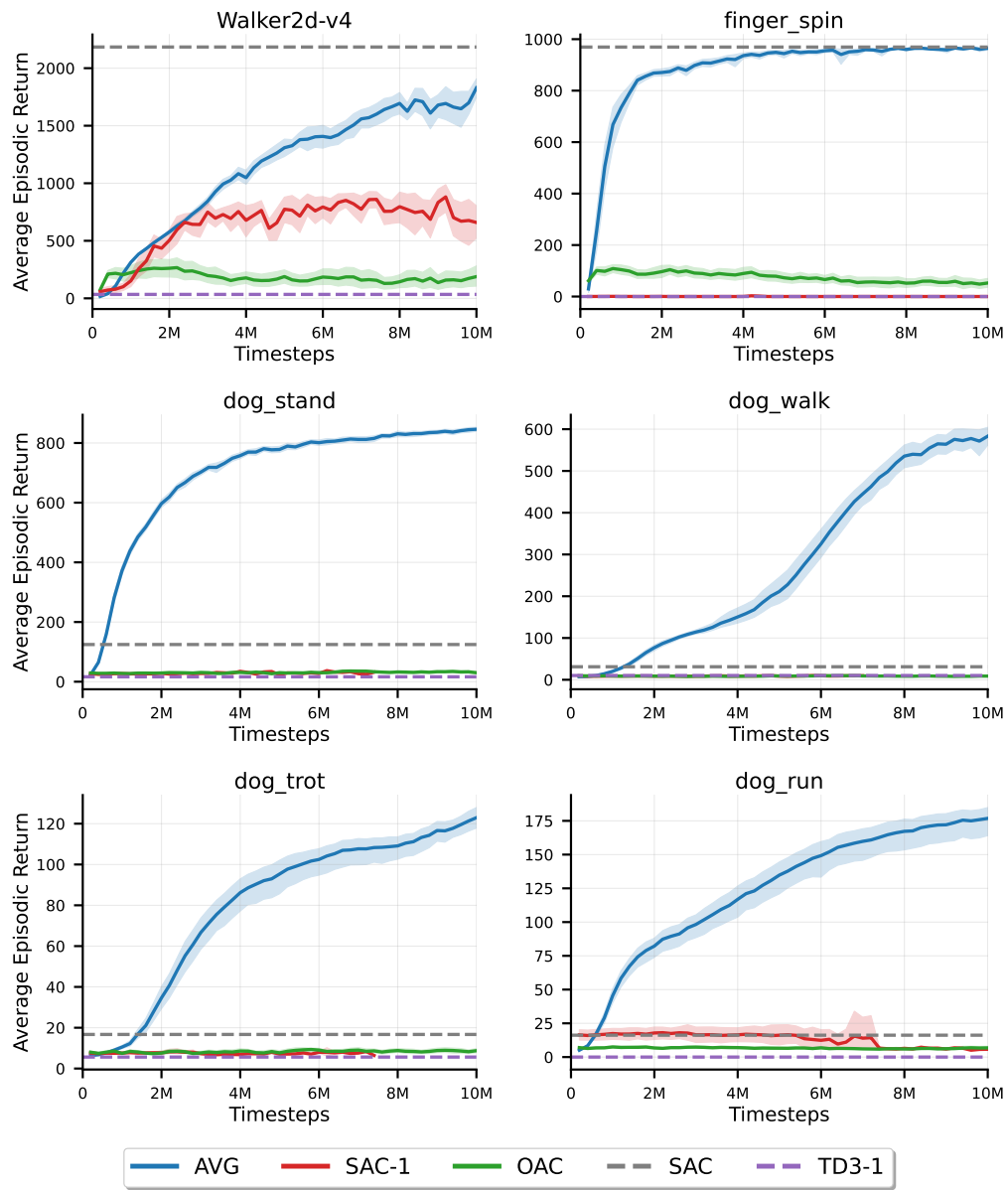


Figure 6.2: AVG on Gymnasium and DeepMind Control Suite tasks, continued.

6.4 Stabilizing Streaming Policy Gradient Methods

Agents using AVG learn stably where agents using unnormalized streaming methods fail, and the stability traces to the three normalization techniques. Without normalization, gradient norms exploded and learning failed; with all three techniques, agents achieved the best performance among the eight possible combinations. Applying the same techniques to other streaming methods recovered the performance of agents trained with OAC but not of agents trained with SAC-1 or TD3-1, and adding a target critic to AVG did not improve performance.

Instability Without Normalization

Instability in deep RL often manifests as high variance (Bjorck et al. 2022), reduced expressivity of the network over time (Nikishin et al. 2022, Sokar et al. 2023), or a gradual drop in performance (Dohare et al. 2024, Elsayed & Mahmood 2024, Elsayed et al. 2024), attributed largely to the non-stationarity of the data stream. Large regression target scales were identified as another common source of learning difficulty (Lyle et al. 2024). In the training runs of this chapter on Humanoid-v4, for instance, bootstrapped targets ranged from -20 to 8000 . The critic must therefore represent values that fluctuate widely across stages of training, which can produce excessively large TD errors and destabilize learning.

Agents trained with AVG without any normalization, a variant denoted AVG^- , failed on Humanoid-v4 with exploding gradient norms (Figure 6.3). The figure compares these agents with agents trained with SAC, which learned successfully. In SAC, batch updates smooth noisy gradients, and target critics stabilize the bootstrap targets; AVG^- includes neither mechanism. The gradient norms of the agents trained with AVG^- grew excessively large, particularly in the critic network. The mean absolute action, $\frac{1}{\dim(\mathcal{A})} \sum_i |a_i|$ averaged over recent timesteps, approached 1,

indicating that the squashed normal policy saturated at the action bounds.

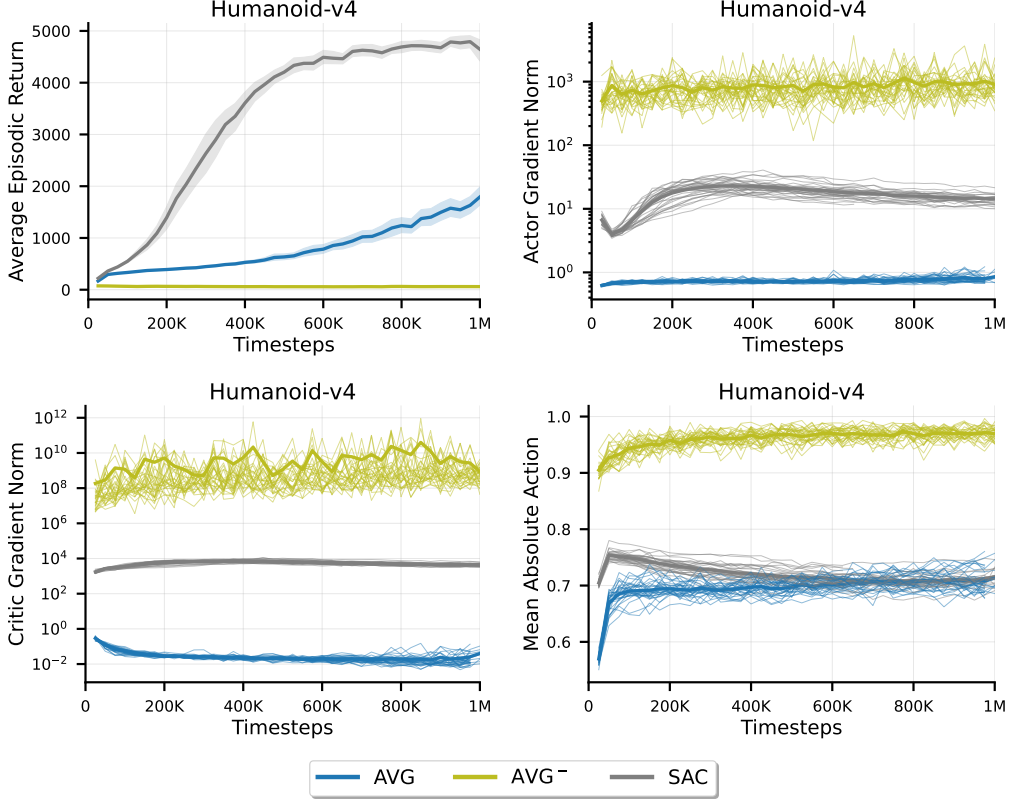


Figure 6.3: Average episodic returns, gradient norms of the actor and critic networks, and mean absolute action values, averaged over action dimensions, for AVG, AVG^- , and SAC on Humanoid-v4. AVG^- denotes AVG without any normalization. A mean absolute action value near 1 indicates saturation at the action bounds. Dark lines are averages over 30 runs; light lines are individual runs. The y-axes of the gradient-norm plots are logarithmic.

The figure invites two explanations, offered here as hypotheses consistent with the diagnostics rather than as demonstrated causes. The critic’s gradient norm grew far larger than the actor’s, by about nine orders of magnitude at its peak. The critic’s gradient, $\delta \nabla_{\mathbf{w}} \hat{q}$, is proportional to the TD error, which can be as large as the bootstrapped target when the estimate is far off. The actor’s gradient, $\nabla_{\theta} g_{\theta} \nabla_a \hat{q}$, contains no TD error and is multiplied by the derivative of the tanh squashing, which vanishes as the actions saturate. A critic that bootstraps on its own inflated estimates

can push its weights in a consistent direction at every update; Adam bounds the size of each parameter’s step, not the parameters or the values, and consistently directed bounded steps still grow the weights, with the gradient norm growing alongside. SAC’s batch updates, twin critics, and target critic each interrupt that growth.

The diagnostic comparison suggests that stable learning in AVG requires balancing update magnitudes across timesteps and episodes, reducing the influence of outlier experiences. Normalizing the inputs, the penultimate features, and the TD errors partly balances these magnitudes. Appropriately normalizing observations improves performance (Andrychowicz et al. 2021), likely because it improves learning dynamics (Sutton 1988b, Schraudolph 2002, LeCun et al. 2002). Normalizing hidden-layer activations serves a similar role inside the network: layer normalization (Ba et al. 2016) bounds the scale of the features, and recent work finds that it stabilizes value learning in deep RL (Ball et al. 2023, Lyle et al. 2023). Normalizing both the targets (e.g., by normalizing the rewards, Engstrom et al. 2019) and the observations is a well-established strategy in widely used algorithms such as PPO, improving performance and stability (Rao et al. 2020, Huang et al. 2022).

Disentangling the Effects of Normalization

Agents trained with the combination of all three techniques consistently achieved good performance: observation normalization maintains good learning dynamics (*norm_obs* in the figures), penultimate normalization reduces instability and improves plasticity (*pnorm*, Bjorck et al. 2022), and TD-error normalization resolves the problem of large bootstrapped target scales (*norm_td*, Schaul et al. 2021).

Welford’s procedure was selected for observation normalization because it maintains statistics over the entire data stream; recency-weighted estimates did not perform well in preliminary experiments. Penultimate normalization improves plasticity in a manner similar to layer normalization (Lyle et al. 2023), and agents

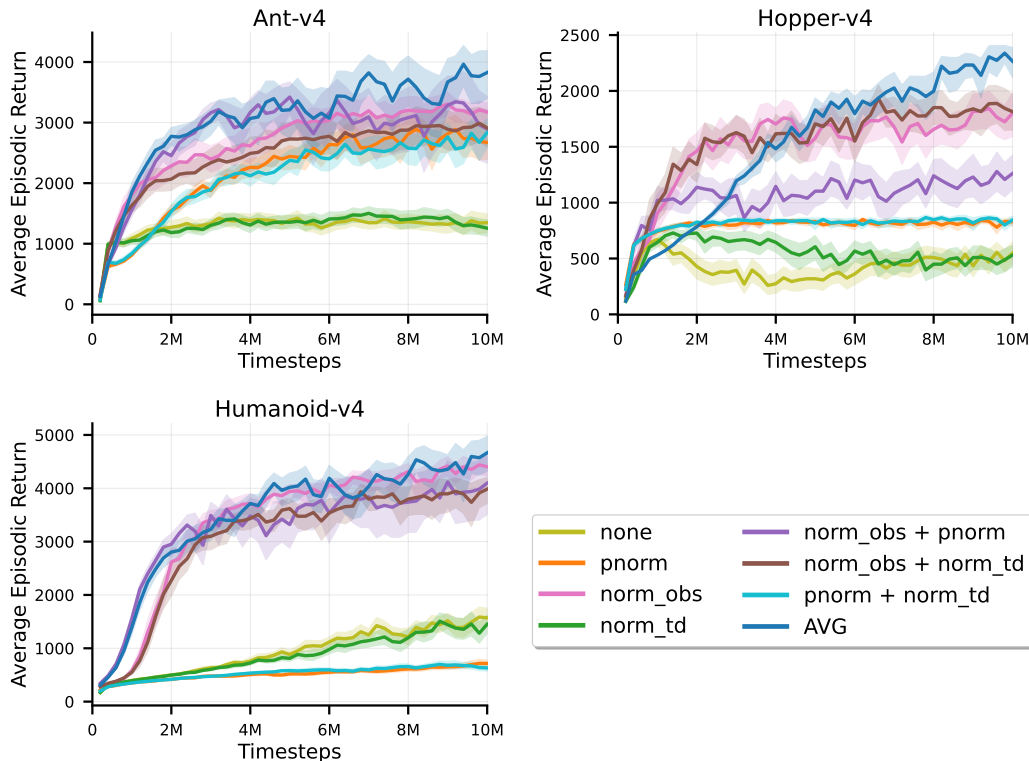


Figure 6.4: Ablation study of normalization techniques used with AVG. Each curve shows the best hyperparameter configuration for one variant on one task, averaged over 30 independent runs. The shaded regions represent a 95% confidence interval.

trained with penultimate normalization outperformed agents trained with layer normalization in the experiments of Appendix A.8 (Figure A.1). Schaul and colleagues illustrate the risks of clipping or normalizing rewards; these risks motivated normalizing the temporal-difference error instead (Schaul et al. 2021). Alternative normalization techniques could achieve similar or better outcomes; the emphasis is on the importance of normalization for streaming methods and on easy-to-use solutions.

The ablation evaluated the three techniques individually and in combination, yielding eight variants. For each variant, Figure 6.4 shows the learning curve of the best configuration found by random search (Appendix A.6). Agents trained with all three techniques achieved the best overall performance.

Impact of Normalization on OAC, SAC-1, and TD3-1

Normalization also improved learning with the other streaming methods (Figure 6.5); the suffix “+” denotes each method plus the three normalization techniques. Agents trained with OAC⁺ performed comparably to agents trained with AVG on most tasks. Agents trained with SAC-1⁺ were inconsistent, performing well on only two tasks and failing or diverging on others, including Hopper-v4 and Walker2d-v4, and agents trained with TD3-1⁺ failed to learn on every task. Among the methods studied, normalization was thus necessary but not sufficient for streaming deep RL: the recovery of agents trained with OAC⁺ shows that normalization can fill the stabilizing role of batch mechanisms for some streaming methods, and the failures of agents trained with SAC-1⁺ and TD3-1⁺ show that normalization alone does not suffice for every method.

Normalization alone did not salvage every method. The three normalization techniques were developed for AVG and tuned on it. They also helped OAC, which, like AVG, updates a single critic without a target network, but not SAC-1 or TD3-1, which keep the twin critics and target critics designed for batch updates. Each streaming method may need normalization suited to its own update, and the right set for each remains to be found. The recent Stream-X class of methods (Elsayed, Vasan & Mahmood 2024), discussed in Section 6.7, developed a different set for likelihood ratio and action-value methods, which supports the same conclusion.

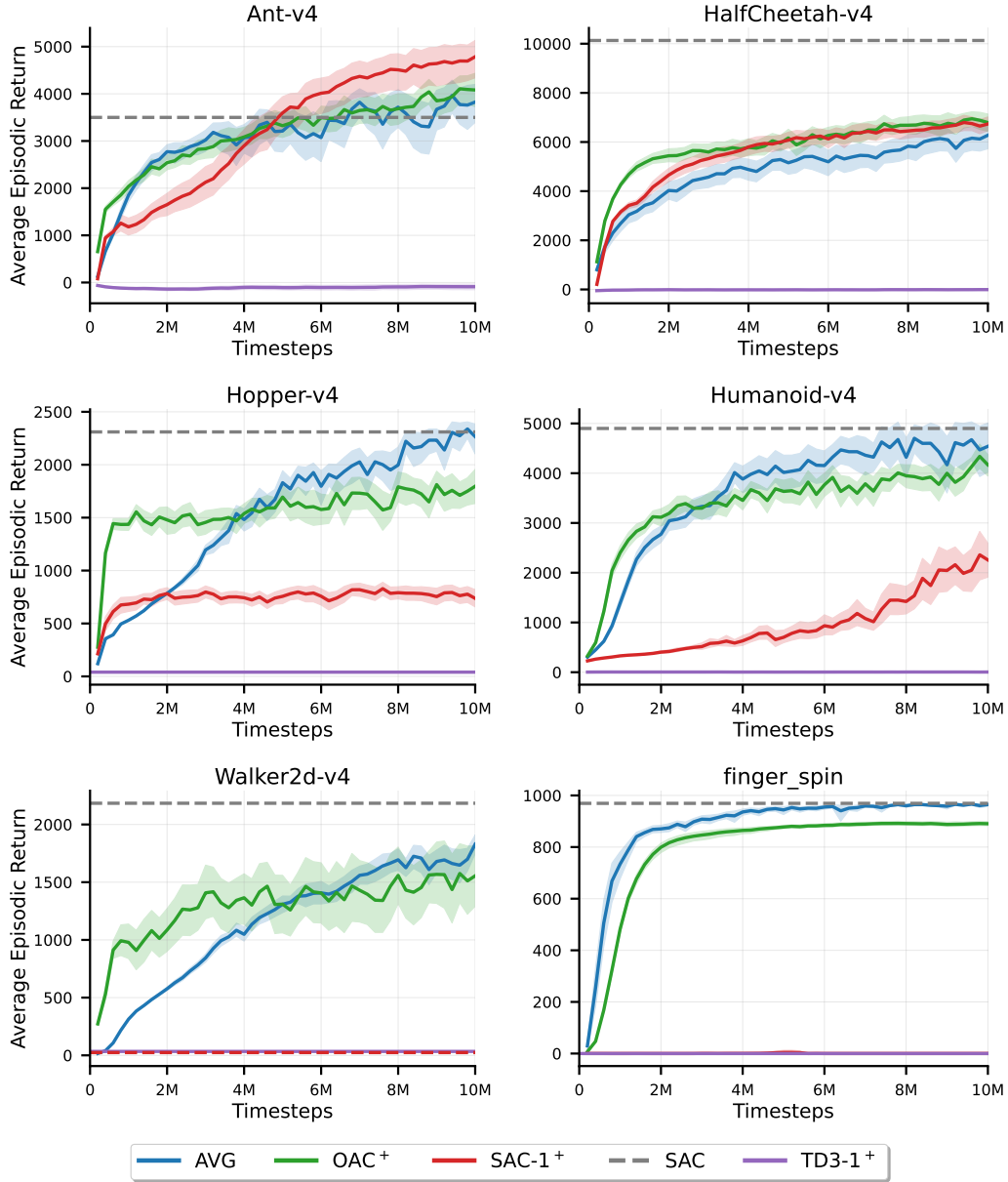


Figure 6.5: Impact of normalization on OAC, SAC-1 and TD3-1. Suffix “+” denotes each algorithm plus normalization. Each solid learning curve is an average of 30 independent runs. The shaded regions represent a 95% confidence interval. Continued on the next page.

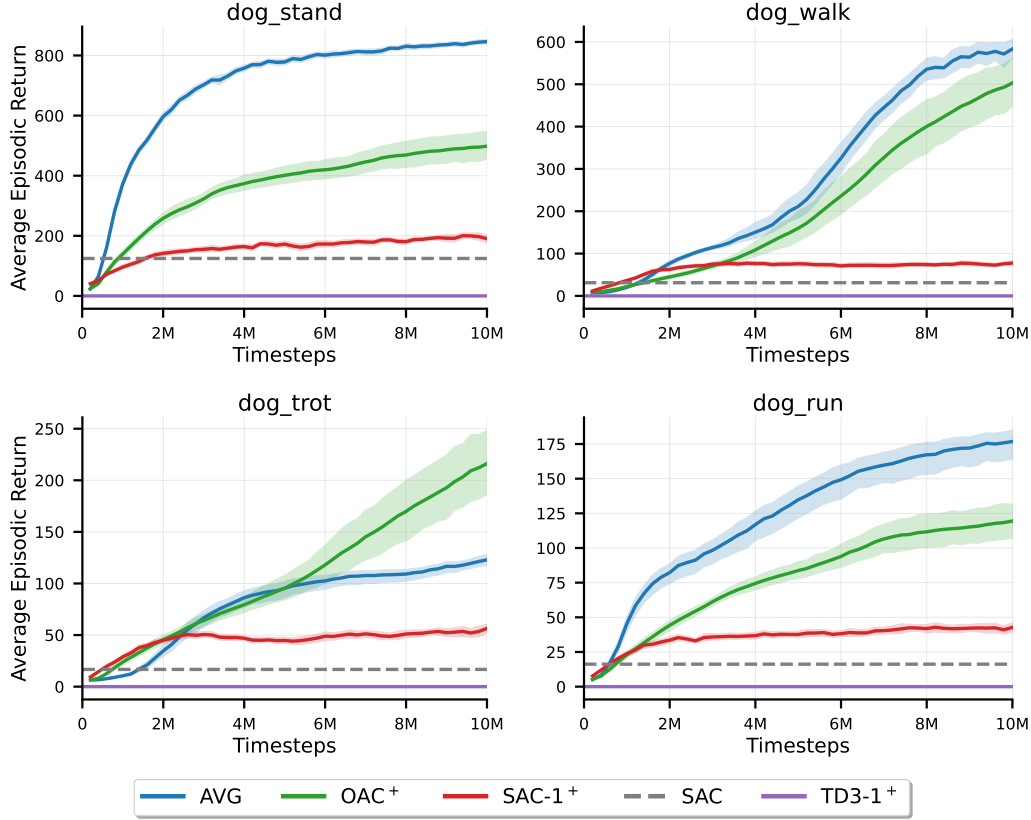


Figure 6.5: Impact of normalization on OAC, SAC-1 and TD3-1, continued.

AVG with Target Q-networks

Target networks are commonly used in off-policy batch methods to stabilize learning (Mnih et al. 2015). A separate network, updated less frequently, delays the propagation of value estimates. The delay helps maintain a stable target in batch methods with large replay buffers (Lillicrap et al. 2016, Fujimoto et al. 2018), but the same delay can slow learning when the agent learns from ongoing interaction (Kim et al. 2019).

Adding a target Q-network did not improve the performance of agents trained with AVG (Figure 6.6). As in SAC, the target Q-network was updated with Polyak averaging, $\bar{\mathbf{w}} \leftarrow \tau \mathbf{w} + (1 - \tau) \bar{\mathbf{w}}$, with τ varied over $[0, 1]$: a value of $\tau = 0$ corresponds to a fixed target Q-network, and $\tau = 1$ makes the target identical to

the current Q-network, that is, no target network (pseudocode in Appendix A.4, Algorithm 7). Performance comparable to AVG without a target Q-network was reached only with large values of τ . Omitting the target Q-network also reduces memory use and simplifies implementation.

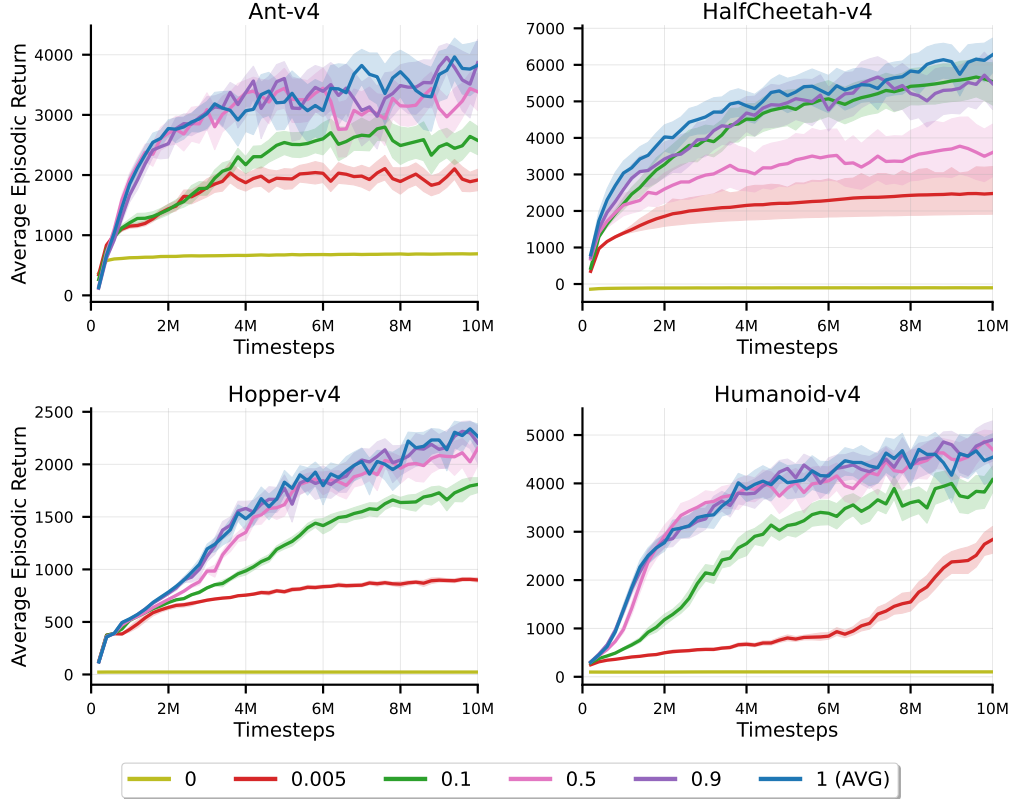


Figure 6.6: Impact of a target Q-network on AVG for different values of τ , the Polyak averaging coefficient. Here, $\tau = 0$ corresponds to a fixed target Q-network, and $\tau = 1$ makes the target identical to the current Q-network, that is, no target network. Each solid learning curve is an average of 30 independent runs. The shaded regions represent a 95% confidence interval.

6.5 AVG with Resource-Limited Robot Learning

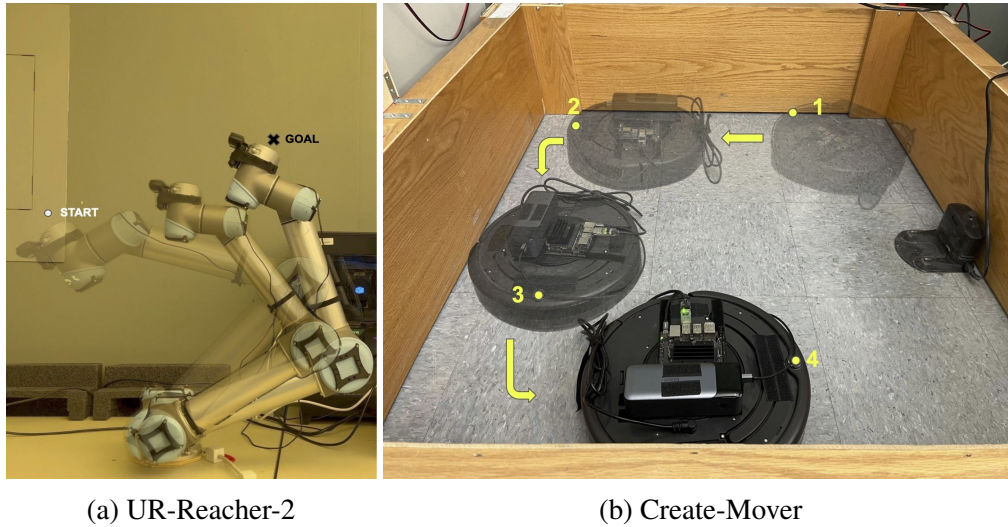


Figure 6.7: The two robot tasks: (a) a UR5 arm on UR-Reacher-2 and (b) an iRobot Create 2 on Create-Mover.

Agents using AVG learned control policies on two physical robots under limited memory and computation, in one case with all computation running onboard an edge device. The evaluation used two tasks developed by Mahmood et al. (2018b): *UR-Reacher-2* and *Create-Mover*.

UR-Reacher-2. The UR-Reacher-2 task adapts the MuJoCo Reacher task to a UR5 arm (Figure 6.7a). The agent moves the arm to reach arbitrary target positions on a plane by commanding angular velocities in $[-0.3, 0.3]$ rad/s for the second and third joints from the base. The observation vector consists of the joint angles, the joint velocities, the previous action, and the vector difference between the target and fingertip coordinates. The agent ran on a workstation with an AMD Ryzen Threadripper 2950X processor, an NVIDIA GeForce RTX 2080 Ti GPU, and 128 GB of memory.

Create-Mover. The Create-Mover task uses an iRobot Create 2, a hobbyist version of the Roomba (Figure 6.7b). The agent drives the robot forward as fast as possible within an enclosed arena. The arena measures 3.92×4.33 ft, which differs from the arena of the original task. The action space is $[-150, 150]^2$ mm/s; each component commands the speed of one wheel. The observation vector consists of six wall-sensor values and the previous action. Each observation uses the latest wall-sensor values received within the action cycle, converted to approximate distances. The reward is the sum of the directed distances reported in the ten most recent sensor packets. An episode lasts 90 seconds but ends early if the robot triggers a bump sensor. When an episode ends, the robot backs away from the wall so that the next episode does not begin with a bump. All computation for this task ran onboard a Jetson Nano with 4 GB of memory.

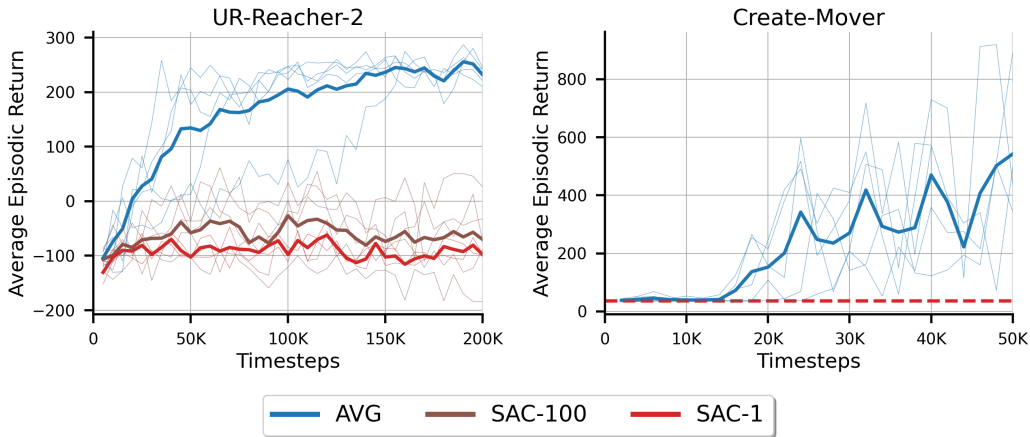


Figure 6.8: Learning curves on UR-Reacher-2 (left) and Create-Mover (right). Dark lines are averages over five runs; light lines are individual runs.

Agents trained with AVG learned effective policies on both tasks (Figure 6.8); each run required slightly over two hours of robot experience.¹ On UR-Reacher-2, the comparison included two memory-constrained variants of SAC: *SAC-1* (Section 6.3) and *SAC-100*, which uses a replay buffer and minibatch of 100 transitions.

¹A video of the learned policies is available at <https://youtu.be/cwwuN6Hyew0>.

Agents trained with either variant failed to learn a meaningful policy. On Create-Mover, sampling an action took 5 ms for both AVG and SAC-1, and one learning update took about 37 ms for AVG and 67 ms for SAC-1, within the 150 ms action cycle time. A batch update for SAC would have exceeded the action cycle time on the onboard computer, so the comparison on Create-Mover included only SAC-1, and agents trained with SAC-1 failed to learn a meaningful policy. Appendix A.9 gives hyperparameter values.

6.6 Conclusion

Streaming policy gradient methods are viable for deep RL and require less memory and computation than batch methods, an advantage for onboard robotic applications. With the Action Value Gradient algorithm, an agent learns stably in the streaming setting, and agents trained with AVG outperformed agents trained with prior streaming methods and resource-constrained batch methods on the simulated benchmarks and physical-robot tasks of this chapter. Normalization of the observations, the penultimate features, and the TD errors enables agents using AVG to learn stably even on high-dimensional control problems, filling the role that replay buffers, batch updates, and target networks serve in batch deep RL methods. The experiments on a robotic manipulator and a mobile robot extend the physical-robot evidence for streaming deep RL, which the survey in Chapter 3 finds limited to a small number of robot implementations and environments. AVG makes deep RL feasible within the limited onboard resources of robots, a step toward continual adaptation and long-term robot autonomy.

6.7 Limitations and Future Work

The main limitation of AVG is low sample efficiency: agents trained with AVG require more experience than agents trained with off-policy batch methods such as SAC. The performance of agents trained with AVG is also sensitive to the choice of hyperparameters; a valuable extension would reduce this sensitivity so that a single configuration performs well across environments. The formulation is limited to continuous action spaces but could be extended to discrete action spaces with the Gumbel-Softmax technique (Jang et al. 2017). The AVG update also omits the factor γ^t in the actor update, as the one-step actor-critic (Algorithm 1) does; the omission is common in practice and biases the update, and an existing correction addresses it (Che, Vasan & Mahmood 2023).

Low sample efficiency and hyperparameter sensitivity both follow from the single-transition update. Without a minibatch to average the gradient or a target network to steady the bootstrap target, each update carries the variance of one sample; the step size must therefore be tuned to the task, and more experience is needed to reach a given performance. Figure 6.2 compares agents trained with AVG for 10M timesteps with agents trained with SAC for 1M. Even at that ratio of timesteps, the agents trained with SAC used far more computation and memory: SAC updated from a minibatch of 256 at every timestep (Table A.4) and held a replay buffer of 1M transitions, about 256M transitions processed over 1M timesteps, whereas AVG processed one transition per update and held none, 10M transitions over 10M timesteps, roughly 25 times fewer. On a physical robot, experience costs more than computation, and the trade favors AVG only where batch methods cannot run, as on the two robot tasks of Section 6.5. Where a workstation is reachable, the distributed setup of Chapter 4 keeps the sample efficiency of batch methods.

Relation to Stream-X. Concurrent with the publication of AVG, the Stream-X class of methods stabilized streaming deep RL for prediction and control with methods such as $TD(\lambda)$, $AC(\lambda)$, and $Q(\lambda)$ (Elsayed, Vasan & Mahmood 2024). The two lines of work differ in the methods they cover and the means they use. Both normalize inputs and features. Stream-X adds reward normalization, eligibility traces (Chapter 12 of Sutton & Barto 2018), sparse initialization, and an optimizer that bounds the step size to prevent overshooting, and it was applied to likelihood ratio and action-value methods. AVG instead normalizes the TD errors and is the first streaming method built on the reparameterization policy gradient, the family to which SAC and TD3 belong. Nothing prevents the techniques of Stream-X from being applied to reparameterization methods; no streaming reparameterization method existed to apply them to before AVG. Neither work compares against the other because the two appeared at the same time. A comparison under matched conditions remains to be done, and replacing Adam with the bounded step-size optimizer (i.e., ObGD) of Stream-X may improve the robustness of AVG.

Second-order optimization. Natural policy gradient and other second-order methods have improved policy gradient learning with batch updates (Kakade 2001, Peters & Schaal 2008, Schulman et al. 2015), but they suit the streaming setting poorly. A second-order method needs an estimate of the curvature, the Fisher information matrix or the Hessian, and a single transition supplies only a rank-one sample of it. A usable estimate requires a batch of transitions or statistics accumulated over many timesteps, which lose accuracy as the policy changes. Storing and inverting even a block-diagonal approximation of the curvature (Martens & Grosse 2015) costs memory and computation that grow with parameter count, beyond the per-update budget of an onboard computer. One practical route is a curvature estimate updated from each transition, such as a running diagonal or Kronecker-

factored Fisher. For the actor, the compatible-features form of the natural gradient is another: incremental natural actor-critic algorithms obtain it at no extra cost with linear function approximation (Bhatnagar et al. 2007). Extending that form to neural networks and to the reparameterization gradient is an open problem.

Eligibility traces. The one-step update generalizes to a weighted average of n -step updates through eligibility traces, with λ setting the weights (Singh & Sutton 1996, van Hasselt et al. 2021); traces improved learning in the Stream-X class of methods, but they do not extend to AVG directly. The critic can use them as in Sarsa(λ), because its update is a temporal-difference update on action values. The actor cannot: a likelihood ratio actor, such as the actor in OAC, accumulates a trace of $\nabla_{\theta} \log \pi$ and multiplies the trace by the TD error, so that one error assigns credit to several past actions. The actor’s update in AVG, $\nabla_{\theta} \hat{q}(S, A_{\theta}; \mathbf{w})$, contains no TD error. The critic’s action gradient at the current state carries all the information the actor receives, and there is nothing for a trace to multiply. Multi-step credit can therefore reach the actor only through a critic trained toward λ -returns, and whether that indirect route recovers the gains traces gave Stream-X is left for future work.

Off-policy learning. AVG is on-policy, and an off-policy variant is a natural next step. Both updates use the transition the current policy just generated, and the critic bootstraps on an action sampled from the current policy, so no importance-sampling ratio appears in either. An off-policy variant would let the agent learn about many policies from one stream of experience. It would also let a safer or more exploratory policy control the robot while the agent learns about a target policy, and it would let the agent learn from demonstrations and stored experience, the pre-training direction of Chapter 7. Such a variant must correct the state distribution of both updates, because the states would be distributed according to the behavior policy. Off-policy actor-critic leaves the mismatch uncorrected for the actor (Degris

et al. 2012b); for the critic, the mismatch combined with bootstrapping and function approximation can cause divergence (Chapter 11 of Sutton & Barto 2018). Multi-step targets would in addition carry per-decision importance-sampling ratios, whose variance grows with the backup length unless truncated as in Retrace (Munos et al. 2016) or V-trace (Espeholt et al. 2018).

Learned models and shared representations. Two extensions could improve the sample efficiency of streaming methods such as AVG. An agent that learns auxiliary tasks over a shared representation gains an additional source of training signal from the same stream. A model-based RL agent learns a model of the environment and updates its policy and value function from simulated transitions as well as real ones, so that each real transition contributes to learning both directly and through the model (Sutton 1991). A model-based agent built on AVG, in the manner of Dyna, would learn the model, the critic, and the actor from the same stream and update the actor and critic from simulated transitions between real ones.

Continuing tasks and continual learning. A continuing task has no episode boundaries; in continual learning, the task or the environment changes over time. The actor and critic updates of AVG suit both: they use one transition at a time, need no episode boundary, and keep memory and computation constant as experience accumulates. For a continuing task, the TD-error scaling would replace the return of the completed episode (Algorithm 6) with a running estimate, and the average-reward objective, with differential values, could replace the discounted one (Wan et al. 2021). For continual learning, the normalization statistics accumulate over the entire stream by Welford’s procedure and would track a change in dynamics, observations, or rewards slowly; recency-weighted estimates, which performed worse on the fixed tasks of this chapter, may be needed instead. Testing AVG in either setting is left for future work; Chapter 7 discusses continual learning further.

Chapter 7

Conclusions and Future Work

7.1 Contributions

Robots that operate in the real world over long periods encounter conditions that their designers cannot anticipate. The prevailing practice in robot learning trains a policy before deployment and holds it fixed afterward, which leaves a deployed robot unable to update its behavior from experience collected after deployment. Real-time reinforcement learning removes the separation between training and deployment: the robot learns from its runtime experience. The dissertation set out to advance real-time RL on physical robots by addressing three practical demands: meeting action-cycle, computation, and memory constraints; reaching competent performance reliably within a practical wall-clock budget; and using rewards computable onboard and simple to specify across tasks. It makes three contributions toward that objective.

Computationally intensive RL methods require more computation and memory than most robots carry onboard. ReLoD addresses that constraint by supplementing the robot’s onboard computer with a remote workstation (Chapter 4). Experiments showed that the performance of an agent using Soft Actor-Critic degraded as on-

board computational resources were reduced, and that a robot using ReLoD recovered performance by drawing on remote computational resources during operation. With ReLoD, vision-based control policies were learned from scratch on physical robots within a few hours, including on a mobile robot carrying only an edge computing device. When a remote workstation is available, the choice of learning method is no longer bound by the robot’s onboard computer alone.

The minimum-time specification assigns a constant reward of -1 at each timestep and terminates the episode when the goal is reached. Under matched interaction budgets, final policies learned under the minimum-time task specification outperformed those learned under guiding-reward specifications on goal-reaching tasks (Chapter 5). The comparison covered four physical robots learning from raw-pixel observations without external instrumentation. Under the minimum-time specification, the burden of task specification shifts from reward design to exploration. For the goal-reaching tasks studied, one simple specification, computed from onboard sensing, served every task.

The Action Value Gradient algorithm demonstrates that an agent can learn stably with streaming policy gradient updates to deep neural networks (Chapter 6). An agent using AVG updates its parameters once from each arriving transition and retains no experience for replay. It therefore requires less memory and less computation per update than batch methods. Agents using AVG outperformed agents using existing streaming methods and resource-constrained batch methods across ten simulated continuous-control tasks. Normalization of the observations, the penultimate features, and the temporal-difference errors enabled agents using AVG to learn stably even on high-dimensional control problems, and may serve a stabilizing role analogous to replay buffers, batch updates, and target networks in batch deep RL methods. The experiments on a robotic manipulator and a mobile robot extend the physical-robot evidence for streaming deep RL, which the survey in Chapter 3 finds

limited to a small number of robot implementations and environments. An agent using AVG can learn on-device under computation and memory constraints, a step toward continual learning and adaptation in the real world.

The three contributions can compose into a single approach. ReLoD and AVG cover opposite ends of the computational spectrum: ReLoD applies when the robot can reach a remote workstation, and AVG runs within the robot’s onboard resources alone. Faster hardware will not merge the two ends: each hardware era in the survey brought methods that used the capacity of that era (Chapter 3), and the tasks that robots are expected to take on will demand more computation than the tasks solved in this dissertation. The gap between onboard and remote computation is likely to persist, and different deployment scenarios will continue to call for different solutions, whether fully on-device real-time learning or a distributed setup such as ReLoD. The minimum-time specification is independent of the learning method and pairs with either. For a goal-reaching task, a practitioner can therefore distribute the learning when remote computation is available, stream the updates on-device when it is not, and in both cases specify the task with a constant reward of -1 terminated at the goal. The dissertation evaluates each contribution separately; demonstrating the full composition on a single robot system remains for future work.

Each contribution suits a different case. ReLoD suits a robot that can maintain a connection to a workstation and a task that calls for a batch method, as the vision-based tasks of Chapter 4 did. AVG suits a robot that must learn on its own onboard computer. Examples include a rover beyond the reach of any workstation and a fleet with more robots than workstations to serve them. AVG also suits any deployment in which the computation and memory that streaming updates save can be spent on other work. The minimum-time specification suits any goal-reaching task in which the robot can detect the goal from its own sensing and the initial policy reaches it often enough, as it did in the tasks of Chapter 5; when the initial policy never

reaches the goal, a guiding reward or a curriculum of easier goals is needed first.

The contributions also serve long-term autonomy. A robot that operates autonomously for months or years must learn continually, acquiring new capabilities without discarding old ones as its world changes. Alternating phases of operation and retraining could supply adaptation, but the robot would respond to change only at the next retraining. Learning in real time removes the lag: the robot adapts as it operates. The empirical studies in the dissertation evaluated learning within bounded experimental runs, not adaptation across sequences of environmental or task changes. Reliable real-time learning is necessary, but not sufficient, for continual adaptation, and the dissertation provides evidence for it on physical robots, a step toward robots that learn throughout their operational lifetimes.

7.2 Directions for Future Work

Real-time reinforcement learning on physical robots remains an early-stage technology. I consider three directions that extend the contributions: combining pre-training with real-time RL, streaming model-based RL, and continual learning for long-term autonomy.

Combining Pre-Training with Real-Time Learning

A real-time learner can start from pre-trained parameters rather than from scratch. All experiments with physical robots in this dissertation involved learning from scratch, which required hours of robot interaction for each task. Learning from demonstrations and sim-to-real transfer avoid training on the robot by pre-training a policy that is deployed without further training, but the policy can degrade in situations that the demonstrations or the simulator did not cover, and correcting failures through further training is expensive and slow (Wagenmaker et al. 2024).

Pre-training and real-time learning are complementary: pre-training provides competence at deployment, and real-time learning can correct failures where they occur.

Recent methods combine pre-training with subsequent reinforcement learning. Reinforcement learning from prior data places offline transitions or demonstrations in a replay buffer and updates on minibatches that mix prior and new experience (Vecerik et al. 2017, Ball et al. 2023). Imitation-bootstrapped reinforcement learning trains a reinforcement learning policy alongside a pre-trained imitation policy that proposes candidate actions for exploration and for the bootstrap targets (Hu et al. 2024). Both approaches depend on replay buffers and batch updates, inheriting the computational demands of batch deep RL, and construct the learner’s remaining components, such as the value function, from scratch at deployment.

Meta-learning trains an agent across a distribution of tasks so that it learns a new task from the same distribution faster than from scratch (Thrun & Pratt 1998). Model-agnostic meta-learning (MAML) optimizes the initial parameters for a fixed adaptation procedure, a few gradient updates computed from batches of trajectories on the new task (Finn et al. 2017). As used, adaptation is performed once: the agent collects a few trajectories on the new task, updates, and then acts without further learning. Nothing in the procedure requires learning to stop there. The PPO agent of Chapter 4 also updates from batches of trajectories between episodes and continues throughout operation; pairing MAML with a learner of that kind would let the pre-trained initialization adapt in real time. The task distribution, however, must be built in advance, in practice in simulation, and adaptation is fast only on the variation it contains. Real-time learning is motivated by conditions no task distribution anticipates, and how to meta-train an agent that keeps learning under them is an open problem.

A promising path pre-trains the agent as a whole, not the policy alone. An agent whose value functions and other components are pre-trained alongside its policy

need not construct them from scratch at deployment, and if it learns by streaming updates, it need not carry a replay buffer onto the robot. Realizing that path requires care in setup, so that every component of the pre-trained agent remains usable by real-time learning at deployment.

Streaming Model-Based Reinforcement Learning

Model-based reinforcement learning offers a path to sample efficiency that model-free streaming methods, as developed in this dissertation, do not exploit. A model-based agent learns a world model, an approximation of the environment dynamics, and uses the model to generate simulated experience for updating value functions and policies (Sutton 1991), or to plan at decision time (Williams et al. 2017). An agent can gain sample efficiency by performing learning updates on simulated experience in addition to real experience, while an agent using a streaming method such as AVG updates only once per real transition. Current model-based deep RL methods rely on large replay buffers and batch updates over stored experience. Their demands therefore exceed the onboard resources of most robots (Hansen et al. 2024, Hafner et al. 2025). Streaming updates could make model-based learning feasible on small onboard computers.

Chapter 6 demonstrates streaming learning of the policy and the value function. Streaming learning of a world model with neural networks remains an open problem. Extending streaming deep RL methods to model learning would yield a fully streaming algorithm for model-based deep RL, one in which the policy, the value function, and the world model all update from each transition as it arrives. A fully streaming algorithm could scale its computation to the available resources, gaining sample efficiency by generating more simulated experience as computation grows, while storing no past experience.

Continual Learning for Long-Term Autonomy

Long-term autonomy in a changing world requires continual learning through shifts in the input distribution, changes in the environment dynamics, and sequences of tasks (Thrun & Mitchell 1995). Two failure modes obstruct continual learning: catastrophic forgetting, the degradation of performance on previously learned tasks as a new task is learned (McCloskey & Cohen 1989), and loss of plasticity, the declining ability of a network to learn as training proceeds (Dohare et al. 2024). Existing continual learning methods largely assume batch data and explicit task boundaries (Lesort et al. 2020, Powers 2023); a robot operating in the real world provides neither.

Onboard memory is fixed while experience accumulates without bound. A continual learner can therefore retain a subset or a summary of its experience but not all of it. Methods whose memory and computation do not grow with experience are suited to learning under a fixed resource budget. AVG meets that requirement: an agent using AVG updates from each transition as it arrives and discards it, keeping its memory and computation per timestep constant as experience accumulates. RL agents face non-stationarity even in a single stationary task: bootstrapping shifts the targets of the value function, and policy updates shift the data distribution. Normalization enabled stable learning under these shifts (Chapter 6), and penultimate normalization has been reported to preserve plasticity in a manner similar to layer normalization (Lyle et al. 2023). The runs of Chapter 6 lasted 10M timesteps on a single task and showed no loss of plasticity, but they did not test for it; a test would run an agent through a sequence of tasks or changes in dynamics and measure whether later tasks are learned as readily as earlier ones (Dohare et al. 2024). Whether streaming methods can mitigate catastrophic forgetting and maintain plasticity over such sequences remains an open question.

References

- Akkaya, I., Andrychowicz, M., Chociej, M., Litwin, M., McGrew, B., . . . , Zhang, L. (2019). Solving Rubik’s Cube with a robot hand. arXiv. <https://doi.org/10.48550/arXiv.1910.07113> (cited on pages 4 and 5)
- Amin, A., Aniceto, R., Balakrishna, A., Black, K., Conley, K., . . . , Zhou, Z. (2025). $\pi_{0.6}^*$: A VLA that learns from experience. arXiv. <https://doi.org/10.48550/arXiv.2511.14759> (cited on page 5)
- Amodei, D., & Clark, J. (2016). Faulty reward functions in the wild. <https://blog.openai.com/faulty-reward-functions> (cited on pages 14 and 74)
- Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, P., O., & Zaremba, W. (2017). Hindsight experience replay. *Advances in neural information processing systems*, 30. (cited on page 75)
- Andrychowicz, M., Raichuk, A., Stanczyk, P., Orsini, M., Girgin, S., Marinier, R., Hussenot, L., Geist, M., Pietquin, O., Michalski, M., et al. (2021). What matters in on-policy reinforcement learning? A large-scale empirical study. In *International Conference on Learning Representations*. (cited on page 110)
- Asada, M., Noda, S., Tawaratsumida, S., & Hosoda, K. (1996). Purposive behavior acquisition for a real robot by vision-based reinforcement learning. *Machine Learning*, 23(2), 279–303. (cited on pages 44 and 46)
- Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer normalization. arXiv. <https://arxiv.org/abs/1607.03426>

[//doi.org/10.48550/arXiv.1607.06450](https://doi.org/10.48550/arXiv.1607.06450) (cited on pages 100, 110, and 172)

Bagnell, J. A., & Schneider, J. C. (2001). Autonomous helicopter control using reinforcement learning policy search methods. In *Proceedings of IEEE International Conference on Robotics and Automation*. (cited on pages 47 and 48)

Ball, P. J., Smith, L., Kostrikov, I., & Levine, S. (2023). Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning*, 1577-1594. (cited on pages 110 and 127)

Barrett, S., Taylor, M. E., & Stone, P. (2010). Transfer learning for reinforcement learning on a physical robot. In *Ninth International Conference on Autonomous Agents and Multiagent Systems-Adaptive Learning Agents Workshop(1)*. (cited on page 46)

Barto, A. G., Sutton, R. S., & Anderson, C. (1983). Neuron-like elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man and Cybernetics*, 13, 835–846. (cited on page 21)

Bellman, R. E. (1957). *Dynamic Programming*. Princeton University Press, Princeton, NJ. (cited on page 14)

Benbrahim, H., Doleac, J. S., Franklin, J. A., & Selfridge, O. G. (1992). Real-time learning: A ball on a beam. In *IJCNN International Joint Conference on Neural Networks*, 1, 98–103. <https://doi.org/10.1109/IJCNN.1992.287219> (cited on pages 3, 44, and 46)

Benbrahim, H., & Franklin, J. A. (1997). Biped dynamic walking using reinforcement learning. *Robotics and Autonomous Systems*, 22(3–4), 283–302. (cited on pages 44 and 46)

Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(2). (cited on page 105)

- Bhatnagar, S., Ghavamzadeh, M., Lee, M., & Sutton, R. S. (2007). Incremental natural actor-critic algorithms. *Advances in neural information processing systems*, 20. (cited on page 121)
- Bi, T., & D’Andrea, R. (2024). Sample-efficient learning to solve a real-world labyrinth game using data-augmented model-based reinforcement learning. In *Proceedings of IEEE International Conference on Robotics and Automation*, 7455–7460. <https://doi.org/10.1109/ICRA57147.2024.10610577> (cited on page 47)
- Bjorck, J., Gomes, C. P., & Weinberger, K. Q. (2022). Is high variance unavoidable in RL? A case study in continuous control. In *International Conference on Learning Representations*. (cited on pages 100, 108, 110, and 172)
- Bloesch, M., Humplik, J., Patraucean, V., Hafner, R., Haarnoja, T., Byravan, A., . . . , Heess, N. (2022). Towards real robot learning in the wild: A case study in bipedal locomotion. In *Proceedings of the 5th Conference on Robot Learning*, 1502–1511. (cited on pages 47, 50, and 55)
- Bobrow, J. E., Dubowsky, S., & Gibson, J. S. (1985). Time-optimal control of robotic manipulators along specified paths. *The international journal of robotics research*, 4(3), 3-17. (cited on page 76)
- Booth, S., Knox, W. B., Shah, J., Niekum, S., Stone, P., & Allievi, A. (2023). The perils of trial-and-error reward design: Mismatch through overfitting and invalid task specifications. In *Thirty-Seventh AAAI Conference on Artificial Intelligence*, 5920–5929. <https://doi.org/10.1609/AAAI.V37I5.25733> (cited on pages 14, 15, 73, and 74)
- Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., . . . , & Zitkovich, B. (2022). RT-1: Robotics transformer for real-world control at scale. arXiv. <https://doi.org/10.48550/arXiv.2212.06817> (cited on page 5)

- Buchli, J., Stulp, F., Theodorou, E., & Schaal, S. (2011). Learning variable impedance control. *The International Journal of Robotics Research*, 30(7), 820–833. (cited on pages 47 and 49)
- Caarls, W., & Schuitema, E. (2015). Parallel online temporal difference learning for motor control. *IEEE Transactions on Neural Networks and Learning Systems*, 27(7), 1457–1468. (cited on pages 47 and 48)
- Che, F., Vasan, G., & Mahmood, A. R. (2023). Correcting discount-factor mismatch in on-policy policy gradient methods. In *Proceedings of the 40th International Conference on Machine Learning*, 4218–4240. (cited on pages 20, 47, 49, 77, and 119)
- Chebotar, Y., Hausman, K., Zhang, M., Sukhatme, G., Schaal, S., & Levine, S. (2017). Combining model-based and model-free updates for trajectory-centric reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning*, 703–711. (cited on pages 47 and 49)
- Chen, Y., Tian, S., Liu, S., Zhou, Y., Li, H., & Zhao, D. (2025). ConRFT: A reinforced fine-tuning method for VLA models via consistency policy. In *Robotics: Science and Systems*. <https://doi.org/10.15607/RSS.2025.XXI.019> (cited on page 47)
- Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., . . . , Song, S. (2024). Diffusion Policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*. (cited on page 4)
- Chui, C. K., & Chen, G. (2012). *Linear systems and optimal control (Vol. 18)*. Springer Science & Business Media. (cited on page 76)
- Degrave, J., Felici, F., Buchli, J., Neunert, M., Tracey, B., Carpanese, F., . . . , & Riedmiller, M. (2022). Magnetic control of tokamak plasmas through deep reinforcement learning. *Nature*, 602, 414–419. <https://doi.org/10.1038/s41586-021-04301-9> (cited on page 4)

- Daniel, C., Neumann, G., Kroemer, O., & Peters, J. (2016). Hierarchical relative entropy policy search. *Journal of Machine Learning Research*, 17(93), 1–50. (cited on pages 47 and 49)
- Degrís, T., Pilarski, P. M., & Sutton, R. S. (2012a). Model-free reinforcement learning with continuous action in practice. In *2012 American Control Conference (ACC)*, 2177–2182. <https://doi.org/10.1109/ACC.2012.6315022> (cited on pages 22, 45, 46, and 105)
- Degrís, T., White, M., & Sutton, R. S. (2012b). Off-policy actor-critic. In *Proceedings of the 29th International Conference on Machine Learning*. (cited on page 122)
- Deisenroth, M. P., Rasmussen, C. E., & Fox, D. (2011). Learning to control a low-cost manipulator using data-efficient reinforcement learning. In *Robotics: Science and Systems VII*. <https://doi.org/10.15607/RSS.2011.VII.008> (cited on page 47)
- Dohare, S., Hernandez-Garcia, J. F., Lan, Q., Rahman, P., Mahmood, A. R., & Sutton, R. S. (2024). Loss of plasticity in deep continual learning. *Nature*, 632(8026), 768–774. (cited on pages 108 and 129)
- Dorigo, M., & Colombetti, M. (1994). Robot shaping: Developing autonomous agents through learning. *Artificial Intelligence*, 71(2), 321–370. (cited on pages 44 and 46)
- Dulac-Arnold, G., Levine, N., Mankowitz, D. J., Li, J., Paduraru, C., Goyal, S., & Hester, T. (2021). Challenges of real-world reinforcement learning: Definitions, benchmarks and analysis. *Machine Learning*, 110(9), 2419–2468. <https://doi.org/10.1007/S10994-021-05961-4> (cited on page 5)
- Ebert, F., Finn, C., Dasari, S., Xie, A., Lee, A., & Levine, S. (2018). Visual foresight: Model-based deep reinforcement learning for vision-based robotic control. arXiv. <https://doi.org/10.48550/arXiv.1812.00568> (cited on page 75)

Edwards, A. L., Dawson, M. R., Hebert, J. S., Sutton, R. S., Chan, K. M., & Pilarski, P. M. (2014). Adaptive switching in practice: Improving myoelectric prosthesis performance through reinforcement learning. *Proceedings of MEC, 14*, 18–22.

(cited on pages 44 and 46)

Elsayed, M., & Mahmood, A. R. (2024). Addressing loss of plasticity and catastrophic forgetting in continual learning. In *International Conference on Learning Representations*. (cited on page 108)

Elsayed, M., Lan, Q., Lyle, C., & Mahmood, A. R. (2024). Weight clipping for deep continual and reinforcement learning. *Reinforcement Learning Journal, 5*, 2198–2217. (cited on page 108)

Elsayed, M., Vasan, G., & Mahmood, A. R. (2024). Streaming deep reinforcement learning finally works. arXiv. <https://doi.org/10.48550/arXiv.2410.14606> (cited on pages 112 and 120)

Endo, G., Morimoto, J., Matsubara, T., Nakanishi, J., & Cheng, G. (2008). Learning CPG-based biped locomotion with a policy gradient method: Application to a humanoid robot. *International Journal of Robotics Research, 27*(2), 213–228.

(cited on pages 44 and 46)

Engstrom, L., Ilyas, A., Santurkar, S., Tsipras, D., Janoos, F., Rudolph, L., & Madry, A. (2019). Implementation matters in deep RL: A case study on PPO and TRPO. In *International Conference on Learning Representations*. (cited on page

110)

Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., ... & Kavukcuoglu, K. (2018). Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. In *Proceedings of the 35th International Conference on Machine Learning*, 1407-1416. (cited on page 122)

Fagg, A. H., Lotspeich, D. L., Hoff, J., & Bekey, G. A. (1998). Rapid reinforcement

- learning for reactive control policy design for autonomous robots. In *Artificial Life in Robotics*. (cited on pages 44 and 46)
- Fan, K., Wang, Z., Beck, J., Kwok, J., & Heller, K. A. (2015). Fast second order stochastic backpropagation for variational inference. *Advances in Neural Information Processing Systems*, 28. (cited on page 27)
- Farrahi, H., & Mahmood, A. R. (2025). Learning without time-based embodiment resets in Soft Actor-Critic. In *Proceedings of the 5th Conference on Lifelong Learning Agents*. (cited on page 47)
- Feng, Y., Hansen, N., Xiong, Z., Rajagopalan, C., & Wang, X. (2023). Finetuning offline world models in the real world. In *Proceedings of the 7th Conference on Robot Learning*, 425–445. (cited on page 47)
- Fidelman, P., & Stone, P. (2004). Learning ball acquisition on a physical robot. In *2004 International Symposium on Robotics and Automation (ISRA)*. (cited on pages 47 and 48)
- Finn, C., Abbeel, P., & Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning*. 1126-1135. (cited on page 127)
- Fu, J., Kumar, A., Nachum, O., Tucker, G., & Levine, S. (2020). D4RL: Datasets for deep data-driven reinforcement learning. arXiv. <https://doi.org/10.48550/arXiv.2004.07219> (cited on page 4)
- Fujimoto, S., van Hoof, H., & Meger, D. (2018). Addressing function approximation error in actor-critic methods. In *Proceedings of the 35th International Conference on Machine Learning*, 1582–1591. (cited on pages 29, 30, 95, and 114)
- Garg, S., Tosatto, S., Pan, Y., White, M., & Mahmood, A. R. (2022). An alternate policy gradient estimator for softmax policies. In *International Conference on Artificial Intelligence and Statistics*. (cited on page 76)

- Gaskett, C., Fletcher, L., & Zelinsky, A. (2000). Reinforcement learning for a vision based mobile robot. In *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems, 1*, 403–409. (cited on pages 3, 45, and 46)
- Geng, T., Porr, B., & Wörgötter, F. (2005). Fast biped walking with a reflexive controller and real-time policy searching. *Advances in Neural Information Processing Systems*. (cited on pages 45 and 46)
- Geramifard, A., Walsh, T. J., Tellex, S., Chowdhary, G., Roy, N., & How, J. P. (2013). A tutorial on linear function approximators for dynamic programming and reinforcement learning. *Foundations and Trends in Machine Learning*, 6(4), 375–451. <https://doi.org/10.1561/22000000042> (cited on page 17)
- Glynn, P. W. (1990). Likelihood ratio gradient estimation for stochastic systems. *Communications of the ACM*, 33(10), 75–84. (cited on page 19)
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. (cited on page 37)
- Graves, A. (2016). Adaptive computation time for recurrent neural networks. arXiv. <https://doi.org/10.48550/arXiv.1603.08983> (cited on page 71)
- Grooten, B., Tomilin, T., Vasan, G., Taylor, M. E., Mahmood, A. R., Fang, M., . . . , Mocanu, D. C. (2024). MaDi: Learning to mask distractions for generalization in visual deep reinforcement learning. In *The 23rd International Conference on Autonomous Agents and Multiagent Systems*. (cited on pages 47 and 91)
- Gu, S., Holly, E., Lillicrap, T., & Levine, S. (2017). Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates. In *Proceedings of IEEE International Conference on Robotics and Automation*, 3389–3396. (cited on pages 4, 47, and 49)
- Gullapalli, V., Franklin, J., & Benbrahim, H. (1994). Acquiring robot skills via reinforcement learning. *Control Systems Magazine*, 14(1), 13–24, IEEE. (cited on

- Günther, J., Pilarski, P. M., Helfrich, G., Shen, H., & Diepold, K. (2016). Intelligent laser welding through representation, prediction, and control learning: An architecture with deep neural networks and reinforcement learning. *Mechatronics*, 34, 1–11. (cited on pages 44 and 46)
- Ha, S., Xu, P., Tan, Z., Levine, S., & Tan, J. (2020). Learning to walk in the real world with minimal human effort. In *4th Conference on Robot Learning*, 1110–1120. (cited on pages 47 and 49)
- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018a). Soft Actor-Critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, 1861–1870. (cited on pages 29, 32, 37, 56, and 95)
- Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., . . . , & Levine, S. (2018b). Soft actor-critic algorithms and applications. arXiv. <https://doi.org/10.48550/arXiv.1812.05905> (cited on page 34)
- Haarnoja, T., Ha, S., Zhou, A., Tan, J., Tucker, G., & Levine, S. (2019). Learning to walk via deep reinforcement learning. In *Robotics: Science and Systems XV*. <https://doi.org/10.15607/RSS.2019.XV.011> (cited on pages 3, 5, 47, 49, and 55)
- Hafner, R., & Riedmiller, M. (2003). Reinforcement learning on an omnidirectional mobile robot. In *Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 1*, 418–423. (cited on pages 3, 47, and 49)
- Hafner, R., Hertweck, T., Klöppner, P., Bloesch, M., Neunert, M., Wulfmeier, M., . . . , & Riedmiller, M. (2021). Towards general and autonomous learning of core skills: A case study in locomotion. In *Conference on Robot Learning*, 1084–1099. (cited on pages 3, 16, 47, 49, and 86)

- Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2025). Mastering diverse control tasks through world models. *Nature*, 640(8059), 647–653. (cited on page 128)
- Hailu, G., & Sommer, G. (1998). Integrating symbolic knowledge in reinforcement learning. In *IEEE International Conference on Systems, Man and Cybernetics (SMC)*. (cited on pages 45 and 46)
- Hansen, N., Su, H., & Wang, X. (2024). TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*. (cited on page 128)
- Hay, N., Russell, S., Tolpin, D., & Shimony, S. E. (2012). Selecting computations: Theory and applications. In *Proceedings of the Twenty-Eighth Conference on Uncertainty in Artificial Intelligence*, 346–355. (cited on page 71)
- Heess, N., Wayne, G., Silver, D., Lillicrap, T., Erez, T., & Tassa, Y. (2015). Learning continuous control policies by stochastic value gradients. *Advances in neural information processing systems*, 28. (cited on page 29)
- Hertweck, T., Riedmiller, M., Bloesch, M., Springenberg, J. T., Siegel, N., Wulfmeier, M., . . . , & Heess, N. (2020). Simple sensor intentions for exploration. arXiv. <https://doi.org/10.48550/arXiv.2005.07541> (cited on pages 47, 49, and 75)
- Hester, T., Quinlan, M., & Stone, P. (2012). RTMBA: A real-time model-based reinforcement learning architecture for robot control. In *Proceedings of IEEE International Conference on Robotics and Automation*, 85–90. (cited on pages 47 and 48)
- Hester, T., & Stone, P. (2013). TEXPLORE: Real-time sample-efficient reinforcement learning for robots. *Machine Learning*, 90(3), 385–429. (cited on pages 47 and 48)
- Hu, H., Mirchandani, S., & Sadigh, D. (2024). Imitation bootstrapped reinforcement learning. In *Robotics: Science and Systems*. (cited on page 127)

Huang, S., Dossa, R. F. J., Raffin, A., Kanervisto, A., & Wang, W. (2022). The 37 implementation details of proximal policy optimization. In *ICLR Blog Track*.

(cited on page 110)

Huber, M., & Grupen, R. A. (1997). A feedback control structure for on-line learning tasks. *Robotics and Autonomous Systems*, 22(3–4), 303–315. (cited on pages 44 and

46)

Ibarz, J., Tan, J., Finn, C., Kalakrishnan, M., Pastor, P., & Levine, S. (2021). How to train your robot with deep reinforcement learning: Lessons we have learned. *The International Journal of Robotics Research*, 40(4–5), 698–721. (cited on pages 3, 5, 37,

and 86)

Ijspeert, A. J., Nakanishi, J., & Schaal, S. (2002). Movement imitation with non-linear dynamical systems in humanoid robots. In *Proceedings of IEEE International Conference on Robotics and Automation*, 2, 1398–1403. <https://doi.org/10.1109/ROBOT.2002.1014739> (cited on page 48)

Jaakkola, T., Singh, S., & Jordan, M. (1994). Reinforcement learning algorithm for partially observable Markov decision problems. *Advances in neural information processing systems*, 7. (cited on page 15)

Jang, E., Gu, S., & Poole, B. (2017). Categorical reparameterization with Gumbel-Softmax. In *International Conference on Learning Representations*. (cited on page 119)

Javed, K., Modayil, J., Kennickell, G., Sutton, R. S., & Carmack, J. (2026). Physical atari: A robust and accessible platform for real-time reinforcement learning on robots. *Reinforcement Learning Journal*. (cited on pages 41 and 47)

Julian, R., Swanson, B., Sukhatme, G., Levine, S., Finn, C., & Hausman, K. (2021). Never stop learning: The effectiveness of fine-tuning in robotic reinforcement learning. In *Proceedings of the Conference on Robot Learning*, 2120–2136. (cited

on page 47)

- Kahn, M. E., & Roth, B. (1971). The near-minimum-time control of open-loop articulated kinematic chains. *Journal of dynamic systems, measurement, and control* 93(3), 164-172. (cited on page 76)
- Kakade, S. M. (2001). A natural policy gradient. *Advances in neural information processing systems*, 14. (cited on page 120)
- Kalakrishnan, M., Righetti, L., Pastor, P., & Schaal, S. (2011). Learning force control policies for compliant manipulation. In *Proceedings of the 2011 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 4639–4644. <http://doi.org/10.1109/IRoS.2011.6095096> (cited on page 47)
- Kalashnikov, D., Irpan, A., Pastor, P., Ibarz, J., Herzog, A., Jang, E., . . . , & Levine, S. (2018). Scalable deep reinforcement learning for vision-based robotic manipulation. In *Conference on Robot Learning*, 651–673. (cited on pages 47 and 49)
- Kalmár, Z., Szepesvári, C., & Lorincz, A. (1997). Modular reinforcement learning: An application to a real robot task. In *European Workshop on Learning Robots*, 29–45. (cited on page 47)
- Katsikopoulos, K. V., & Engelbrecht, S. E. (2003). Markov decision processes with delays and asynchronous cost collection. *IEEE Transactions on Automatic Control*, 48(4), 568–574. (cited on page 42)
- Kelly, I. D., & Keating, D. A. (1998). Increased learning rates through the sharing of experiences of multiple autonomous mobile robot agents. In *1998 IEEE International Conference on Fuzzy Systems Proceedings*, 1, 129–134. (cited on pages 44 and 46)
- Kim, S., Asadi, K., Littman, M., & Konidaris, G. (2019). DeepMellow: Removing the need for a target network in deep Q-learning. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*. (cited on page 114)

- Kimura, H., Yamashita, T., & Kobayashi, S. (2001). Reinforcement learning of walking behavior for a four-legged robot. In *Proceedings of the 40th IEEE Conference on Decision and Control*, 1, 411–416. <https://doi.org/10.1109/CD C.2001.980135> (cited on pages 44 and 46)
- Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. In *International Conference on Learning Representations*. (cited on page 27)
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations*. (cited on page 99)
- Knox, W. B., Allievi, A., Banzhaf, H., Schmitt, F., & Stone, P. (2023). Reward (mis)design for autonomous driving. *Artificial Intelligence*, 316, 103829. (cited on pages 73 and 74)
- Kober, J., Mohler, B., & Peters, J. (2008a). Learning perceptual coupling for motor primitives. In *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 834–839. (cited on pages 47 and 48)
- Kober, J., & Peters, J. (2008b). Policy search for motor primitives in robotics. *Advances in Neural Information Processing Systems*, 21. (cited on pages 15, 47, and 48)
- Kober, J., Oztop, E., & Peters, J. (2010). Reinforcement learning to adjust robot movements to new situations. In *Robotics: Science and Systems (R:SS)*. (cited on pages 47 and 49)
- Kober, J., Bagnell, J. A., & Peters, J. (2013). Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11), 1238–1274. <https://doi.org/10.1177/0278364913495721> (cited on pages 2, 14, 15, 45, and 86)
- Kohl, N., & Stone, P. (2004). Policy gradient reinforcement learning for fast quadrupedal locomotion. In *Proceedings of IEEE International Conference on Robotics and Automation*, 3, 2619–2624. <https://doi.org/10.1109/ROBO T.2004.1307456> (cited on pages 47 and 48)

- Konidaris, G., Kuindersma, S., Grupen, R., & Barto, A. (2011). Autonomous skill acquisition on a mobile manipulator. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 25(1), 1468–1473. (cited on page 46)
- Konidaris, G., Kuindersma, S., Grupen, R., & Barto, A. (2012). Robot learning from demonstration by constructing skill trees. *The International Journal of Robotics Research*, 31(3), 360–375. (cited on page 46)
- Korenkevych, D., Mahmood, A. R., Vasan, G., & Bergstra, J. (2019). Autoregressive policies for continuous control deep reinforcement learning. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, 2754–2762. (cited on pages 47, 49, and 77)
- Kormushev, P., Calinon, S., & Caldwell, D. G. (2010a). Robot motor skill coordination with EM-based reinforcement learning. In *Proceedings of 2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 3232–3237. <https://doi.org/10.1109/IRoS.2010.5649089> (cited on page 47)
- Kormushev, P., Calinon, S., Saegusa, R., & Metta, G. (2010b). Learning the skill of archery by a humanoid robot iCub. In *Proceedings of the 10th IEEE-RAS International Conference on Humanoid Robots*, 417–423. <https://doi.org/10.1109/ICHR.2010.5686841> (cited on page 47)
- Kwok, C., & Fox, D. (2004). Reinforcement learning for sensing strategies. In *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 4, 3158–3163. <https://doi.org/10.1109/IRoS.2004.1389903> (cited on pages 47 and 48)
- Lan, Q., Tosatto, S., Farrahi, H., & Mahmood, A. R. (2022). Model-free policy learning with reward gradients. In *International Conference on Artificial Intelligence and Statistics*. (cited on pages 28, 30, and 162)
- Lan, Q., Vasan, G., & Mahmood, A. R. (2025). Efficient reinforcement learning by

reducing forgetting with elephant activation functions. arXiv. <https://doi.org/10.48550/arXiv.2509.19159> (cited on page 47)

Lange, S., Riedmiller, M., & Voigtländer, A. (2012). Autonomous reinforcement learning on raw visual input data in a real world application. In *The 2012 International Joint Conference on Neural Networks (IJCNN)*, 1–8. (cited on pages 3, 47, and 49)

LaSalle, J. P. (1959). Time optimal control systems. *Proceedings of the National Academy of Sciences*, 45(4), 573–577. (cited on page 76)

Laskin, M., Lee, K., Stooke, A., Pinto, L., Abbeel, P., & Srinivas, A. (2020). Reinforcement learning with augmented data. *Advances in Neural Information Processing Systems*, 33, 19884–19895. (cited on page 64)

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324. (cited on page 17)

LeCun, Y., Bottou, L., Orr, G. B., & Müller, K.-R. (2002). Efficient backprop. In *Neural Networks: Tricks of the Trade*, 9–50. Springer. (cited on page 110)

L’Ecuyer, P. (1990). A unified view of the IPA, SF, and LR gradient estimation techniques. *Management Science*, 36(11), 1364–1383. (cited on page 19)

Lee, M. A., Zhu, Y., Srinivasan, K., Shah, P., Savarese, S., Fei-Fei, L., Garg, A., and Bohg, J. (2019). Making sense of vision and touch: Self-supervised learning of multimodal representations for contact-rich tasks. In *Proceedings of the 2019 International Conference on Robotics and Automation*, 8943–8950. (cited on page 74)

Lesort, T., Lomonaco, V., Stoian, A., Maltoni, D., Filliat, D., & Díaz-Rodríguez, N. (2020). Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges. *Information Fusion*, 58, 52–68. (cited on page 129)

- Levine, S., Finn, C., Darrell, T., & Abbeel, P. (2016). End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(39), 1–40. (cited on page 63)
- Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., . . . , Wierstra, D. (2016). Continuous control with deep reinforcement learning. In *4th International Conference on Learning Representations*. (cited on pages 29, 30, and 114)
- Lin, L. J. (1992). Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8, 293–321. <https://doi.org/10.1007/BF00992699> (cited on pages 37 and 48)
- Luo, J., Hu, Z., Xu, C., Tan, Y. L., Berg, J., Sharma, A., Schaal, S., Finn, C., Gupta, A., & Levine, S. (2024). SERL: A software suite for sample-efficient robotic reinforcement learning. In *Proceedings of IEEE International Conference on Robotics and Automation*, 16961–16969. <https://doi.org/10.1109/ICRA57147.2024.10610040> (cited on page 47)
- Luo, J., Xu, C., Wu, J., & Levine, S. (2025). Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 10(105), eads5033. <https://doi.org/10.1126/scirobotics.ads5033> (cited on pages 5 and 47)
- Lupu, E. S., Spieler, P., Javed, K., De Asis, K., Martin, J. D., Steenstrup, M., & Modayil, J. (2026). The open ant: A robot platform for reinforcement learning research. *Reinforcement Learning Journal* (cited on pages 46 and 47)
- Lyle, C., Zheng, Z., Nikishin, E., Pires, B. A., Pascanu, R., & Dabney, W. (2023). Understanding plasticity in neural networks. In *Proceedings of the 40th International Conference on Machine Learning*, 23190–23211. (cited on pages 110 and 129)
- Lyle, C., Zheng, Z., Khetarpal, K., van Hasselt, H., Pascanu, R., Martens, J., & Dabney, W. (2024). Disentangling the causes of plasticity loss in neural networks. In *Conference on Lifelong Learning Agents*. (cited on page 108)

- Ma, Y. J., Liang, W., Wang, G., Huang, D.-A., Bastani, O., Jayaraman, D., . . . , Anandkumar, A. (2024). Eureka: Human-level reward design via coding large language models. In *the Twelfth International Conference on Learning Representations*. (cited on page 74)
- Mahadevan, S., & Connell, J. (1991). Automatic programming of behavior-based robots using reinforcement learning. In *Proceedings of the 9th National Conference on Artificial Intelligence*, 2, 768–773. AAAI Press / The MIT Press. (cited on pages 3, 44, and 46)
- Mahmood, A. R., Korenkevych, D., Komer, B. J., & Bergstra, J. (2018a). Setting up a reinforcement learning task with a real-world robot. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 4635–4640. (cited on pages 5, 42, 43, 47, 49, 56, and 67)
- Mahmood, A. R., Korenkevych, D., Vasan, G., Ma, W., & Bergstra, J. (2018b). Benchmarking reinforcement learning algorithms on real-world robots. In *Conference on Robot Learning*, 561–591. (cited on pages 5, 47, 49, and 116)
- Martens, J., & Grosse, R. (2015). Optimizing neural networks with kronecker-factored approximate curvature. In *Proceedings of the 32nd International Conference on Machine Learning*, 2408–2417. (cited on page 120)
- Matarić, M. J. (1994). Reward functions for accelerated learning. In *Machine Learning Proceedings 1994*, 181–189. Elsevier. (cited on pages 15, 44, 46, and 74)
- Matarić, M. J. (1997). Reinforcement learning in the multi-robot domain. *Autonomous Robots*, 4(1), 73–83. (cited on pages 44 and 46)
- Mathewson, K. W., & Pilarski, P. M. (2017). Actor-critic reinforcement learning with simultaneous human control and feedback. arXiv. <https://doi.org/10.48550/arXiv.1703.01274> (cited on pages 45 and 46)

- McCloskey, M., & Cohen, N. J. (1989). Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, 24, 109–165. Academic Press. (cited on page 129)
- Miki, T., Lee, J., Hwangbo, J., Wellhausen, L., Koltun, V., & Hutter, M. (2022). Learning robust perceptive locomotion for quadrupedal robots in the wild. *Science Robotics*, 7(62). <https://doi.org/10.1126/SCIROBOTICS.ABK2822>
(cited on page 4)
- Millán, J. D. R. (1996). Rapid, safe, and incremental learning of navigation strategies. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 26(3), 408–420. (cited on pages 45 and 46)
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., . . . , Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://doi.org/10.1038/nature14236> (cited on pages 37 and 114)
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., . . . , Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, 1928–1937. (cited on page 23)
- Modayil, J., White, A., & Sutton, R. S. (2014). Multi-timescale nexting in a reinforcement learning robot. *Adaptive Behavior*, 22(2), 146–160. (cited on pages 44 and 46)
- Mohamed, S., Rosca, M., Figurnov, M., & Mnih, A. (2020). Monte Carlo gradient estimation in machine learning. *Journal of Machine Learning Research*, 21(132), 1–62. (cited on page 27)
- Moore, G. E. (1965). Cramming more components onto integrated circuits. *Electronics*, 38(8). (cited on pages 50 and 54)

- Moore, A. (1991). *Efficient Memory-Based Learning for Robot Control*. Doctoral dissertation, University of Cambridge. (cited on pages 76 and 83)
- Moore, A. W., & Atkeson, C. G. (1993). Prioritized sweeping: Reinforcement learning with less data and less time. *Machine Learning*, 13(1), 103–130. (cited on page 48)
- Morimoto, J., & Doya, K. (2001). Acquisition of stand-up behavior by a real robot using hierarchical reinforcement learning. *Robotics and Autonomous Systems*, 36(1), 37–51. (cited on pages 44 and 46)
- Morimoto, J., Nakanishi, J., Endo, G., Cheng, G., Atkeson, C. G., & Zeglin, G. (2005). Poincaré-map-based reinforcement learning for biped walking. In *Proceedings of IEEE International Conference on Robotics and Automation*, 2381–2386. <https://doi.org/10.1109/ROBOT.2005.1570469> (cited on page 47)
- Munos, R., Stepleton, T., Harutyunyan, A., & Bellemare, M. (2016). Safe and efficient off-policy reinforcement learning. *Advances in neural information processing systems*, 29. (cited on page 122)
- Muslimani, C., Johnstonbaugh, K., Chandramouli, S., Booth, S., Knox, W. B., & Taylor, M. E. (2025). Towards improving reward design in rl: A reward alignment metric for rl practitioners. *Reinforcement Learning Journal* (cited on page 74)
- Nair, A., Srinivasan, P., Blackwell, S., Alcicek, C., Fearon, R., Maria, A. D., . . . , & Silver, D. (2015). Massively parallel methods for deep reinforcement learning. arXiv. <https://doi.org/10.48550/arXiv.1507.04296> (cited on pages 54 and 65)
- Nair, A. V., Pong, V., Dalal, M., Bahl, S., Lin, S., and Levine, S. (2018). Visual reinforcement learning with imagined goals. *Advances in neural information processing systems*, 31. (cited on page 75)

- Nair, A., Gupta, A., Dalal, M., & Levine, S. (2020). AWAC: Accelerating online reinforcement learning with offline datasets. arXiv. <https://doi.org/10.48550/arXiv.2006.09359> (cited on page 47)
- Narvekar, S., Peng, B., Leonetti, M., Sinapov, J., Taylor, M. E., & Stone, P. (2020). Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, 21(181), 1-50. (cited on page 93)
- Nauman, M., Bortkiewicz, M., Miłoś, P., Trzciński, T., Ostaszewski, M., & Cygan, M. (2024). Overestimation, overfitting, and plasticity in actor-critic: The bitter lesson of reinforcement learning. In *Proceedings of the 41st International Conference on Machine Learning*. (cited on page 106)
- Nemec, B., Zorko, M., & Žlajpah, L. (2010). Learning of a ball-in-a-cup playing robot. In *19th International Workshop on Robotics in Alpe-Adria-Danube Region (RAAD 2010)*, 297–301. (cited on pages 44 and 46)
- Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, 278–287. (cited on page 74)
- Nikishin, E., Schwarzer, M., D’Oro, P., Bacon, P.-L., & Courville, A. (2022). The primacy bias in deep reinforcement learning. In *Proceedings of the 39th International Conference on Machine Learning*, 16828–16847. (cited on page 108)
- Nota, C., & Thomas, P. S. (2020). Is the policy gradient a gradient? In *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems*, 939–947. (cited on pages 20 and 22)
- Paletta, L., Fritz, G., Kintzler, F., Irran, J., & Dorffner, G. (2007). Perception and developmental learning of affordances in autonomous robots. In *Annual Conference on Artificial Intelligence*, 235–250. Springer. (cited on page 46)

- Parmas, P., & Sugiyama, M. (2021). A unified view of likelihood ratio and reparameterization gradients. In *International Conference on Artificial Intelligence and Statistics*, 4078–4086. (cited on page 27)
- Pastor, P., Kalakrishnan, M., Chitta, S., Theodorou, E. A., & Schaal, S. (2011). Skill learning and task outcome prediction for manipulation. In *Proceedings of IEEE International Conference on Robotics and Automation*, 3828–3834. <https://doi.org/10.1109/ICRA.2011.5980200> (cited on page 47)
- Penicka, R., Song, Y., Kaufmann, E., & Scaramuzza, D. (2022). Learning minimum-time flight in cluttered environments. *IEEE Robotics and Automation Letters*, 7(3), 7209–7216. (cited on page 76)
- Peters, J., & Schaal, S. (2008). Natural actor-critic. *Neurocomputing*, 71(7–9), 1180–1190. (cited on pages 47, 48, and 120)
- Piater, J., Jodogne, S., Detry, R., Kraft, D., Krüger, N., Kroemer, O., & Peters, J. (2011). Learning visual representations for perception-action systems. *The International Journal of Robotics Research*, 30(3), 294–307. (cited on page 46)
- Pilarski, P. M., Dawson, M. R., Degris, T., Fahimi, F., Carey, J. P., & Sutton, R. S. (2011). Online human training of a myoelectric prosthesis controller via actor-critic reinforcement learning. In *2011 IEEE International Conference on Rehabilitation Robotics*, 1–7. (cited on pages 22, 45, and 46)
- Pilarski, P. M., Dick, T. B., & Sutton, R. S. (2013). Real-time prediction learning for the simultaneous actuation of multiple prosthetic joints. In *2013 IEEE 13th International Conference on Rehabilitation Robotics (ICORR)*, 1–8. (cited on pages 45 and 46)
- Platt, R., Grunert, R. A., & Fagg, A. H. (2006). Improving grasp skills using schema structured learning. In *5th International Conference on Development and Learning*. (cited on pages 44 and 46)

- Powell, W., Ruszczyński, A., & Topaloglu, H. (2004). Learning algorithms for separable approximations of discrete stochastic optimization problems. *Mathematics of Operations Research*, 29(4), 814-836. (cited on page 29)
- Powers, S. (2023). *Continual Robot Learning: Benchmarks and Modular Methods*. Doctoral dissertation, Carnegie Mellon University. <https://doi.org/10.1184/R1/24438466.V1> (cited on page 129)
- Prokhorov, D. V., & Wunsch, D. C. (1997). Adaptive critic designs. *IEEE transactions on Neural Networks*, 8(5), 997-1007. (cited on page 29)
- Ramstedt, S., & Pal, C. (2019). Real-time reinforcement learning. *Advances in Neural Information Processing Systems*, 32. (cited on page 42)
- Rao, N., Aljalbout, E., Sauer, A., & Haddadin, S. (2020). How to make deep RL work in practice. arXiv. <https://doi.org/10.48550/arXiv.2010.13083> (cited on page 110)
- Rezende, D. J., Mohamed, S., & Wierstra, D. (2014). Stochastic backpropagation and approximate inference in deep generative models. In *Proceedings of the 31st International Conference on Machine Learning*, 1278–1286. (cited on page 27)
- Riedmiller, M. (2005a). Neural fitted Q iteration – First experiences with a data efficient neural reinforcement learning method. In *Machine Learning: ECML 2005. Lecture Notes in Computer Science*(3720), 317–328. https://doi.org/10.1007/11564096_32 (cited on pages 37 and 38)
- Riedmiller, M. (2005b). Neural reinforcement learning to swing-up and balance a real pole. In *IEEE International Conference on Systems, Man and Cybernetics*, 4, 3191–3196. (cited on pages 3, 47, and 49)
- Riedmiller, M., Montemerlo, M., & Dahlkamp, H. (2007). Learning to drive a real car in 20 minutes. In *2007 Frontiers in the Convergence of Bioscience and Information Technologies*, 645–650. (cited on pages 47 and 49)

- Riedmiller, M., Gabel, T., Hafner, R., & Lange, S. (2009). Reinforcement learning for robot soccer. *Autonomous Robots*, 27(1), 55–73. (cited on pages 47 and 49)
- Riedmiller, M. (2012). 10 steps and some tricks to set up neural reinforcement controllers. In *Neural Networks: Tricks of the Trade*, 735–757. Springer. https://doi.org/10.1007/978-3-642-35289-8_39 (cited on page 37)
- Riedmiller, M. A., Hafner, R., Lampe, T., Neunert, M., Degraeve, J., Van de Wiele, T., . . . , Springenberg, J. T. (2018). Learning by playing: Solving sparse reward tasks from scratch. In *Proceedings of the 35th International Conference on Machine Learning*, 4341–4350. (cited on pages 16, 47, 49, 73, and 75)
- Riedmiller, M., Springenberg, J. T., Hafner, R., & Heess, N. (2022). Collect & infer: A fresh look at data-efficient reinforcement learning. In *Conference on robot learning*, 1736–1744. (cited on page 4)
- Robbins, H., & Monro, S. (1951). A stochastic approximation method. *Annals of Mathematical Statistics*, 22(3), 400–407. <https://doi.org/10.1214/aoms/1177729586> (cited on page 18)
- Rocamonde, J., Montesinos, V., Nava, E., Perez, E., & Lindner, D. (2024). Vision-language models are zero-shot reward models for reinforcement learning. In *Twelfth International Conference on Learning Representations*. (cited on page 74)
- Rubinstein, R. Y. (1986). The score function approach for sensitivity analysis of computer simulation models. *Mathematics and Computers in Simulation*, 28, 351–379. (cited on page 19)
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536. (cited on page 17)
- Russell, S. J., & Wefald, E. (1991). *Do the right thing: Studies in limited rationality*. MIT press. (cited on page 71)

- Saxe, A. M., McClelland, J. L., & Ganguli, S. (2013). Exact solutions to the non-linear dynamics of learning in deep linear neural networks. arXiv. <https://doi.org/10.48550/arXiv.1312.6120> (cited on page 97)
- Schaal, S. (1996). Learning from demonstration. *Advances in Neural Information Processing Systems*, 9. (cited on page 47)
- Schaul, T., Ostrovski, G., Kemaev, I., & Borsa, D. (2021). Return-based scaling: Yet another normalisation trick for deep RL. arXiv. <https://doi.org/10.48550/arXiv.2105.05347> (cited on pages 100, 110, and 111)
- Schraudolph, N. N. (2002). Centering neural network gradient factors. In *Neural Networks: Tricks of the Trade*, 207–226. Springer. (cited on page 110)
- Schulman, J., Levine, S., Abbeel, P., Jordan, M. I., & Moritz, P. (2015). Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning*, 1889–1897. (cited on pages 49 and 120)
- Schulman, J., Moritz, P., Levine, S., Jordan, M. I., & Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation. In *4th International Conference on Learning Representations*. (cited on page 24)
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv. <https://doi.org/10.48550/arXiv.1707.06347> (cited on pages 20, 23, 24, 25, 56, and 95)
- Schwab, D., Springenberg, J. T., Martins, M. F., Neunert, M., Lampe, T., Abdolmaleki, A., . . . , Riedmiller, M. A. (2019). Simultaneously learning vision and feature-based control policies for real-world ball-in-a-cup. In *Robotics: Science and Systems XV*. <https://doi.org/10.15607/RSS.2019.XV.027> (cited on pages 47 and 49)
- Shahriar, F., Wang, C., Azimi, A., Vasan, G., Elanwar, H. H., Naik, A., Mahmood, A. R., & Bellinger, C. (2026). Dynamic object masks as goal representations for

visual goal-conditioned reinforcement learning. *Reinforcement Learning Journal*. (cited on page 47)

Sherstan, C., Modayil, J., & Pilarski, P. M. (2015). A collaborative approach to the simultaneous multi-joint control of a prosthetic arm. In *2015 IEEE International Conference on Rehabilitation Robotics (ICORR)*, 13–18. (cited on pages 44 and 46)

Shin, K., & McKay, N. (1985). Minimum-time control of robotic manipulators with geometric path constraints. *IEEE Transactions on Automatic Control*, 30(6), 531-541. (cited on page 76)

Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014). Deterministic policy gradient algorithms. In *Proceedings of the 31st International Conference on Machine Learning*, 387–395. (cited on pages 28 and 30)

Singh, S. P., & Sutton, R. S. (1996). Reinforcement learning with replacing eligibility traces. *Machine Learning*, 22(1), 123–158. (cited on page 121)

Smart, W. D., & Kaelbling, L. P. (2002). Effective reinforcement learning for mobile robots. In *Proceedings of IEEE International Conference on Robotics and Automation*, 3404–3410. (cited on page 47)

Smith, L., Kew, J. C., Peng, X. B., Ha, S., Tan, J., & Levine, S. (2022). Legged robots that keep on learning: Fine-tuning locomotion policies in the real world. In *Proceedings of the 2022 International Conference on Robotics and Automation*, 1593–1599. <https://doi.org/10.1109/ICRA46639.2022.9812166>
(cited on pages 15, 47, and 49)

Smith, L., Kostrikov, I., & Levine, S. (2023). Demonstrating a walk in the park: Learning to walk in 20 minutes with model-free reinforcement learning. *Robotics: Science and Systems (RSS) Demo*, 2(3), 4. (cited on pages 4, 47, and 49)

Smith, L., Cao, Y., & Levine, S. (2024). Grow your limits: Continuous improvement with real-world RL for robotic locomotion. In *Proceedings of IEEE In-*

- ternational Conference on Robotics and Automation*, 10829–10836. <https://doi.org/10.1109/ICRA57147.2024.10610485> (cited on pages 47 and 49)
- Sokar, G., Agarwal, R., Castro, P. S., & Evci, U. (2023). The dormant neuron phenomenon in deep reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning*, 32145–32168. (cited on page 108)
- Soni, V., & Singh, S. (2006). Reinforcement learning of hierarchical skills on the Sony Aibo robot. In *Proceedings of the Fifth International Conference on Development and Learning*. (cited on page 46)
- Stulp, F., & Sigaud, O. (2012). Path integral policy improvement with covariance matrix adaptation. In *Proceedings of the 29th International Conference on Machine Learning*. (cited on pages 47 and 49)
- Sutton, R. S. (1988a). Learning to predict by the methods of temporal differences. *Machine learning*, 3(1), 9-44. (cited on page 17)
- Sutton, R. S. (1988b). Nadaline: A normalized adaptive linear element that learns efficiently. *Technical Report GTE TR88-509.4*. GTE Laboratories Incorporated. (cited on page 110)
- Sutton, R. S. (1991). Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4), 160–163. (cited on pages 48, 122, and 128)
- Sutton, R. S., McAllester, D. A., Singh, S., & Mansour, Y. (1999). Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, 12, 1057–1063. (cited on page 20)
- Sutton, R. S., Modayil, J., Delp, M., Degris, T., Pilarski, P. M., White, A., & Precup, D. (2011). Horde: A scalable real-time architecture for learning knowledge from unsupervised sensorimotor interaction. In *10th International Conference on Autonomous Agents and Multiagent Systems*, 1, 761–768. (cited on pages 44 and 46)

- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. (cited on pages 2, 11, 17, 20, 21, 22, 73, 120, and 122)
- Tamei, T., & Shibata, T. (2008). Policy gradient learning of cooperative interaction with a robot using user's biological signals. In *International Conference on Neural Information Processing*, 1029–1037. (cited on page 46)
- Tamosiunaite, M., Nemec, B., Ude, A., & Wörgötter, F. (2011). Learning to pour with a robot arm combining goal and shape learning for dynamic movement primitives. *Robotics and Autonomous Systems*, 59, 910–922. (cited on pages 47 and 49)
- Tassa, Y., Doron, Y., Muldal, A., Erez, T., Li, Y., Casas, D. D. L., . . . , & Riedmiller, M. (2018). DeepMind Control Suite. arXiv. <https://doi.org/10.48550/arXiv.1801.00690> (cited on pages 77, 78, 83, 84, and 106)
- Tedrake, R., Zhang, T. W., & Seung, H. S. (2004). Stochastic policy gradient reinforcement learning on a simple 3D biped. In *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3, 2849–2854. <https://doi.org/10.1109/IROS.2004.1389841> (cited on pages 44 and 46)
- Tedrake, R., Zhang, T. W., & Seung, H. S. (2005). Learning to walk in 20 minutes. In *Proceedings of the Fourteenth Yale Workshop on Adaptive and Learning Systems*. (cited on pages 44 and 46)
- Theodorou, E. A., Buchli, J., & Schaal, S. (2010). Reinforcement learning of motor skills in high dimensions: A path integral approach. In *Proceedings of IEEE International Conference on Robotics and Automation*. (cited on pages 47 and 49)
- Thrun, S. (1995). An approach to learning mobile robot navigation. *Robotics and Autonomous Systems*, 15, 301–319. (cited on pages 45 and 46)
- Thrun, S., & Mitchell, T. M. (1995). Lifelong robot learning. *Robotics and autonomous systems*, 15(1-2), 25-46. (cited on page 129)

- Thrun, S., & Pratt, L. (1998). Learning to learn: Introduction and overview. In *Learning to Learn*, 3–17. Kluwer Academic Publishers. (cited on page 127)
- Todorov, E., Erez, T., & Tassa, Y. (2012). MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 5026–5033. (cited on pages 41 and 95)
- Tokic, M., Ertel, W., & Fessler, J. (2009). The Crawler, a class room demonstrator for reinforcement learning. In *Proceedings of the Twenty-Second International FLAIRS Conference*. (cited on pages 44 and 46)
- Torne, M., Simeonov, A., Li, Z., Chan, A., Chen, T., Gupta, A., & Agrawal, P. (2024). Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation. In *Robotics: Science and Systems* (cited on page 5)
- Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., . . . , et al. (2024). Gymnasium: A standard interface for reinforcement learning environments. arXiv. <https://doi.org/10.48550/arXiv.2407.17032> (cited on pages 77, 83, and 106)
- Touzet, C. F. (1997). Neural reinforcement learning for behaviour synthesis. *Robotics and Autonomous Systems*, 22(3–4), 251–281. (cited on pages 45 and 46)
- Travnik, J. B., Mathewson, K. W., Sutton, R. S., & Pilarski, P. M. (2018). Reactive reinforcement learning in asynchronous environments. *Frontiers in Robotics and AI*, 5, 79. (cited on pages 42 and 46)
- Travnik, J. (2018). *Reinforcement Learning on Resource Bounded Systems*. MSc thesis, University of Alberta. (cited on page 46)
- van Hasselt, H. (2010). Double Q-learning. *Advances in Neural Information Processing Systems*, 23, 2613–2621. (cited on page 30)
- van Hasselt, H., Madjiheurem, S., Hessel, M., Silver, D., Barreto, A., & Borsa, D.

- (2021). Expected eligibility traces. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 35, 9997–10005. (cited on page 121)
- Vasan, G. (2017). *Teaching a Powered Prosthetic Arm with an Intact Arm Using Reinforcement Learning*. MSc thesis, University of Alberta. (cited on pages 45 and 46)
- Vasan, G., & Pilarski, P. M. (2017). Learning from demonstration: Teaching a myoelectric prosthesis with an intact limb via reinforcement learning. In *2017 International Conference on Rehabilitation Robotics (ICORR)*, 1457–1464. <https://doi.org/10.1109/ICORR.2017.8009453> (cited on pages 22, 45, 46, and 105)
- Vasan, G., & Pilarski, P. M. (2018). Context-aware learning from demonstration: Using camera data to support the synergistic control of a multi-joint prosthetic arm. In *2018 7th IEEE International Conference on Biomedical Robotics and Biomechatronics (Biorob)*, 199–206. (cited on pages 45, 46, and 105)
- Vasan, G., Wang, Y., Shahriar, F., Bergstra, J., Jägersand, M., & Mahmood, A. R. (2024a). Revisiting sparse rewards for goal-reaching reinforcement learning. *Reinforcement Learning Journal*, 4, 1841–1854. (cited on pages 47 and 50)
- Vasan, G., Elsayed, M., Azimi, S. A., He, J., Shahriar, F., Bellinger, C., White, M., & Mahmood, A. R. (2024b). Deep policy gradient methods without batch updates, target networks, or replay buffers. *Advances in Neural Information Processing Systems*, 37. (cited on pages 45, 46, and 102)
- Vecerik, M., Hester, T., Scholz, J., Wang, F., Pietquin, O., Piot, B., . . . , & Riedmiller, M. (2017). Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards. arXiv. <https://doi.org/10.48550/arXiv.1707.08817> (cited on page 127)
- Verma, V., Maimone, M. W., Gaines, D. M., Francis, R., Estlin, T. A., Kuhn, S. R., Rabideau, G. R., Chien, S. A., McHenry, M. M., Graser, E. J., et al. (2023).

- Autonomous robotics is driving Perseverance rover’s progress on Mars. *Science Robotics*, 8(80), eadi3099. (cited on page 55)
- Vlassis, N., Toussaint, M., Kontes, G., & Piperidis, S. (2009). Learning model-free robot control by a Monte Carlo EM algorithm. *Autonomous Robots*, 27(2), 123–130. (cited on pages 47 and 48)
- Wagenmaker, A., Huang, K., Ke, L., Jamieson, K., & Gupta, A. (2024). Overcoming the sim-to-real gap: Leveraging simulation to learn to explore for real-world RL. *Advances in Neural Information Processing Systems*, 37. (cited on page 126)
- Walsh, T. J., Nouri, A., Li, L., & Littman, M. L. (2009). Learning and planning in environments with delayed feedback. *Autonomous Agents and Multi-Agent Systems*, 18(1), 83–105. (cited on page 42)
- Wan, Y., Naik, A., & Sutton, R. S. (2021). Learning and planning in average-reward markov decision processes. In *Proceedings of the 40th International Conference on Machine Learning*, 10653–10662. (cited on page 122)
- Wang, Y., Vasan, G., & Mahmood, A. R. (2023). Real-time reinforcement learning for vision-based robotics utilizing local and remote computers. In *Proceedings of IEEE International Conference on Robotics and Automation*, 9435–9441. (cited on pages 47, 50, and 86)
- Wang, Y., Sun, Z., Zhang, J., Xian, Z., Biyik, E., Held, D., & Erickson, Z. (2024). RL-VLM-F: Reinforcement learning from vision language foundation model feedback. In *Proceedings of the 41st International Conference on Machine Learning*. (cited on page 74)
- Watkins, C. J. C. H. (1989). *Learning from Delayed Rewards*. Doctoral dissertation, King’s College, Cambridge. (cited on page 15)
- Welford, B. (1962). Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3), 419–420. (cited on pages 100 and 103)

- Werbos, P. J. (1990). A menu of designs for reinforcement learning over time. In *Neural Networks for Control*, 67–95. MIT Press. (cited on page 29)
- Willgoss, R. A., & Iqbal, J. (1999). Reinforcement learning of behaviors in mobile robots using noisy infrared sensing. In *Australian Conference on Robotics and Automation*. (cited on pages 44 and 46)
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8, 229–256. <https://doi.org/10.1007/BF00992696> (cited on pages 20 and 21)
- Williams, G., Wagener, N., Goldfain, B., Drews, P., Rehg, J. M., Boots, B., & Theodorou, E. A. (2017). Information theoretic MPC for model-based reinforcement learning. In *Proceedings of IEEE International Conference on Robotics and Automation*, 1714–1721. (cited on page 128)
- Wu, Y., Mozifian, M., & Shkurti, F. (2021). Shaping rewards for reinforcement learning with imperfect demonstrations using generative models. In *Proceedings of IEEE International Conference on Robotics and Automation*, 6628–6634. (cited on page 74)
- Wu, P., Escontrela, A., Hafner, D., Abbeel, P., & Goldberg, K. (2023). Day-Dreamer: World models for physical robot learning. In *Conference on Robot Learning*, 2226–2240. (cited on pages 47, 50, and 75)
- Yahya, A., Li, A., Kalakrishnan, M., Chebotar, Y., & Levine, S. (2017). Collective robot reinforcement learning with distributed asynchronous guided policy search. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 79–86. (cited on pages 47 and 49)
- Yamaguchi, T., Masubuchi, M., Fujihara, K., & Yachida, M. (1996). Realtime reinforcement learning for a real robot in the real environment. In *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems*,

3, 1321–1328. <https://doi.org/10.1109/IROS.1996.568988> (cited on page 46)

Yang, Y., Caluwaerts, K., Iscen, A., Zhang, T., Tan, J., & Sindhvani, V. (2019). Data efficient reinforcement learning for legged robots. In *3rd Annual Conference on Robot Learning*, 1–10. (cited on pages 47 and 49)

Yasuda, T., & Ohkura, K. (2008). A reinforcement learning technique with an adaptive action generator for a multi-robot system. In *International Conference on Simulation of Adaptive Behavior*, 250–259. (cited on page 46)

Yuan, Y., & Mahmood, A. R. (2022). Asynchronous reinforcement learning for real-time control of physical robots. In *Proceedings of IEEE International Conference on Robotics and Automation*, 5546–5552. (cited on pages 4, 47, 50, 61, 85, 86, and 91)

Zhang, B., & Sennrich, R. (2019). Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32. (cited on page 172)

Ziebart, B. D., Bagnell, J. A., & Dey, A. K. (2010). Modeling interaction via the principle of maximum causal entropy. In *Proceedings of the 27th International Conference on Machine Learning*. (cited on page 32)

Appendix A

A.1 Theoretical Foundations

Lan et al. provide detailed proofs of the reparameterization policy gradient theorem (Lan et al. 2022). This appendix records a proof sketch of that result, stated as Theorem 3, and derives the entropy-regularized extension used by AVG. The notation follows Sections 2.4 through 2.6: $J(\theta)$ denotes the start-state objective, $d_{\pi_{\theta}, \gamma}$ the normalized discounted state-visitation distribution, g_{θ} the reparameterization map, and h the density of the noise variable Z . The derivations assume that the relevant functions are differentiable, that differentiation and integration may be interchanged, and that the support of the policy does not depend on the policy parameters.

Proof Sketch of the Reparameterization Policy Gradient Theorem

Proof sketch of Theorem 3.

$$\begin{aligned}
\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) &= \nabla_{\boldsymbol{\theta}} v_{\pi_{\boldsymbol{\theta}}}(s_0) \\
&\propto \int d_{\pi_{\boldsymbol{\theta}}, \gamma}(s) \nabla_{\boldsymbol{\theta}} \int \pi_{\boldsymbol{\theta}}(a \mid s) q_{\pi_{\boldsymbol{\theta}}}(s, a) da ds \\
&= \int d_{\pi_{\boldsymbol{\theta}}, \gamma}(s) \nabla_{\boldsymbol{\theta}} \int h(z) q_{\pi_{\boldsymbol{\theta}}}(s, g_{\boldsymbol{\theta}}(z; s)) dz ds \\
&= \int d_{\pi_{\boldsymbol{\theta}}, \gamma}(s) \int h(z) \nabla_{\boldsymbol{\theta}} g_{\boldsymbol{\theta}}(z; s) \nabla_a q_{\pi_{\boldsymbol{\theta}}}(s, a)|_{a=g_{\boldsymbol{\theta}}(z; s)} dz ds \\
&= \int d_{\pi_{\boldsymbol{\theta}}, \gamma}(s) \int \pi_{\boldsymbol{\theta}}(a \mid s) \nabla_{\boldsymbol{\theta}} g_{\boldsymbol{\theta}}(z; s)|_{z=g_{\boldsymbol{\theta}}^{-1}(a; s)} \nabla_a q_{\pi_{\boldsymbol{\theta}}}(s, a) da ds \\
&= \mathbb{E}_{S \sim d_{\pi_{\boldsymbol{\theta}}, \gamma}, A \sim \pi_{\boldsymbol{\theta}}} \left[\nabla_{\boldsymbol{\theta}} g_{\boldsymbol{\theta}}(z; S)|_{z=g_{\boldsymbol{\theta}}^{-1}(A; S)} \nabla_a q_{\pi_{\boldsymbol{\theta}}}(S, a)|_{a=A} \right].
\end{aligned}$$

The second line follows from the policy gradient theorem (Theorem 1), the third from reparameterization, the fourth from the chain rule, and the fifth from back substitution. The gradient in the second line acts on the policy only; the dependence of $q_{\pi_{\boldsymbol{\theta}}}$ on $\boldsymbol{\theta}$ is absorbed by $d_{\pi_{\boldsymbol{\theta}}, \gamma}$. $\square$

Action Value Gradient Theorem

Theorem 4 (Action Value Gradient Theorem). *Under the conditions of Theorem 3, for the entropy-regularized objective $J(\boldsymbol{\theta}) \doteq v_{\pi_{\boldsymbol{\theta}}}^{\text{Ent}}(s_0)$, the action value gradient is*

$$\begin{aligned} \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) &\propto \mathbb{E}_{S \sim d_{\pi_{\boldsymbol{\theta}}, \gamma}, A \sim \pi_{\boldsymbol{\theta}}} \left[\nabla_{\boldsymbol{\theta}} g_{\boldsymbol{\theta}}(z; S) \Big|_{z=g_{\boldsymbol{\theta}}^{-1}(A; S)} \right. \\ &\quad \left. \nabla_a (q_{\pi_{\boldsymbol{\theta}}}^{\text{Ent}}(S, a) - \eta \log \pi_{\boldsymbol{\theta}}(a \mid S)) \Big|_{a=A} \right]. \end{aligned}$$

Proof.

$$\begin{aligned} \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) &= \nabla_{\boldsymbol{\theta}} v_{\pi_{\boldsymbol{\theta}}}^{\text{Ent}}(s_0) \\ &\propto \int d_{\pi_{\boldsymbol{\theta}}, \gamma}(s) \nabla_{\boldsymbol{\theta}} \int \pi_{\boldsymbol{\theta}}(a \mid s) (q_{\pi_{\boldsymbol{\theta}}}^{\text{Ent}}(s, a) - \eta \log \pi_{\boldsymbol{\theta}}(a \mid s)) da ds \\ &= \int d_{\pi_{\boldsymbol{\theta}}, \gamma}(s) \nabla_{\boldsymbol{\theta}} \int h(z) (q_{\pi_{\boldsymbol{\theta}}}^{\text{Ent}}(s, g_{\boldsymbol{\theta}}(z; s)) - \eta \log \pi_{\boldsymbol{\theta}}(g_{\boldsymbol{\theta}}(z; s) \mid s)) dz ds \\ &= \int d_{\pi_{\boldsymbol{\theta}}, \gamma}(s) \int h(z) \nabla_{\boldsymbol{\theta}} g_{\boldsymbol{\theta}}(z; s) \\ &\quad \nabla_a (q_{\pi_{\boldsymbol{\theta}}}^{\text{Ent}}(s, a) - \eta \log \pi_{\boldsymbol{\theta}}(a \mid s)) \Big|_{a=g_{\boldsymbol{\theta}}(z; s)} dz ds \\ &= \int d_{\pi_{\boldsymbol{\theta}}, \gamma}(s) \int \pi_{\boldsymbol{\theta}}(a \mid s) \nabla_{\boldsymbol{\theta}} g_{\boldsymbol{\theta}}(z; s) \Big|_{z=g_{\boldsymbol{\theta}}^{-1}(a; s)} \\ &\quad \nabla_a (q_{\pi_{\boldsymbol{\theta}}}^{\text{Ent}}(s, a) - \eta \log \pi_{\boldsymbol{\theta}}(a \mid s)) da ds \\ &= \mathbb{E}_{S \sim d_{\pi_{\boldsymbol{\theta}}, \gamma}, A \sim \pi_{\boldsymbol{\theta}}} \left[\nabla_{\boldsymbol{\theta}} g_{\boldsymbol{\theta}}(z; S) \Big|_{z=g_{\boldsymbol{\theta}}^{-1}(A; S)} \right. \\ &\quad \left. \nabla_a (q_{\pi_{\boldsymbol{\theta}}}^{\text{Ent}}(S, a) - \eta \log \pi_{\boldsymbol{\theta}}(a \mid S)) \Big|_{a=A} \right]. \end{aligned}$$

The second line follows from the policy gradient theorem (Theorem 1), the third from reparameterization, the fourth from the chain rule, and the fifth from back substitution. The step from the third to the fourth line differentiates $\log \pi_{\boldsymbol{\theta}}$ only through its action argument; the term arising from its direct dependence on $\boldsymbol{\theta}$ is $-\eta \mathbb{E}_{A \sim \pi_{\boldsymbol{\theta}}} [\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(A \mid s)]$, which vanishes because the score function has zero mean. The soft value satisfies $v_{\pi_{\boldsymbol{\theta}}}^{\text{Ent}}(s) = \mathbb{E}_{A \sim \pi_{\boldsymbol{\theta}}(\cdot \mid s)} [q_{\pi_{\boldsymbol{\theta}}}^{\text{Ent}}(s, A) - \eta \log \pi_{\boldsymbol{\theta}}(A \mid s)]$ because $\mathcal{H}(\pi_{\boldsymbol{\theta}}(\cdot \mid s)) = -\mathbb{E}_{A \sim \pi_{\boldsymbol{\theta}}(\cdot \mid s)} [\log \pi_{\boldsymbol{\theta}}(A \mid s)]$. $\square$

A.2 Estimation of FLOPs per Learning Update

I assign each learning update a computational cost in floating-point operations (FLOPs) using the approximation

$$\text{FLOPs per update} \doteq \begin{cases} 6 P_{\text{eff}} n & \text{for neural networks,} \\ 2 P_{\text{eff}} n & \text{for linear function approximation,} \end{cases}$$

where P_{eff} is the effective parameter count of the function approximator and n is the number of samples processed per update. The factor of six follows the standard accounting of two FLOPs per parameter per sample for the forward pass and four for the backward pass. The factor of two counts one multiplication and one addition per active weight for the prediction. The gradient of a linear prediction with respect to its weights is the feature vector itself, and computing it therefore requires no propagation comparable to a neural network’s backward pass.

The effective parameter count depends on the representational regime. For neural networks, P_{eff} equals the total number of trainable parameters, computed from the layer dimensions reported in each algorithm’s source publication or official implementation. For linear function approximation with sparse features, P_{eff} equals the number of active features multiplied by the number of weight vectors maintained by the algorithm, because inactive features contribute no operations.

The number of samples per update depends on the update regime. For batch methods, n equals the minibatch size multiplied by the number of gradient steps per update. Streaming methods, which update from the most recent transition alone, have $n = 1$. When a paper reports multiple experiments, the estimate uses one representative configuration, typically the primary physical-robot experiment.

A.3 SAC Hyperparameters for Experiments with Minimum-Time Tasks

Hyperparameter	Value
Replay buffer size	100,000
Actor step size	3×10^{-4}
Critic step size	3×10^{-4}
Entropy coefficient step size (α^κ)	3×10^{-4}
Minibatch size	256
Discount factor (γ)	0.99
Update interval (timesteps)	2
Gradient updates per interval	1
Actor MLP hidden sizes	(512, 512)
Critic MLP hidden sizes	(512, 512)
Warm-up period (timesteps)	1,000
Adam optimizer betas	(0.9, 0.999)
Initial entropy coefficient (η)	0.1
Neural network activation	ReLU

Table A.1: SAC hyperparameters for the experiments with minimum-time tasks.

In most configurations, agents learned effectively without hyperparameter tuning, which indicates the reliability of the algorithm, the learning architecture, and the task setup. For the experiments in Section 5.4, the warm-up period was 20,000 timesteps.

A.4 AVG with Target Q Networks

Algorithm 7 extends AVG with a target critic $\bar{w}$ that tracks the critic through Polyak averaging with coefficient τ , as in SAC (Algorithm 3). The remaining components, including the `Normalize` and `NormalizeTDError` procedures, are unchanged from Chapter 6.

Algorithm 7 AVG with Target Q Networks

```
1: Initialize  $\gamma, \eta, \tau, \alpha^\theta, \alpha^\mathbf{w}$ , and  $\varepsilon > 0$ ; actor parameters  $\theta$  and critic parameters  $\mathbf{w}$  with orthogonal initialization
2: Use penultimate normalization in the actor and critic
3:  $n \leftarrow 0, \quad \mu_S \leftarrow \mathbf{0}, \quad \mathbf{M}_S \leftarrow \mathbf{0}$ 
4:  $\mathbf{n}_\delta \leftarrow [0, 0, 0], \quad \mu_\delta \leftarrow [0, 0, 0], \quad \mathbf{M}_\delta \leftarrow [0, 0, 0]$ 
5:  $\bar{\mathbf{w}} \leftarrow \mathbf{w}$ 
6: for each episode do
7:   Initialize  $S$  as the first state of the episode
8:    $S, n, \mu_S, \mathbf{M}_S, - \leftarrow \text{Normalize}(S, n, \mu_S, \mathbf{M}_S, \varepsilon)$ 
9:    $\tilde{G} \leftarrow 0$ 
10:  while  $S$  is not terminal do
11:     $A_\theta \leftarrow g_\theta(Z; S)$  where  $Z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
12:    Take action  $A_\theta$  and observe  $R$  and  $S'$ 
13:     $S', n, \mu_S, \mathbf{M}_S, - \leftarrow \text{Normalize}(S', n, \mu_S, \mathbf{M}_S, \varepsilon)$ 
14:     $\tilde{R} \leftarrow R - \eta \log \pi_\theta(A_\theta | S)$ 
15:     $\tilde{G} \leftarrow \tilde{G} + \tilde{R}$ 
16:    if  $S'$  is terminal then
17:       $\sigma_\delta, \mathbf{n}_\delta, \mu_\delta, \mathbf{M}_\delta \leftarrow \text{NormalizeTDError}(\tilde{R}, 0, \tilde{G}, \mathbf{n}_\delta, \mu_\delta, \mathbf{M}_\delta, \varepsilon)$ 
18:       $\delta \leftarrow R - \hat{q}(S, \text{sg}(A_\theta); \mathbf{w})$ 
19:    else
20:       $\sigma_\delta, \mathbf{n}_\delta, \mu_\delta, \mathbf{M}_\delta \leftarrow \text{NormalizeTDError}(\tilde{R}, \gamma, \emptyset, \mathbf{n}_\delta, \mu_\delta, \mathbf{M}_\delta, \varepsilon)$ 
21:       $A' \sim \pi_\theta(\cdot | S')$ 
22:       $\delta \leftarrow R + \gamma [\hat{q}(S', A'; \bar{\mathbf{w}}) - \eta \log \pi_\theta(A' | S')] - \hat{q}(S, \text{sg}(A_\theta); \mathbf{w})$ 
23:    end if
24:     $\bar{\delta} \leftarrow \delta / \sigma_\delta$ 
25:     $\theta \leftarrow \theta + \alpha^\theta \nabla_\theta [\hat{q}(S, A_\theta; \mathbf{w}) - \eta \log \pi_\theta(A_\theta | S)]$ 
26:     $\mathbf{w} \leftarrow \mathbf{w} + \alpha^\mathbf{w} \bar{\delta} \nabla_\mathbf{w} \hat{q}(S, \text{sg}(A_\theta); \mathbf{w})$ 
27:     $\bar{\mathbf{w}} \leftarrow \tau \mathbf{w} + (1 - \tau) \bar{\mathbf{w}}$ 
28:     $S \leftarrow S'$ 
29:  end while
30: end for
```

A.5 Hyperparameters for Simulation Experiments

PPO Hyperparameters

Section 2.5 describes PPO, and Algorithm 2 gives its pseudocode. The experiments use the CleanRL implementation of PPO¹ with its default hyperparameters, listed in Table A.2.

Hyperparameter	Value
Rollout length (N)	2048
Minibatch size (M)	32
Advantage estimation parameter (λ)	0.95
Discount factor (γ)	0.99
Number of epochs (K)	10
Entropy coefficient	0
Actor step size (α^θ)	3×10^{-4}
Critic step size (α^w)	3×10^{-4}
Clipping parameter (ϵ)	0.1
Value loss coefficient	0.5
Maximum gradient norm	0.5

Table A.2: PPO hyperparameters for the simulation experiments, the defaults of the CleanRL implementation.

¹https://github.com/vwxyzjn/cleanrl/blob/8cbca61360ef98660f149e3d76762350ce613323/cleanrl/ppo_continuous_action.py

TD3 Hyperparameters

Section 2.6 describes TD3. The experiments use the CleanRL implementation of TD3² with its default hyperparameters, listed in Table A.3.

Hyperparameter	Value
Replay buffer size	1,000,000
Minibatch size (M)	256
Discount factor (γ)	0.99
Target policy noise (σ)	0.2
Exploration noise	0.1
Actor step size (α^θ)	3×10^{-4}
Critic step size (α^w)	3×10^{-4}
Policy update interval (updates)	2
Noise clipping bound (c)	0.5
Warm-up period (timesteps)	25,000
Polyak averaging coefficient (τ)	0.005

Table A.3: TD3 hyperparameters for the simulation experiments, the defaults of the CleanRL implementation.

²https://github.com/vwxyzjn/cleanrl/blob/8cbca61360ef98660f149e3d76762350ce613323/cleanrl/td3_continuous_action.py

SAC Hyperparameters

Section 2.6 describes SAC, and Algorithm 3 gives its pseudocode. Table A.4 lists the hyperparameters for the simulation experiments.

Hyperparameter	Value
Replay buffer size	1,000,000
Minibatch size (M)	256
Discount factor (γ)	0.99
Actor step size (α^θ)	3×10^{-4}
Critic step size (α^w)	3×10^{-4}
Entropy coefficient step size (α^κ)	3×10^{-4}
Actor update interval (timesteps)	1
Critic update interval (timesteps)	1
Target-critic update interval (timesteps)	1
Warm-up period (timesteps)	100
Polyak averaging coefficient (τ)	0.005
Target entropy ($\bar{\mathcal{H}}$)	$-\dim(\mathcal{A})$

Table A.4: SAC hyperparameters for the simulation experiments.

A.6 Hyperparameter Optimization Using Random Search

The random search procedure initializes a random number generator with a unique seed for each configuration to ensure reproducibility. The generator samples the actor step size, the critic step size, the Adam optimizer parameters, the entropy coefficient step size, and the discount factor from the ranges in Table A.5. For SAC and TD3, the Polyak averaging coefficient is fixed at 0.005.

Hyperparameter	Value or Range
Actor step size (α^θ)	$[10^{-6}, 10^{-2}]$, log-uniform
Critic step size (α^w)	$[10^{-6}, 10^{-2}]$, log-uniform
Entropy coefficient step size (α^κ)	$[10^{-5}, 10^0]$, log-uniform
Discount factor (γ)	$\{0.95, 0.97, 0.99, 0.995, 1.0\}$
Optimizer	Adam
Adam β_1	$\{0, 0.9\}$
Adam β_2	0.999
Polyak averaging coefficient (τ)	0.005 for SAC and TD3
Neural network activation	Leaky ReLU
Number of hidden layers	2
Number of hidden units per layer	256
Weight initialization	Orthogonal

Table A.5: Hyperparameter values and search ranges for the random search procedure.

A.7 AVG Hyperparameters Across Tasks

Table A.6 lists the best AVG hyperparameters found by the random search procedure of Appendix A.6 for each simulation task.

Task	α^θ	α^w	Adam (β_1, β_2)	α^κ	γ
Hopper-v4, Walker2d-v4	1.1×10^{-5}	7.7×10^{-5}	(0.0, 0.999)	0.3	0.99
Ant-v4, HalfCheetah-v4, Humanoid-v4	6.3×10^{-3}	8.7×10^{-3}	(0.0, 0.999)	0.07	0.99
reacher_hard	3×10^{-6}	4.9×10^{-3}	(0.0, 0.999)	0.05	0.97
dog_walk, dog_trot, dog_stand	6×10^{-6}	8×10^{-5}	(0.0, 0.999)	0.009	0.95
finger_spin	3.8×10^{-4}	8.7×10^{-5}	(0.9, 0.999)	0.006	0.95
dog_run	1.8×10^{-5}	4.8×10^{-5}	(0.0, 0.999)	0.007	0.97

Table A.6: Best AVG hyperparameters for each simulation task, found by random search.

A.8 The Effect of Other Network Normalization Techniques

Penultimate normalization, used by AVG, is one of several techniques for normalizing neural network features. Layer normalization normalizes the pre-activations across the features within each layer (Ba et al. 2016). Root mean square (RMS) normalization rescales the pre-activations by their root mean square without centering their mean (Zhang & Sennrich 2019). Penultimate normalization normalizes only the activations of the penultimate layer, immediately before the final layer (Bjorck et al. 2022).

Figure A.1 compares the three normalizations in AVG on Ant-v4, Hopper-v4, and Humanoid-v4. Agents using AVG with penultimate normalization consistently outperformed agents using AVG with layer normalization or RMS normalization.

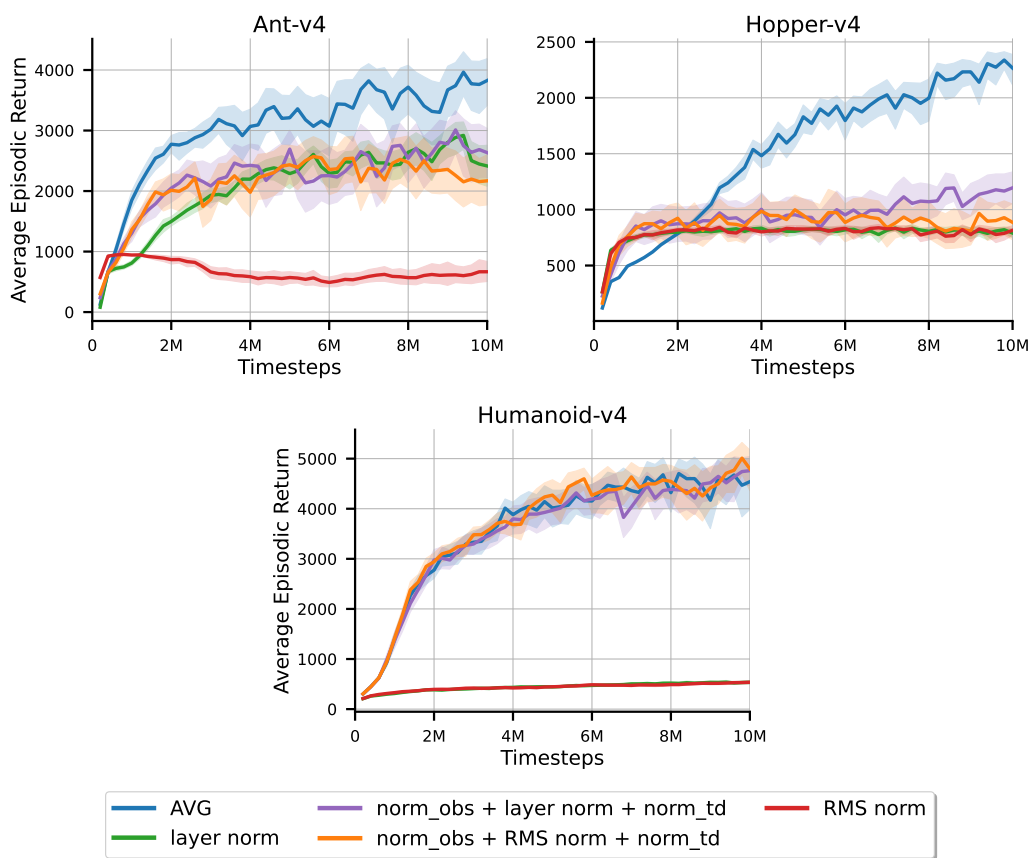


Figure A.1: Learning curves of AVG agents with penultimate normalization (pnorm), layer normalization (layer norm), and RMS normalization (RMS norm).

A.9 AVG Hyperparameters for Physical Robot Experiments

Table A.7 lists the hyperparameters of AVG and SAC-1, the SAC variant with a replay buffer and minibatch of size one, in the physical-robot experiments of Chapter 6.

Hyperparameter	AVG	SAC-1
Replay buffer size	N/A	1
Minibatch size (M)	N/A	1
Discount factor (γ)	0.95	0.99
Actor step size (α^θ)	3×10^{-4}	3×10^{-4}
Critic step size (α^w)	8.7×10^{-4}	3×10^{-4}
Entropy coefficient step size (α^κ)	N/A	3×10^{-4}
Actor update interval (timesteps)	1	1
Critic update interval (timesteps)	1	1
Target-critic update interval (timesteps)	N/A	1
Polyak averaging coefficient (τ)	N/A	0.005
Target entropy ($\overline{\mathcal{H}}$)	N/A	$-\dim(\mathcal{A})$
Entropy coefficient (η)	0.05, fixed	learned, $\eta \doteq \exp(\kappa)$
Optimizer	Adam	Adam

Table A.7: AVG and SAC-1 hyperparameters for the physical-robot experiments.